Vision embarquée par réseaux de neurones

FPGA, GPU embarqués, NPU et microcontrôleurs
État de l'art établi à partir du corpus PaperDatabase

Léonard Rivals

1er octobre 2026 — brouillon

Résumé

Ce document fait l'état de l'art de la vision embarquée par réseaux de neurones sur quatre familles de plateformes : SoC FPGA, GPU embarqués (Jetson), NPU et VPU en coprocesseur, et microcontrôleurs. Il couvre la détection d'objets, les autoencodeurs et l'apprentissage continu. Il s'appuie sur les 101 fiches de lecture et les 299 lignes de mesures de la base du 1er octobre 2026. Chaque chiffre porte sa source et chaque puissance sa portée. Il en ressort trois constats :

— chaque famille de plateformes a ses outils et ses métriques, et les surveys du corpus traitent soit l'embarqué sans FPGA, soit le FPGA sans comparaison chiffrée ;
— les rares comparaisons à tâche égale ne montrent pas d'avantage énergétique net du FPGA sur un GPU embarqué pour la détection ; la seule exception, un autoencodeur, compare des puissances de portées différentes, et un seul protocole récent mesure les deux cibles à la carte ;
— la précision est rarement remesurée sur la cible, alors que la quantification et la compilation la dégradent (de 6 à 12 points de mAP50-95 en INT8 sur Jetson, et une chute à zéro après conversion OpenVINO dans un cas).

Les zones non couvertes (anomalie visuelle RGB par autoencodeur sur FPGA, passerelle MCU–FPGA, apprentissage continu en détection) sont rassemblées en fin de document.

Table des matières

1 Introduction **3**

2 Méthode et conventions **3**

3 Plateformes de la vision embarquée **4**
3.1 Quatre familles de cibles 4
3.2 Ce que le corpus mesure, par famille 5
3.3 Les surveys : l'embarqué sans FPGA, ou le FPGA sans comparaison 5

4 Architectures d'accélérateurs CNN sur FPGA **6**
4.1 Le modèle roofline et l'exploration de l'espace de conception 6
4.2 Deux familles : moteur de calcul unique et architecture *streaming* 6
4.3 La mémoire on-chip, goulot de l'architecture streaming 7
4.4 Quantification et arithmétique : ce que le DSP y gagne 7

5 Toolflows : du réseau entraîné au bitstream **8**
5.1 FINN et FINN-R : réseaux binaires puis quantifiés 8

5.2 fpgaConvNet : partitionner ou recharger les poids . . . 9
5.3 hls4ml : la très faible latence, venue de la physique des particules . . . 10
5.4 QONNX et MLPerf Tiny : un format commun, une première comparaison . . . 10
5.5 SATAY : un toolflow streaming pour YOLO . . . 10
5.6 Vitis AI et la DPU : le flot par défaut, contraint par ses opérateurs . . . 11
5.7 Co-design et NAS guidés par le matériel . . . 11
5.8 Synthèse . . . 12

6 Détection d'objets sur FPGA 12
6.1 Trajectoire du corpus . . . 12
6.2 Tableau des détecteurs FPGA du corpus . . . 15
6.3 Post-traitement et interaction PS–PL . . . 17
6.4 Détection de personnes . . . 17

7 Détection d'objets hors FPGA : GPU embarqués, NPU et microcontrôleurs 17
7.1 GPU embarqués et NPU . . . 17
7.2 Microcontrôleurs . . . 19
7.3 Tableau des détecteurs hors FPGA . . . 19

8 Comparer les plateformes : ce que le corpus permet, et ce qu'il ne permet pas 21
8.1 Débit et puissance rapportés . . . 21
8.2 Les comparaisons à tâche égale . . . 21
8.3 Énergie par inférence : des ordres de grandeur . . . 22
8.4 Pourquoi la comparaison reste ouverte . . . 24

9 Autoencodeurs sur FPGA 24
9.1 Physique des particules et signaux : la très faible latence . . . 24
9.2 Images : compression apprise et encodeurs . . . 25
9.3 Détection d'anomalies sur SoC FPGA : surtout des signaux . . . 25
9.4 Bilan . . . 26

10 Apprentissage embarqué et continu sur FPGA 28

11 Apprentissage continu sur microcontrôleurs et ASIC 30

12 Analyse critique : ce que les chiffres publiés permettent de conclure 31
12.1 La puissance : rarement mesurée sur la carte, presque jamais sur la puce FPGA . . . 31
12.2 Des métriques qui flattent . . . 32
12.3 La précision est rarement remesurée sur la cible . . . 33
12.4 Baselines GPU embarqué faibles ou absentes . . . 34
12.5 Reproductibilité . . . 34

13 Zones non couvertes et pistes de recherche 34
13.1 Quatre questions de thèse candidates . . . 35

13.2 Gaps entre plateformes 38

14 Conclusion **38**

Glossaires **39**

Glossaire des plateformes **39**

Glossaire des modèles **42**

Glossaire des métriques et unités **47**

1 Introduction

La vision embarquée demande d'exécuter un réseau de neurones au plus près du capteur, sous des contraintes de latence, de puissance et de mémoire. Plusieurs familles de plateformes répondent à ce besoin :

- le GPU embarqué, programmable et outillé (TensorRT) ;
- le NPU ou le VPU en coprocesseur, qui accélère des modèles INT8 ;
- le microcontrôleur, éventuellement doté d'un accélérateur, pour les budgets de quelques milliwatts ;
- le SoC FPGA, qui permet un accélérateur taillé pour le réseau, au prix d'une chaîne de conception plus lourde.

Ce document répond à trois questions, à partir du corpus seulement :

- quelles plateformes et quels outils existent, et que disent les surveys (section 3) ?
- où en sont la détection d'objets, les autoencodeurs et l'apprentissage continu sur chacune (sections 4 à 11) ?
- que permettent de conclure les chiffres publiés, et que reste-t-il à faire (sections 8, 12 et 13) ?

Le corpus a été constitué pour une thèse sur la vision embarquée sur FPGA : le FPGA y est donc surreprésenté (101 lignes de mesures sur 172). Une version centrée sur le FPGA existe à part.

2 Méthode et conventions

Corpus. Ce document ne s'appuie que sur la base PaperDatabase, dans son état du 1er octobre 2026 : 101 articles, tous fichés, et 299 lignes de mesures dans `benchmarks.csv`, issues de 54 articles. Parmi ces lignes, 216 sont mesurées sur FPGA (44 articles) et 83 sur d'autres plateformes (GPU Jetson, NPU, VPU, CPU, microcontrôleurs, ASIC ; 15 articles). Les 47 articles sans ligne sont surtout des surveys et des travaux de sûreté, de vérification et de certification (37 sur 47, guides de l'EASA compris), dont les résultats ne tiennent pas dans les colonnes du fichier. Tous les comptes du document sont faits sur cet état. Aucune référence extérieure à la base n'est citée comme source.

Origine des chiffres. Chaque valeur est reprise de la fiche ou de `benchmarks.csv`, avec la section ou le tableau de l'article entre crochets, par exemple [Table 8]. Une valeur que les auteurs ne donnent pas s'écrit `n/r` : elle n'est jamais estimée. Les chiffres d'un survey sont ceux des travaux qu'il cite. Ils ne sont pas vérifiés à la source et servent à orienter la lecture, pas à comparer. La colonne `confidence` vaut *high* pour 158 lignes, *medium* pour 140 (dont 101 des 216 lignes FPGA) et *low* pour une seule. Les lignes *medium* viennent de tableaux reconstruits automatiquement (Docling, PyMuPDF) ou de valeurs lues dans le texte qui commente une figure ; elles sont signalées par ^{m} dans les tableaux. La ligne *low* (un autoencodeur de débruitage, arxiv-2301.07050^{PP}) est signalée dans le texte.

Portée de la puissance. Une puissance n'a de sens qu'avec sa portée. La base distingue *board* (carte entière, au wattmètre), *chip* (rails de la puce ou télémétrie embarquée), *board-dynamic* (carte, repos soustrait), *estimated* (outil de conception, par exemple le rapport de Vivado) et *unspecified* (méthode non décrite). Sur les 299 lignes, 49 sont *board*, 18 *chip*, 17 *estimated*, 73 *unspecified*, aucune *board-dynamic*, et 142 n'ont pas de puissance. Les tableaux donnent cette portée à côté de chaque valeur. Deux puissances de portées différentes ne se comparent pas.

Lu contre déduit. Sauf mention, une affirmation résume ce que disent les auteurs. Une inférence de la fiche ou de ce document est marquée *(déduit)*. Un article marqué [PP] est un preprint non relu par les pairs (au moins dans la version ingérée).

Limites de la méthode. Le corpus reste petit (101 articles) et biaisé par sa construction. Les 41 premières fiches (24 et 26 septembre 2026) viennent d'une bibliographie de départ constituée à partir de résumés, centrée sur les plateformes, les toolflows et les autoencodeurs. Les 60 suivantes (29 septembre) ont été ajoutées avec le thème de thèse : le volet sûreté et certification (29 articles) et la détection de personnes (5), ainsi que des détecteurs sur FPGA et des travaux d'apprentissage continu. Les effectifs par axe (un article peut en porter plusieurs) restent déséquilibrés : 44 articles sur les plateformes, 29 sur la sûreté et la certification, 23 sur les toolflows, 18 sur YOLO sur FPGA, 18 surveys, 15 sur les autoencodeurs, 15 sur l'apprentissage continu (dont 7 sans FPGA) et 5 sur la détection de personnes. `benchmarks.csv` est lui aussi déséquilibré : cinq articles fournissent 90 des 299 lignes, et les 29 fiches de sûreté n'en donnent que 4, faute de colonnes pour les métriques de fiabilité. Enfin, 34 des 101 articles sont des preprints. Un « gap » signifie donc qu'un sujet est absent *de ce corpus*. Son absence de la littérature reste une hypothèse tant qu'une recherche ciblée ne l'a pas confirmée. Les fiches ont été rédigées à partir du texte extrait des PDF : les valeurs données uniquement dans des figures sont souvent `n/r`, et les liens en note de bas de page sont souvent perdus.

3 Plateformes de la vision embarquée

3.1 Quatre familles de cibles

Le corpus couvre quatre familles de plateformes, avec des outils et des métriques propres :

- **SoC FPGA** (Zynq-7000, Zynq UltraScale+, Kria) : un accélérateur taillé pour le réseau, en précision arbitraire. Outils : Vitis AI (moteur unique, INT8) ou flots dataflow (FINN, hls4ml, SATAY). Voir les sections 4 et 5.
- **GPU embarqués** (Jetson Nano, TX2, Xavier NX, Orin Nano et NX) : un accélérateur généraliste. Outil : TensorRT, en FP32, FP16 ou INT8. Selon la table de Luo et al. (2024, Table 4), une Jetson AGX Orin annonce 15 à 40 W et 275 TOPS (données constructeur).
- **NPU et VPU** en coprocesseur : Intel NCS2 et Myriad 2 (OpenVINO), Coral Edge TPU (INT8), Hailo-8 (HailoRT).
- **Microcontrôleurs**, avec ou sans accélérateur (STM32, GAP9, MAX78000) : moins de 1 Mo de SRAM, un budget de quelques mW à quelques dizaines de mW. El Zeinaty et al. (2025, Table 4) en recensent huit, dont quatre à accélérateur IA (STM32N6, GAP9, K210, MAX78000).

Les ordres de grandeur de puissance vont donc de quelques dizaines de milliwatts (VEGA : 62 mW, Ravaglia et al., 2021, §V-D) à une quinzaine de watts en télémétrie de module (Jetson Orin NX : 10 à 14 W, Rey et al., 2025, Table 4), et à plus de 20 W mesurés à l'entrée d'une carte de développement (ZCU104 : 23 W ; AGX Xavier : 16,5 à 28 W selon la précision, Schlosser et al., 2023, Table 1). Côté FPGA, la puissance de carte la plus basse pour un détecteur est celle d'un Zynq XC7Z020 en dataflow FINN : 2,0 à 2,9 W (Gutierrez et al., 2026, §V)[PP]. Ces chiffres n'ont pas la même portée et ne se comparent pas directement (section 12).

3.2 Ce que le corpus mesure, par famille

Un recompte de `benchmarks.csv` (1er octobre 2026, classement par mots-clés de la plateforme) donne 152 lignes de détection d'objets : 61 sur petite FPGA ou SoC (Zynq, Kria, Ultra96, PYNQ), 26 sur grosse FPGA (Virtex, Kintex, Alveo), 47 sur Jetson, 5 sur MCU et 13 sur d'autres cibles (CPU, GPU de bureau, Raspberry Pi, VPU, Hailo). Les familles se croisent peu :

- quatre articles seulement ont des lignes à la fois FPGA et Jetson : SkyNet sur TX2 (Zhang et al., 2019)[PP], Schlosser et al. (2023) sur AGX Xavier, Castelino et al. (2024) sur Xavier NX et Aggarwal et al. (2024) sur Nano ; deux seulement portent sur la détection d'objets (section 7) ;
- aucun ne compare une FPGA à une Jetson Orin : les lignes Orin viennent toutes de Rey et al. (2025) et Alqahtani et al. (2024)[PP] ;
- aucun n'a de lignes à la fois FPGA et MCU.

Le seul protocole de mesure d'énergie normalisé appliqué à des FPGA dans le corpus est celui de MLPerf Tiny (EEMBC EnergyRunner, Joulescope), utilisé par Borras et al. (2022)[PP] sur Pynq-Z2 et Arty A7 [§4.4]. Il porte sur la classification CIFAR-10 et sur des tâches audio, pas sur la détection, et l'article ne compare à aucune autre plateforme [§1].

3.3 Les surveys : l'embarqué sans FPGA, ou le FPGA sans comparaison

L'axe `surveys` compte dix-huit articles. Les huit qui traitent du matériel se répartissent en deux groupes qui ne se recoupent pas ; les dix autres portent sur la sûreté ou sur une application, et traitent le matériel au mieux en passant.

Surveys « embarqué ».

- Luo et al. (2024) couvrent la conception manuelle, la NAS guidée par le matériel, la compression, l'apprentissage sur l'appareil et les LLM. Leur matériel se limite aux Jetson, TPU et NPU, sans aucun FPGA [Table 4]. Ils rappellent que les FLOPs ne prédisent pas la latence réelle [§2.3] et que l'entraînement embarqué est limité par la mémoire des activations, pas par les paramètres [§5.1].
- Shuvo et al. (2023) couvrent toute la chaîne de l'inférence sur l'edge (Proc. IEEE). Le FPGA y occupe une sous-section [§XI-C1], et la comparaison entre plateformes reste qualitative.
- Mittal (2024) recense 26 détecteurs légers. Le FPGA n'a qu'un paragraphe [§4.3.3], et les FPS des Tables 5–6 ne sont associés à aucune plateforme, ce qui les rend inexploitables pour une comparaison matérielle.
- El Zeinaty et al. (2025) traitent la détection sur MCU. Le FPGA n'y est évoqué qu'en passant [§6.1.3, §8].

Surveys FPGA. Hozhabr and Giorgi (2025), Sali et al. (2025)[PP], Abdelouahab et al. (2018)[PP] et Venieris et al. (2018) ne comparent pas quantitativement les FPGA aux Jetson ou aux MCU. Hozhabr et Giorgi le font de façon qualitative [Table 5] ; les gains face au GPU qu'ils citent sont ceux des travaux d'origine, souvent obtenus contre des GPU de bureau.

Surveys de sûreté et d'application. Les dix autres surveys ne comblent pas ce manque :

- sûreté et fiabilité : Ruospo et al. (2023) ne citent que de petits réseaux en simulation matérielle et peu de travaux FPGA ; Rech (2024) est centré sur le GPU et traite le FPGA brièvement et qualitativement [§IV-C.1] ; Huang et al. (2018)[PP], Schwalbe and Schels (2020) et Ashmore et al. (2019)[PP] n'ont aucune cible matérielle ;
- applications : anomalie visuelle industrielle (Cui et al., 2023), apprentissage incrémental (Masana et al., 2020)[PP], ré-identification de personnes (Ye et al., 2020)[PP], détection probabiliste (Feng et al., 2020) et vision pour la collaboration humain–robot (Halme et al., 2018).

Aucun ne mesure sur une cible embarquée.

Aucun survey du corpus ne propose donc une synthèse quantitative FPGA / Jetson / MCU à NPU sur des critères communs. La section 8 en esquisse une à partir de `benchmarks.csv`, et montre pourquoi elle reste fragile.

4 Architectures d'accélérateurs CNN sur FPGA

4.1 Le modèle roofline et l'exploration de l'espace de conception

Zhang et al. (2015) posent le problème qui structure la suite : à ressources égales, deux accélérateurs peuvent différer de 90 % en performance, parce que le débit de calcul ne correspond pas à la bande passante mémoire [§1]. Ils appliquent le modèle *roofline* : la performance atteignable est le minimum entre le toit de calcul et le produit du ratio calcul/communication par la bande passante [§2.3, Éq. 1]. Le tuilage des boucles de la convolution fixe ces deux grandeurs, et une énumération exhaustive (au plus 98 000 designs légaux, explorés en 10 minutes) choisit la meilleure configuration [§3.4–3.5]. Des facteurs de déroulage uniformes pour toutes les couches restent à moins de 5 % de l'optimum par couche et évitent de reprogrammer le FPGA entre couches [Table 4].

Le résultat, 61,62 GFLOPS sur les convolutions d'AlexNet en FP32 sur un Virtex-7 VX485T pour 18,61 W mesurés sur la carte [Tables 7–8], est daté : calcul flottant, et seules les convolutions sont accélérées (ni pooling, ni couches denses, [§2.1]). Le gain énergétique de 24,6× sur le CPU repose sur le TDP du Xeon et non sur une mesure [§5.2, Table 8]. La méthode, elle, reste la référence du dimensionnement d'un accélérateur.

4.2 Deux familles : moteur de calcul unique et architecture *streaming*

Les deux surveys de 2018 du corpus proposent des taxonomies complémentaires. Abdelouahab et al. (2018)[PP] classent les optimisations en trois familles [§2.4.4, Fig. 2] :

— les *transformations de calcul* : GEMM, Winograd (2,25× moins de multiplications pour $F(2\times2, 3\times3)$), FFT [§3] ;
— l'*optimisation du chemin de données* : réseaux systoliques, SIMD, déroulage et tuilage des boucles, dataflow [§4] ;
— le *calcul approché* : virgule fixe, réseaux binaires ou ternaires, élagage, rang faible [§5].

Ils rappellent que les convolutions représentent plus de 90 % du temps de calcul et les couches denses plus de 90 % des poids [§2.4.1, Table 2].

Venieris et al. (2018) recensent 13 toolflows et les séparent selon leur architecture [§2.3] :

— **streaming** (un bloc matériel par couche, en pipeline) : fpgaConvNet, DeepBurning, Haddoc2, AutoCodeGen, FINN. Ces designs sont efficaces, mais il faut un bitstream par réseau, et la taille du réseau est bornée par la mémoire on-chip ;
— **moteur de calcul unique**, piloté par logiciel (réseau systolique ou produit matriciel) : Angel-Eye, ALAMO, DnnWeaver, Caffeine, FP-DNN, Snowflake, SysArrayAccel, FFTCodeGen. Un seul bitstream sert plusieurs réseaux.

Leurs enseignements [§2.7] : le RTL optimisé bat le HLS, une exploration plus fine donne de meilleurs designs, et les moteurs uniques profitent des réseaux réguliers (VGG16) mais souffrent des noyaux hétérogènes (AlexNet). Ils proposent aussi une grille d'évaluation [§3.1] : débit en GOp/s avec le nombre d'opérations, les poids et le batch, latence à batch 1, ressources, puissance, précision applicative et répartition mémoire on-chip/off-chip. Les métriques normalisées (GOp/s par DSP ou par kLUT) ne valent, selon eux, qu'à l'intérieur d'une même famille de FPGA.

Deux réserves sur ce survey : ses chiffres sont repris des publications ou des auteurs, sans qu'aucun outil soit relancé [§2.6], et ses tableaux comparatifs ne contiennent ni puissance ni précision. Les auteurs du survey sont aussi ceux de fpgaConvNet, ce qui invite à lire avec recul les comparaisons qui le concernent *(déduit)*.

Sept ans plus tard, le survey de Hozhabr and Giorgi (2025), consacré à la détection temps réel sur FPGA, reprend la même dichotomie, moteur unique ou streaming [§IV-A]. Il propose de normaliser le débit par la résolution (*pixel throughput*, FPS × pixels par image, [§II-C2, Éq. 2]), ce qui ne règle ni la métrique de précision ni la portée de la puissance. Ses études de cas de détection n'emploient aucun toolflow dataflow quantifié (FINN, hls4ml) : ce sont des DPU Vitis AI et des accélérateurs HLS maison [Annexe B]. La dichotomie de 2018 tient donc toujours, mais la détection sur FPGA s'est surtout développée du côté du moteur unique.

4.3 La mémoire on-chip, goulot de l'architecture streaming

Une architecture streaming garde les poids en mémoire on-chip. Sur une petite puce, c'est la BRAM qui limite. L'ordre de grandeur est donné par Gutierrez et al. (2026)[PP] : un YOLOv3-tiny complet demanderait environ 34,7 Mbit de poids à 4 bits, pour environ 4,9 Mbit de BRAM sur le XC7Z020 [§III-A]. Il faut donc d'abord réduire le réseau (section 5). Petrica et al. (2020)[PP], de l'équipe FINN, en donnent la cause : la forme des mémoires de poids, dictée par le parallélisme (PE × SIMD), s'accorde mal aux blocs de 18 Kb, dont une grande partie reste vide [§II-B]. Le rendement d'occupation des BRAM tombe à 52,9 % pour un ResNet-50 quantifié sur Alveo U250 [Table IV]. Leur technique, FCMP, sur-cadence la mémoire par rapport au calcul et regroupe plusieurs tampons de poids par BRAM :

- jusqu'à 30 % de mémoire on-chip en moins, pour environ 5 % de LUT en plus [§I] ;
- le classifieur CNV-W1A1 (CIFAR-10) passe du Zynq 7020 au 7012S sans perte de débit [Table V] ;
- ResNet-50 (W1A2, 67,3 % de top-1) tourne en dataflow sur une seule puce, mais sur une Alveo U250 : 2 703 FPS, 1,9 ms de latence, 71 W dont la méthode de mesure n'est pas décrite [Table II].

L'empaquetage des activations (FIFO, tampons de lignes) reste en travail futur [§VI].

D'autres réponses au même problème apparaissent dans le corpus :

- le rechargement des poids de fpgaConvNet (section 5) ;
- la compression structurée de REQ-YOLO, qui fait tenir tous les poids en BRAM (section 6) ;
- la FIFO logicielle de SATAY, qui déporte en DDR les plus gros tampons des connexions résiduelles (section 5) ;
- l'éviction d'activations et la fragmentation des poids vers la DDR de SMOF, qui généralisent l'idée de SATAY dans fpgaConvNet (Toupas et al., 2024)[PP] (section 5) ;
- le partitionnement d'un accélérateur FINN sur plusieurs SLR ou plusieurs FPGA, d'Elastic-DF (Alonso et al., 2021), qui suppose un cluster d'Alveo reliés en 100 GbE [§5.1].

Les deux dernières ont un prix que l'architecture « tout on-chip » n'a pas : le débit de SMOF dépend du taux de compression des activations, donc de l'image (Toupas et al., 2024, Fig. 8), et Elastic-DF vise le datacenter. Aucune ne mesure de puissance sur SoC embarqué.

Le moteur unique n'échappe pas à la contrainte. Sur Kria KV260, pour éviter la DDR, Fata and Elmannai (2026) placent un contrôleur mémoire RTL entre la DPU et la mémoire (couches 1 à 10 en BRAM, 11 à 21 en URAM, repli en DDR) et modifient le fichier `arch.json` du compilateur Vitis AI [§IV-E–F]. Le design occupe 100 % des BRAM et des URAM [Table 9], et le contournement n'est décrit que qualitativement. Côté MCU, le même verrou existe pour la SRAM des activations : Lin et al. (2021) le contournent par une inférence par patchs, qui divise le pic mémoire par 4 à 6 sur un STM32F746 [Fig. 6]. Transposer cet ordonnancement par patchs à la BRAM d'un accélérateur streaming n'est pas traité dans le corpus *(déduit)*.

4.4 Quantification et arithmétique : ce que le DSP y gagne

La quantification ne réduit pas seulement la mémoire : elle change le rendement des DSP. Trois accélérateurs YOLO à moteur unique, écrits en HLS ou en RTL, illustrent l'écart :

— Yan et al. (2024) quantifient tout YOLOv5s en 4 bits et empaquettent quatre multiplications 4 bits signées par DSP48, avec une correction précalculée [§4–5]. Sur ZCU102, ils obtiennent 808,6 GOPS et 0,49 GOPS/DSP [§7.4, Table 3], pour une mAP VOC en baisse de 1,5 point [Table 2, alignement de la ligne à vérifier] ;
— Zhang et al. (2024) empaquettent deux multiplications 8 bits par DSP grâce au pré-additionneur [§3.1], et choisissent leur ordonnancement (réutilisation des cartes d'activation plutôt que des poids) avec le modèle roofline [§4.4] : la méthode de Zhang et al. (2015) sert toujours. Ils atteignent 87 % du débit d'un design de référence avec 29,8 % de ses DSP [§5.3] ;
— à l'opposé, Kim and Kim (2025), en INT16 avec un MAC par DSP sur XC7Z020, restent à 2,33 GOPS et 0,018 GOPS/DSP, avec environ 12,6 s par image [§3.1, §4.2, Table 9 reconstruite par Docling].

Le rapport entre le premier et le dernier dépasse 25 en GOPS/DSP *(déduit)*. Ces métriques normalisées ne comparent toutefois que des puces d'une même famille (Venieris et al., 2018, §3.1), et la puissance de 12,3 W de Yan et al. (2024) n'a pas de méthode de mesure décrite [§7.4]. Côté streaming, la même logique pousse vers les LUT plutôt que les DSP : accumulation par popcount de FINN, ou réduction de 6 377 à 174 DSP par QAT et élagage dans hls4ml (section 5). L'empaquetage de plusieurs produits dans un DSP ajoute une correction arithmétique qu'il faudra vérifier bit à bit dans une chaîne de preuve *(déduit)*.

5 Toolflows : du réseau entraîné au bitstream

Un toolflow traduit un réseau décrit dans un framework d'apprentissage en une architecture matérielle, sous les contraintes de ressources d'une puce donnée (Venieris et al., 2018, §1). Le corpus couvre quatre lignées dataflow (FINN, fpgaConvNet, hls4ml, SATAY), leurs extensions pour les réseaux qui débordent de la puce (SMOF, Elastic-DF), un format d'échange commun (QONNX) et le flot industriel d'AMD, Vitis AI, qui exécute le réseau sur un moteur unique (la DPU).

5.1 FINN et FINN-R : réseaux binaires puis quantifiés

Umuroglu et al. (2016) partent d'un constat : sur une Zynq UltraScale+ ZU19EG, le pic binaire (66 TOPS) vaut environ 16 fois le pic 8 bits, et un AlexNet binarisé tiendrait on-chip [§3.1]. FINN génère une architecture streaming, un moteur par couche, tous les poids en mémoire on-chip [§4.1]. Trois idées de ce papier sont reprises partout ensuite :

— l'accumulation par *popcount*, qui coûte à peu près moitié moins de LUT qu'une accumulation signée [§4.2.1] ;
— le remplacement de la batch normalization suivie du signe par un simple seuil [§4.2.2] ;
— le *folding* (P PE × S voies SIMD par couche), qui équilibre les débits des couches [§4.4].

La mesure est soignée : puissance de la puce (P_{chip}, lue par PMBus) et de la carte (P_{wall}, au wattmètre), moyennées sur 10 s [§5.1]. Les réseaux sont petits : MLP sur MNIST, et un CNN (CNV) sur CIFAR-10 à 80,1 % de précision. CNV-max atteint 21 900 images/s pour 3,6 W de puce et 11,7 W de carte sur une ZC706 [Tables 3–4]. La carte consomme environ 7 W au repos [§5.2.2] : un FPS/W calculé sur la carte ne se transpose donc pas à une carte plus sobre *(déduit)*.

Blott et al. (2018) étendent FINN aux précisions multiples. FINN-R est structuré comme un compilateur, avec une représentation intermédiaire qui connaît la quantification de chaque nœud [§4]. Il apporte :

— des modèles de coût : les LUT d'un produit scalaire sont proportionnelles au produit (nombre d'éléments × bits de poids × bits d'activation), et le modèle prédit la synthèse à ±30 % près [§3.1–3.2] ;
— deux architectures : *dataflow* (un moteur par couche) et *multilayer offload* (MO), qui réutilise une partie du pipeline en boucle avec les cartes d'activation en mémoire externe, pour les réseaux trop gros [§3.3].

Sur petite cible, le MO fonctionne mal. Tincy YOLO (W1A3) n'atteint que 133 GOp/s sur PYNQ-Z1 et 288 GOp/s sur Ultra96, avec une prédiction juste à 66–68 % seulement [Table 5]. Le papier ne donne aucun FPS pour ce détecteur. L'outil ne gère que les ressources et le débit : latence et puissance ne sont pas modélisées [§4].

FINN sur un détecteur et une petite puce. Gutierrez et al. (2026)[PP] sont les seuls du corpus à faire tourner un détecteur en dataflow FINN sur un Zynq-7020 (PYNQ-Z1). Un YOLOv3-tiny complet ne tenant pas en BRAM (section 4), ils l'allègent (LP-YOLO, environ 0,35 M paramètres) et le quantifient par QAT avec Brevitas, puis compilent via QONNX [§III-A, §IV]. Trois constats :

- la configuration retenue, w2a4 en 256×256 à 142,86 MHz, fait 35,66 FPS pour 2,91 W mesurés sur le rail 12 V de la carte, avec 0,594 de mAP@0,5 sur les classes de véhicules de VOC [§V-D, §VI, Table II] ;
- sur le YOLOv3-tiny de référence en 4 bits, la QAT reste à environ 0,006 de mAP du flottant, la PTQ en perd environ 0,17 [§IV-B] ;
- l'estimation analytique de FINN annonçait 38 FPS pour un folding qui en mesure environ 22, et le dimensionnement automatique des FIFO porte la mesure à 35 FPS [§III-B, §V-A–B].

Les ressources ne sont pas chiffrées (tableau absent de l'extraction), le débit est mesuré sur un lot de 100 images, et la tâche (véhicules, VOC filtré) ne se compare pas à une mAP COCO à 80 classes. Le résultat montre néanmoins que le dataflow quantifié sur petite puce existe pour la détection, au prix d'un réseau taillé pour la BRAM.

Elastic-DF : FINN au-delà d'une puce. Alonso et al. (2021) partitionnent un accélérateur FINN déjà replié sur plusieurs SLR ou plusieurs Alveo, par programmation linéaire en nombres entiers sous contraintes de congestion [§4]. Le partitionnement relève la fréquence maximale de 20 % en moyenne face aux builds monolithiques [Table 5], et deux U280 divisent presque par deux la latence noyau de ResNet-50 (1,52 à 1,55 ms contre 2,72 ms) [Table 4]. Les puissances (100 à 134 W pour deux cartes, [Table 4]) situent ce travail hors de l'embarqué : il montre la limite de l'approche « tout on-chip » plus qu'il n'offre une solution pour un SoC.

5.2 fpgaConvNet : partitionner ou recharger les poids

Venieris and Bouganis (2017) modélisent le réseau en *synchronous dataflow* et explorent quatre transformations [§2.4] :

- le partitionnement avec reconfiguration du FPGA, amortie par lots, qui vise le débit ;
- le repli à gros grain (nombre d'unités par couche) ;
- le repli à grain fin (parallélisme du produit scalaire) ;
- le rechargement des poids sur une architecture unique, sans reconfiguration, qui vise la latence.

C'est la lignée du corpus conçue d'emblée pour des réseaux qui ne tiennent pas on-chip (FINN n'y vient qu'avec le MO de FINN-R et le multi-FPGA d'Elastic-DF). Elle calcule en virgule fixe 16 bits et ne descend pas plus bas ; FINN-R le relève (Blott et al., 2018, §2.2). La comparaison au Tegra X1 est l'une des rares du corpus à budget de puissance égal (5 W) et avec la même méthode de mesure des deux côtés : jusqu'à 5,53× le débit du GPU, et jusqu'à 6,65× en mode latence [§3.1]. Deux réserves : le GPU est bridé à 76,8 MHz, et les valeurs absolues ne figurent que dans une figure. Le papier, un article de workshop, ne donne ni ressources ni précision.

SMOF : évincer vers la DDR. Toupas et al. (2024)[PP] ajoutent à fpgaConvNet trois leviers pour les réseaux à longues connexions résiduelles (UNet, YOLO, X3D) : éviction des activations vers la DDR par DMA, chargement dynamique d'une fraction des poids, et partitionnement avec reconfiguration ; les flux DMA peuvent être compressés sans perte [§III, §IV-A]. Les poids passent en

block floating-point 8 bits sans fine-tuning [§V-A] : YOLOv8n perd 2 points de mAP (35,10 contre 37,10 en flottant, [Table III]). Les résultats appellent trois réserves :

— le gain est modeste à batch 1 : jusqu'à 1,3× sur UNet, 1,1× sur UNet3D [Fig. 6] ;
— la reconfiguration pèse 31 % de la latence à batch 1 sur UNet3D [Table IV], et le débit dépend du taux de compression, donc de l'image [Fig. 8] ;
— YOLOv8n atteint 184 FPS, mais sur VCU118, pas sur un SoC embarqué [Table V, reconstruite par Docling]. Les 23× de FPS revendiqués face à SATAY comparent YOLOv8n à YOLOv8s, et l'efficacité par DSP est légèrement inférieure (0,16 contre 0,18 GOP/s/DSP, [§V-D]).

Aucune puissance n'est mesurée. Le code est publié [Résumé].

5.3 hls4ml : la très faible latence, venue de la physique des particules

hls4ml est né pour le déclenchement de premier niveau du LHC, avec une latence de l'ordre de la microseconde. Chaque couche y devient un bloc matériel dont le parallélisme se règle par le *reuse factor* (Fahim et al., 2021, §4.2). Le volet CNN (Aarrestad et al., 2021) traite la convolution en flux, avec une fenêtre glissante [§3]. Sur un petit classifieur SVHN (12 858 poids) :

— en quantification post-entraînement (PTQ), la précision s'effondre sous 14 bits ;
— en quantification pendant l'entraînement (QAT, avec QKeras), elle tient jusqu'à 3 à 4 bits [§7–8] ;
— le modèle QAT élagué sur 7 bits prend 174 DSP au lieu de 6 377 sur VU9P, pour une latence d'environ 5 µs [Table 3] ;
— sur XC7Z020, il atteint 2 831 images/s pour 171 µs de latence [Table 4].

Aucune puissance FPGA n'est mesurée : l'énergie citée est une estimation ASIC 45 nm.

La version 2025 (Schulte et al., 2025)[PP] ajoute l'arithmétique distribuée sans DSP et la quantification par paramètre (HGQ). Elle s'évalue en placement-routage hors contexte, sur de petits modèles, sans puissance, et ses auteurs reconnaissent que les gros modèles échouent en synthèse [§9, §11]. Campos et al. (2024) relient la quantification mixte guidée par la Hessienne (HAWQ, largeurs choisies par programmation linéaire en nombres entiers sous budget de BOPs) à hls4ml via QONNX. Sur un MLP, ils obtiennent −95,7 % de DSP pour −1,07 point de précision [Table 2]. Mais les latences sont estimées par Vivado HLS, aucun design ne tourne sur carte [§6.2], et la puce cible est ambiguë (voir la fiche). Enfin, Govorkova et al. (2026) ajoutent un backend pour les FPGA flash durcis Microchip PolarFire, limité aux couches denses et ReLU [§5.1].

5.4 QONNX et MLPerf Tiny : un format commun, une première comparaison

Pappalardo et al. (2022)[PP] partent du constat que les formats ONNX quantifiés existants s'arrêtent à 8 bits [§III]. QONNX ajoute trois opérateurs (Quant, BipolarQuant, Trunc) qui décrivent une quantification uniforme de largeur arbitraire, par tenseur ou par canal [§V, Table II]. Brevitas et QKeras exportent vers QONNX ; FINN et hls4ml l'importent [§VI]. L'ingestion reste partielle : FINN ne gère que ReLU, hardtanh et l'identité comme activations [§VI-D], ce qui exclut SiLU et LeakyReLU, présents dans la plupart des YOLO *(déduit)*. Le papier est une spécification, sans mesure, et la représentation entière est laissée à chaque outil [§V] : rien ne garantit que FINN et hls4ml arrondissent et saturent de la même façon *(déduit)*.

Borras et al. (2022)[PP] fournissent la seule comparaison mesurée de deux toolflows FPGA du corpus. Pour MLPerf Tiny, hls4ml et FINN sont déployés sur les mêmes cartes (Pynq-Z2 et Arty A7-100T) et mesurés avec le même harnais EEMBC, énergie au Joulescope [§4.4]. Sur la classification CIFAR-10 et la Pynq-Z2 [Table 5] :

— hls4ml : 27,3 ms et 44 330 µJ par inférence (28 544 LUT, 42 BRAM, 4 DSP) ;
— FINN : 1,5 ms et 2 535 µJ (24 502 LUT, 100 BRAM, 0 DSP).

FINN est 18,2 fois plus rapide, hls4ml prend 58 % de BRAM en moins [§4.2.2]. Mais les modèles diffèrent : un ResNet-8 trouvé par NAS bayésienne, 58 k paramètres sur 8 à 12 bits, à 83,5 % contre

87,0 % pour la référence [§3.1], face à un CNV binaire de 1,5 M paramètres. La comparaison mesure donc deux couples modèle-flot, pas deux flots. C'est aussi la seule mesure d'énergie de hls4ml dans le corpus, sans puissance en watts et avec une portée (carte ou puce) non précisée. Aucune tâche n'est une détection, et Vitis AI est absent.

5.5 SATAY : un toolflow streaming pour YOLO

Montgomerie-Corcoran et al. (2023) sont les premiers du corpus à automatiser une architecture streaming pour la détection (YOLOv3-tiny, YOLOv5, YOLOv8), de l'ONNX au bitstream [§IV]. Leurs réponses aux opérations propres à YOLO :

— HardSwish en remplacement de SiLU [§III-B] ;
— redimensionnement (*resize*) calculé à la volée [§III-B] ;
— FIFO logicielle en DDR pour les longues connexions résiduelles. Déporter les 5 plus gros tampons réduit la mémoire on-chip totale de 17 % [§V, Fig. 9].

Tous les poids restent on-chip (jusqu'à 26 MB, [§IV-C]). La seule carte embarquée testée est la ZCU104, avec YOLOv5n (section 6).

5.6 Vitis AI et la DPU : le flot par défaut, contraint par ses opérateurs

Dans le corpus, Vitis AI n'apparaît que comme flot utilisateur. Il porte aujourd'hui 38 lignes de `benchmarks.csv`, venues de 9 papiers, dont 23 d'un seul (arxiv-2607.13106) ; 29 sont des lignes de détection. Quatre constats en ressortent.

Le modèle s'adapte à la DPU, pas l'inverse. La DPU n'accepte que de l'INT8 obtenu depuis un modèle FP32 (Sali et al., 2025, §V-A)[PP], et ses opérateurs sont fixés. Pour RetinaNet ResNet-50, il faut dupliquer une convolution du FPN pour que tout le réseau tienne sur la DPU sans repli sur le CPU (Magalhães et al., 2022, §2.3). Pour les YOLO récents, Karki et al. (2026)[PP] remplacent SiLU par ReLU, le *split* du bloc C2f par deux convolutions 1×1, et approchent l'attention C2PSA par des convolutions et une hard-sigmoid [§III-B]. Ces modifications coûtent environ 3 points de mAP en moyenne [§IV-B], et le réseau est réentraîné : le modèle déployé n'est plus le modèle de référence, ce que devra tracer une démarche de certification *(déduit)*. Les contraintes d'opérateurs du DPU (ReLU) et celles de FINN via QONNX (ReLU, hardtanh, identité) se recoupent, mais un même YOLO n'a encore été porté sur les deux dans aucun article *(déduit)*.

La quantification coûte de la précision. Sur ZCU104, Schlosser et al. (2023) mesurent une perte de 2,2 points de mAP due au quantificateur Vitis AI, la compilation DPU en retirant encore ; tous les modèles TensorRT de la Jetson AGX Xavier font mieux en mAP [§4.3–4.4]. La calibration reste en PTQ dans arxiv-2607.13106, avec des chutes fortes sur certains couples (YOLOv8n sur KITTI : 0,64 → 0,34 de mAP@0.5 :0.95, [Table II]).

Une comparaison mesurée face au GPU existe. Le même papier mesure la ZCU104 et la Jetson au multimètre en série avec la carte, repos soustrait ou non [§4.1]. Pour YOLOv4 en INT8, la DPU fait 54,42 ms pour 23 W de carte, la Jetson 22,0 ms pour 16,5 W, à puissance dynamique égale (7,0 W) [Table 1] : la DPU n'y gagne pas. Le seul autre papier du corpus qui mesure Vitis AI et un GPU embarqué de la même façon (multimètre à l'entrée de la carte) voit l'inverse : RetinaNet sur ZCU104 à 2 DPU fait 25 FPS, contre 3 à 5 FPS sur Jetson TX2, mais ses puissances ne sont données qu'en figure (Magalhães et al., 2022, §2.5, §3, Fig. 12). Le sens de l'écart dépend donc du GPU (TX2 ou AGX Xavier) et du modèle *(déduit)*.

Le flot est fermé au niveau mémoire. Pour garder les poids hors DDR sur KV260, Fata and Elmannai (2026) ajoutent un contrôleur mémoire RTL et modifient le fichier `arch.json`

du compilateur, ce qui n'est décrit que qualitativement [§IV-E–F]. Ailleurs, la puissance est une estimation Vivado (Karki et al., 2026, Table III) ou n'a pas de portée précisée (Fata and Elmannai, 2026, §V-B). Enfin, le débit de bout en bout plafonne à environ 34 FPS pour YOLOv26n sur ZCU104, limité par le post-traitement sur l'ARM et non par la DPU (Karki et al., 2026, §IV-C).

5.7 Co-design et NAS guidés par le matériel

Trois travaux du corpus adaptent le réseau au matériel plutôt que l'inverse :

— SkyNet (Zhang et al., 2019)[PP] construit un détecteur *bottom-up* à partir de blocs évalués sur la cible, par une recherche PSO dont la fitness combine précision et latence estimée par l'outil HLS [§4, Éq. 1]. Sur Ultra96, il gagne DAC-SDC 2019 avec 0,716 d'IoU à 25,05 FPS [Table 6], mais en lot de 4 images et sans ressources FPGA rapportées ;

— la NAS bayésienne de MLPerf Tiny cherche un ResNet-8 pour hls4ml, au prix de 3,5 points de précision (Borras et al., 2022, §3.1)[PP] ;

— HAWQ choisit les largeurs de bits par couche sous budget de BOPs, vers hls4ml (Campos et al., 2024, Table 2).

Dans les trois cas, le coût matériel guidant la recherche est estimé (HLS, BOPs) ou indirect, jamais mesuré sur carte. Aucun ne vise un YOLO récent.

5.8 Synthèse

Table 1 – Toolflows du corpus (lu dans les fiches). « Puissance » : existe-t-il une mesure sur carte dans les articles du corpus ?

Toolflow	Architecture	Précisions	Tâches démontrées	Puissance mesurée
FINN / FINN-R	streaming ; MO pour gros réseaux ; multi-FPGA (Elastic-DF)	binaire à 8 bits	classification ; Tincy YOLO en MO ; LP-YOLO w2a4 sur Zynq-7020	oui (puce et carte, FINN ; carte, FINN-R, Elastic-DF, arxiv-2609.24757)
fpgaConvNet / SMOF	SDF ; partitionnement ou rechargement des poids ; éviction DDR (SMOF)	FXP16 ; BFP8 (SMOF)	classification ; UNet, X3D, YOLOv8n (SMOF, hors SoC pour YOLOv8n)	dynamique de carte, repos soustrait ; aucune pour SMOF
hls4ml	un bloc par couche, reuse factor	virgule fixe arbitraire, HGQ	MLP, petits CNN, AE et VAE (physique), MLPerf Tiny ; aucune détection	énergie par inférence (EEMBC, MLPerf Tiny), sans W ; sinon estimations
SATAY	streaming, FIFO logicielle DDR	W8A16 (PTQ)	YOLOv3-tiny à v8	méthode non décrite
Vitis AI (DPU)	moteur unique	INT8	YOLOv3-tiny à v26, RetinaNet, compression d'image, apprentissage continu	carte au multimètre (arxiv-2304.11580 ; arxiv-2211.11647, en figure) ; ailleurs estimée ou portée non précisée

Aucun article du corpus ne compare deux de ces flots, mesurés, sur un même réseau et une même carte. Le cas le plus proche, MLPerf Tiny, mesure hls4ml et FINN sur la même carte avec la

même méthode, mais avec des modèles différents et en classification (Borras et al., 2022, Table 5)[PP]. Les papiers d'outil, eux, comparent des chiffres publiés (Venieris et al., 2018, §2.6) ou ne comparent pas (Schulte et al., 2025; Fahim et al., 2021). Dans `benchmarks.csv`, hls4ml n'a aucune ligne de détection (34 lignes, toutes variantes de backend), FINN en a 9 (dont 6 de arxiv-2609.24757), Vitis AI 29 et SATAY 14. QONNX, lisible par FINN et par hls4ml (Pappalardo et al., 2022, §VI)[PP], rend la comparaison techniquement faisable pour un YOLO à activations ReLU ; le DPU accepte les mêmes opérateurs (Karki et al., 2026, §III-B)[PP].

6 Détection d'objets sur FPGA

6.1 Trajectoire du corpus

Les premiers travaux sur la détection partent d'un constat de Ding et al. (2019, §1) : la littérature FPGA accélère surtout la classification. En 2018, aucun toolflow ne visait la détection (Venieris et al., 2018, §2.6, §3.1). Le corpus montre ensuite cinq approches : compression puis design sur mesure, protocole commun de concours, co-conception du réseau et de l'accélérateur, design HLS ou RTL sur SoC, et flots automatiques (dataflow ou DPU de Vitis AI).

Compression puis design sur mesure. REQ-YOLO compresse Tiny YOLO (YOLOv2-tiny) de deux façons :

- des matrices bloc-circulantes, qui transforment la convolution en FFT, produit terme à terme et FFT inverse [§3] ;
- une quantification hétérogène par ADMM qui choisit, couche par couche, entre des multiplications sur DSP et des décalages sur LUT [§4].

Tous les poids tiennent alors en BRAM [§5]. Sur un Virtex-7 690T en carte PCIe, le modèle atteint 314,2 FPS pour 21 W [Table 2]. Trois réserves :

- la latence de 3,18 ms est une somme des latences par couche, sans transfert PCIe ni NMS [Table 1] ;
- la mAP absolue n'est pas dans le texte : seules des pertes relatives le sont [§6.2] ;
- la plateforme n'a rien d'embarqué.

Face à une Jetson TX2 exécutant le même modèle (28,4 FPS, 10,8 W, [Table 2]), le gain est net, mais TensorRT n'est pas mentionné : la TX2 est vraisemblablement sous-exploitée *(déduit)*.

Protocole commun, mesuré par un tiers. Le concours DAC-SDC 2018 (Xu et al., 2021) impose deux plateformes (PYNQ-Z1 et Jetson TX2), un jeu de test caché d'images de drone, un débit minimal et une énergie mesurée au wattmètre par les organisateurs [§2.3]. C'est le seul protocole commun FPGA/GPU du corpus. Les équipes FPGA gagnantes utilisent des détecteurs maison en HLS ou RTL sur XC7Z020, avec les cartes d'activation en DDR [§4.2] :

- TGIIF, un SSD élagué en 8 bits : IoU de 0,62 à 11,96 FPS [§5.2.1] ;
- SystemsETHZ, un SqueezeNet binaire suivi d'une tête YOLO : IoU de 0,49 à 25,97 FPS [§5.2.1].

Limites : la métrique est l'IoU d'un seul objet par image, pas une mAP ; les puissances par équipe ne figurent qu'en figures ; le score composite utilise des bases de logarithme différentes pour le FPGA et le GPU [§2.3]. Les auteurs estiment que le FPGA consomme 1/3 à 1/2 de l'énergie du GPU pour une précision proche, sans chiffre dans le texte [§5.3].

Co-conception du réseau et de l'accélérateur. Deux travaux conçoivent le détecteur en fonction de la cible plutôt que de compresser un réseau de référence :

- SkyNet (Zhang et al., 2019)[PP], vainqueur de DAC-SDC 2019, assemble des blocs « Bundles » évalués sur la cible et cherche les canaux par essaim particulaire sous contrainte de latence

[§4]. Le même modèle est mesuré par les organisateurs sur TX2 (IoU 0,731, 67,33 FPS, 13,50 W) et sur Ultra96 (IoU 0,716, 25,05 FPS, 7,26 W) [Tables 5–6]. Le FPGA traite des lots de 4 images [§6.4] : les 25,05 FPS sont un débit, pas l'inverse d'une latence par image *(déduit)*. Aucune ressource FPGA n'est rapportée.

— CoDeNet (Dong et al., 2020) adapte la convolution déformable au matériel (offsets bornés, entiers) et utilise une tête CenterNet sans NMS [§3.1, §4.1]. Sur Ultra96, en W4A8, la configuration 256×256 obtient 51,1 d'AP50 VOC07 à 32,2 FPS, 31 ms à batch 1 et 5,6 W de carte, dont 4,3 W au repos [Tables 3 et 5, §5]. En 512×512, l'AP50 monte à 61,7 mais le débit tombe à 9,3 FPS. Le résumé annonce « 26,9 fps » et « 61,7 AP50 », deux chiffres de configurations différentes.

Design HLS ou RTL sur mesure. Quatre travaux écrivent leur propre accélérateur, du SoC d'entrée de gamme au FPGA sans processeur dur :

— Zhai et al. (2023) élaguent YOLOv3-tiny par canaux (calcul divisé par 7,4), le quantifient en virgule fixe 16 bits dynamique et l'implantent en HLS sur un Zynq XC7Z035. Leur accélérateur combine Winograd pour les convolutions 3×3 et im2col + GEMM pour les 1×1 [§3.1, §3.5]. Le NMS et le suivi Deepsort tournent sur l'ARM [§3.3]. Avec un module, ils obtiennent 91,65 FPS pour 12,51 W ; avec deux modules, 168,72 FPS pour 15,18 W [Tables 8–9]. La méthode de mesure de puissance n'est pas décrite. L'AP@0.5 est identique en FP32 et en INT16 à trois décimales, ce qui laisse penser qu'elle n'a pas été remesurée sur la cible *(déduit)*. La ligne Jetson Nano de leur comparaison vient d'un autre article et porte sur un modèle non élagué [Table 7] : le gain affiché vient donc surtout de l'élagage.

— Yan et al. (2024) quantifient tout YOLOv5s en 4 bits, première couche comprise, et empaquettent quatre multiplications par DSP [§4–5]. Sur ZCU102, en 512×512, ils annoncent 79,1 FPS, 12,63 ms et 12,3 W sans méthode de mesure, pour 73,8 de mAP VOC (1,5 point sous le flottant) [Tables 2–3]. Le « +29 % face à Vitis AI » du résumé est un rapport de GOPS/DSP face au YOLOv3 d'un autre travail, pas une mesure de la DPU sur la même carte [Table 2].

— Zhang et al. (2024) visent un FPGA sans cœur ARM (Kintex-7 XC7K325T, MicroBlaze) et mettent tout le post-traitement en matériel, NMS compris [§3.5]. YOLOv3-tiny INT8 tient en 15,2 ms de bout en bout [Table 5], pour 12,3 W au wattmètre de carte et 7,8 W de puce estimés par Vivado [§5.1, Tables 3–4]. C'est la seule paire puce/carte FPGA du corpus, mais la puce est estimée. Le débit n'est donné qu'en GOP/s, sans FPS.

— Kim and Kim (2025) implantent YOLOv2 INT16 en RTL sur le plus petit SoC du corpus (Zynq XC7Z020, 100 MHz). Une image 416×416 demande 12,64 s [§4.2], soit moins de 0,1 FPS *(déduit)*. La précision n'est donnée que sur quelques détections d'exemple, pas en mAP [§4.2], et les gains en FF et DSP (jusqu'à −26 et −15 %) sont obtenus contre des designs bien plus rapides [Table 9].

Toolflow dataflow. SATAY (section 5) couvre YOLOv3-tiny à YOLOv8s, mais ses meilleurs chiffres viennent de cartes Virtex de 23 à 67 W [Table III]. Sur la seule carte SoC testée, la ZCU104, YOLOv5n en 640×640 obtient 26,7 de mAP50 COCO pour 21,41 ms et 14,5 W, contre 27,7 de mAP50, 32,28 ms et 8,58 W pour une Jetson TX2 [Table IV]^{m}. L'énergie par inférence est du même ordre (317 contre 277 mJ, [Table IV]). La quantification y est simulée par ONNX Runtime [§IV-A] : la mAP n'est pas mesurée sur la carte. SMOF (Toupas et al., 2024)^{PP} étend fpgaConvNet aux réseaux qui débordent de la puce : YOLOv8n atteint 184 FPS, mais sur VCU118 et sans mesure de puissance [Table V]^{m}.

Le dataflow quantifié existe désormais sur le plus petit SoC du corpus. Gutierrez et al. (2026)^{PP} entraînent par QAT (Brevitas) un YOLOv3-tiny très allégé, LP-YOLO Slim, en poids 2 bits et activations 4 bits, et le compilent par FINN sur un Zynq XC7Z020 (PYNQ-Z1). Ils mesurent 35,66 FPS, 45,1 ms de latence dans la PL et 2,91 W sur le rail 12 V de la carte, pour 0,594 de

mAP@0,5 sur les véhicules de VOC [§V-D, Table II]^{m}. Ils distinguent les estimations de FINN des mesures sur carte (38 contre 22 FPS pour un même repliement, [§III-B]). Sur leur YOLOv3-tiny de référence en 4 bits, la QAT reste à environ 0,006 de mAP du flottant, contre environ 0,17 de perte en PTQ [§IV-B]. Réserves : classes de véhicules seules, débit mesuré par lots de 100 images, ressources non extraites, seuils de réussite fixés par les auteurs.

Flot industriel : la DPU de Vitis AI. Quatre travaux utilisent la DPU, le moteur unique d'AMD :

- Schlosser et al. (2023) construisent un banc où un hôte séparé envoie les images et calcule la mAP à partir des sorties de la carte [§3]. YOLOv4 512×512 INT8 sur ZCU104 obtient 37,5 de mAP50-95 COCO pour 54,42 ms d'inférence et 23 W de carte au multimètre [§4.1, Table 1]. Le quantificateur coûte 2,2 points, la compilation pour la DPU en retire encore [§4.3]. C'est le travail du corpus qui s'approche le plus d'un protocole complet ; il compare aussi le même modèle sur Jetson AGX Xavier (voir plus bas).
- Karki et al. (2026)^{PP} remplacent les opérateurs que la DPU ne sait pas exécuter (SiLU par ReLU, split par des convolutions 1×1, attention C2PSA approchée par un hard-sigmoid) et déploient YOLOv8n, v11n et v26n en PTQ INT8 sur ZCU104 [§III-B]. La mAP50-95 mesurée avant et après le passage sur la DPU varie de façon imprévisible : YOLOv11n perd 2 points sur VOC (0,59 à 0,57), YOLOv8n en perd 15 sur VOC (0,60 à 0,45) et 30 sur KITTI (0,64 à 0,34) [Table II]. Les modifications seules coûtent environ 3 points [§IV-B, Table I]. Le résumé annonce environ 5 % de perte, §IV-B environ 7 %. YOLOv26n tourne à 34,05 FPS de bout en bout sur une vidéo de personnes [§IV-C], et la puissance (4,8 à 8,8 W selon la taille de DPU) est une estimation de Vivado [Table III].
- Fata and Elmannai (2026) sont les seuls à cibler la KV260 avec une mesure complète de débit. Ils élaguent YOLOv3-tiny à 70 %, le quantifient par QAT INT8 et ajoutent un contrôleur RTL qui garde les poids en BRAM et URAM plutôt qu'en DDR [§IV-C–E]. Ils obtiennent 24,3 FPS, 41,59 ms et 2,13 W de portée non précisée [Table 8]^{m}. Leur précision est une « accuracy » (TP+TN)/(TP+TN+FP+FN), mal définie pour la détection (Eq. 4), et aucune baseline flottante ne mesure la perte due à la compression. Leur GPU de comparaison, une RTX 2070 de bureau, va plus vite (44,64 FPS, [Table 10]).
- Al Amin and Obermaisser (2025) déploient YOLOv5 INT8 sur KV260 : 9 FPS et 3,5 W, méthode non décrite [Table I]. Leur précision (98,6 %) est une « classification accuracy » sur 72 images choisies, qui ne se compare à aucune mAP. Leur propre tableau contredit leur conclusion : 2,6 FPS/W, sous quatre des cinq travaux cités [§IV-B, Table I].

6.2 Tableau des détecteurs FPGA du corpus

Sources : REQ-YOLO (Ding et al., 2019, Tables 1–2) ; FINN-R (Blott et al., 2018, Table 5) ; DAC-SDC (Xu et al., 2021, §5.2.1) ; SkyNet (Zhang et al., 2019, Tables 6–7)^{PP} ; CoDeNet (Dong et al., 2020, Tables 3, 5, 6) ; Zhai (Zhai et al., 2023, Tables 8–9) ; Zhang 2024 (Zhang et al., 2024, Tables 4–5) ; Yan (Yan et al., 2024, Tables 2–3) ; Kim (Kim and Kim, 2025, Tables 8–9) ; SATAY (Montgomerie-Corcoran et al., 2023, Tables III–IV) ; SMOF (Toupas et al., 2024, Tables III, V)^{PP} ; Gutierrez (Gutierrez et al., 2026, §V-D, Table II)^{PP} ; Schlosser (Schlosser et al., 2023, Table 1) ; Karki (Karki et al., 2026, Tables II–III, §IV-C)^{PP} ; Fata (Fata and Elmannai, 2026, Tables 7–8) ; Magalhães (Magalhães et al., 2022, §3) ; Al Amin (Al Amin and Obermaisser, 2025, Table I). La latence de REQ-YOLO est une somme des latences par couche ; celle de Schlosser, l'inférence seule ; celle de Gutierrez, la PL seule (49,8 ms de bout en bout). Chez Karki, la précision vient du jeu de personnes des auteurs et le débit d'une autre vidéo (PETS2009). Le mAP* de SATAY porte un astérisque dont la définition n'a pas été extraite.

TABLE 2 – Détecteurs sur FPGA dans `benchmarks.csv` (une ou deux lignes par travail). Précisions non comparables d'une ligne à l'autre (jeux, métriques et résolutions différents). ^{m} : confiance *medium*. Portée de puissance : u = *unspecified*, b = *board*, e = *estimated* (Vivado).

Travail	Modèle, précision	Puce	Métrique	Préc.	FPS	ms	W (portée)
REQ-YOLO	Tiny YOLO bloc-circulant, bits `n/r`	XC7VX690T	mAP VOC	`n/r`	314,2	3,18	21 (u)
FINN-R	Tincy YOLO W1A3 (MO)	XC7Z020	mAP VOC	50,1	`n/r`	`n/r`	2,5 (b)
FINN-R	Tincy YOLO W1A3 (MO)	XCZU3EG	mAP VOC	50,1	`n/r`	`n/r`	9,7 (b)
DAC-SDC	TGIIF : SSD élagué INT8	XC7Z020	IoU, 1 objet	62,4	11,96	`n/r`	`n/r`^{m}
DAC-SDC	SystemsETHZ : SqueezeNet binaire + YOLO	XC7Z020	IoU, 1 objet	49,2	25,97	`n/r`	`n/r`^{m}
SkyNet	FM9/W11, lot de 4	XCZU3EG	IoU, 1 objet	71,6	25,05	`n/r`	7,26 (u)
CoDeNet	W4A8, 256, sans NMS	XCZU3EG	AP50 VOC07	51,1	32,2	31	5,6 (b)^{m}
CoDeNet	W4A8, 512	XCZU3EG	AP50 VOC07	61,7	9,3	`n/r`	`n/r`^{m}
Zhai	YOLOv3-tiny élagué INT16, 1 module	XC7Z035	AP@0.5, 1 classe	59,9	91,65	`n/r`	12,51 (u)
Zhai	idem, 2 modules	XC7Z035	—	`n/r`	168,72	`n/r`	15,18 (u)
Zhang 2024	YOLOv3-tiny INT8, NMS matériel	XC7K325T	mAP VOC	58,1	`n/r`	15,2	12,3 (b)
Yan	YOLOv5s 512 W4A4	ZCU102	mAP VOC	73,8	79,1	12,63	12,3 (u)
Kim	YOLOv2 416 INT16, RTL	XC7Z020	`n/r`	`n/r`	`n/r`	12 639	2,04 (u)^{m}
SATAY	YOLOv5n 640, PTQ	ZCU104	mAP50 COCO	26,7	`n/r`	21,41	14,5 (u)^{m}
SATAY	YOLOv8s 640, W8A16	VCU118	mAP* COCO	61	`n/r`	24,5	57,4 (u)^{m}
SMOF	YOLOv8n 640, bfp8	VCU118	mAP COCO (non nommée)	35,1	184,27	`n/r`	`n/r`^{m}
Gutierrez	LP-YOLO Slim 256 W2A4 (FINN)	XC7Z020	mAP@0,5 véhicules VOC	59,4	35,66	45,11	2,91 (b)^{m}
Schlosser	YOLOv4 512 INT8 (DPU)	ZCU104	mAP50-95 COCO	37,5	`n/r`	54,42	23 (b)
Karki	YOLOv26n 416 INT8 (DPU)	ZCU104	mAP50-95 personnes	67	34,05	29,37	8,76 (e)^{m}
Fata	YOLOv3-tiny élagué INT8 (DPU)	XCK26	« accuracy » VOC	93,8	24,3	41,59	2,13 (u)^{m}
Magalhães	RetinaNet R50 INT8, 2 DPU	ZCU104	mAP VineSet	`n/r`	25	`n/r`	`n/r`^{m}
Al Amin	YOLOv5 INT8 (DPU)	KV260	acc., 72 images	98,6	9	`n/r`	3,5 (u)

Lecture du tableau. Un recompte de `benchmarks.csv` au 1er octobre 2026 (299 lignes, dont 216 sur FPGA) donne 87 lignes de détection sur FPGA, issues de 17 articles. Elles emploient 14 libellés de précision différents, et 12 n'ont aucune précision ; 54 n'ont pas de FPS et 33 pas de puissance. Seules 21 lignes portent une mAP COCO, issues de quatre articles : SATAY (14 lignes, quantification simulée), Karki et al. (3), Schlosser et al. (3) et SMOF (1). Les mAP de Karki et de Schlosser sont les seules mesurées sur la carte. Dix lignes, de quatre articles, réunissent une précision, un débit ou une latence et une puissance de carte mesurée : Zhang 2024 (mAP VOC), CoDeNet (AP50 VOC07), Gutierrez et al. (véhicules VOC) et Schlosser et al. (mAP COCO). Aucune ne réunit une mAP COCO, un FPS de bout en bout à batch 1 et une puissance de carte mesurée sur une carte de la gamme Kria. Schlosser et al. s'en approchent le plus, avec une latence d'inférence seule et une ZCU104 de 23 W.

Les surveys de 2025 confirment ce manque d'homogénéité :

— Hozhabr and Giorgi (2025) proposent de normaliser par le *pixel throughput* (FPS × pixels par image, [§II-C2]). Cela corrige la résolution, mais ni la précision ni la portée de puissance ;
— Sali et al. (2025)[PP] alignent des designs mesurés dans des conditions hétérogènes, avec des incohérences entre texte et tableaux (voir la fiche).

Hozhabr et Giorgi affirment que « FPGAs have emerged as the optimal hardware platform, offering both speed and power efficiency » [§IV], sans comparaison mesurée avec une Jetson récente ni un MCU à NPU. Les deux travaux du corpus qui mesurent le même détecteur sur FPGA et sur Jetson ne vont pas dans ce sens. Chez Schlosser et al., la Jetson AGX Xavier en INT8 sous TensorRT est plus rapide (22,0 contre 54,42 ms), plus précise (38,6 contre 37,5) et plus efficace (2,05 contre 1,31 FPS/W) que la ZCU104 [Table 1]. Chez SkyNet, la TX2 fait 4,99 FPS/W et l'Ultra96 3,45, valeurs calculées à partir des Tables 5–6 *(déduit)*.

6.3 Post-traitement et interaction PS–PL

Le NMS, le suivi et le redimensionnement tournent presque toujours sur le processeur du SoC et sont rarement comptés :

— Zhai et al. : NMS et Deepsort sur l'ARM, inclus ou non dans le FPS, on ne sait pas [§3.3] ;
— REQ-YOLO : latence sans NMS [Table 1] ;
— Magalhães et al. : cœur du réseau seul, pré- et post-traitement exclus [§2.5] ;
— DAC-SDC : redimensionnement sur l'ARM [§4.2] ;
— Yan et al. : pré- et post-traitement, NMS compris, sur l'ARM [§5] ; Kim et al. : couches de concaténation et de détection sur le Cortex-A9 [§4.2] ;
— Gutierrez et al. : décodage et NMS sur l'ARM [§II-B], soit environ 4,7 ms entre la latence de la PL et celle de bout en bout [§V-D] *(déduit)* ;
— CoDeNet : pas de NMS, mais la première couche tourne sur l'ARM [§4.2].

Le post-traitement limite souvent le débit. Karki et al. attribuent l'écart entre le débit de la DPU et les 34,05 FPS de bout en bout au post-traitement mono-cœur [§IV-C]. Sur Jetson, Schlosser et al. mesurent un post-traitement en numpy plus long que l'inférence, alors que celui de la DPU est optimisé par la Vitis AI Library [§4.2, Table 2]. Le survey de Sali et al. cite un cas où le débit passe de 195 FPS pour la détection seule à 24 FPS de bout en bout, à cause d'un post-traitement en Python [§VI-C] (chiffre d'un travail cité, non vérifié).

Le contre-exemple est maintenant dans le corpus. Hozhabr et Giorgi citaient deux systèmes où le NMS est en matériel, sur Stratix 10 et sur Kintex-7 [§V-A]. Le second est l'article de Zhang et al. (2024) : leur unité de post-traitement traite 845 boîtes en 0,38 ms avec 8 893 LUT et 19 DSP [Table 2]. Elle tourne sur un FPGA sans cœur ARM, avec un MicroBlaze : aucun travail ne la place à côté d'une DPU sur un SoC Zynq.

6.4 Détection de personnes

Le cas d'application de la thèse est la détection de personnes. Un seul travail du corpus mesure un détecteur de personnes sur FPGA. Karki et al. (2026)[PP] entraînent leurs YOLO nano sur un jeu maison (images de vidéosurveillance et de VOC, 18 820 images, une classe) et obtiennent sur la DPU une mAP50-95 de 0,67 à 0,79 selon le modèle [§III-A, Table II]. Le jeu n'est pas publié, et le débit est mesuré sur une autre vidéo. Les autres détecteurs FPGA visent des véhicules (Zhai, Kim, Gutierrez), des défauts de circuits imprimés (Yan), les classes de VOC ou des objets vus de drone.

Côté données, CrowdHuman (Shao et al., 2018)[PP] est le seul jeu de personnes du corpus : 15 000 images d'entraînement, 22,64 personnes par image en moyenne, avec des boîtes de tête, de corps visible et de corps entier [§3, Table 2]. Ses images viennent du web, sans vidéo ni scène industrielle [§3.1], et ses baselines sont des FPN ResNet-50 sur GPU [§4.1]. Dans tous ces travaux, la précision est une moyenne (mAP, ou mMR moyenné sur les fausses alarmes par image). Aucun ne donne un taux de personnes manquées à taux de fausses alarmes fixé, par distance ou par occultation, encore moins sur la cible.

7 Détection d'objets hors FPGA : GPU embarqués, NPU et microcontrôleurs

7.1 GPU embarqués et NPU

Premiers bancs d'essai : des protocoles incomplets. Feng et al. (2022) comparent YOLOv3 et YOLOv4 (complets et tiny) sur Jetson Nano, Xavier NX et Raspberry Pi 4 + NCS2, en FP16. Leurs choix limitent la portée des résultats :

- les Jetson tournent sous Darknet, sans TensorRT, choisi exprès pour rester comparable au Raspberry Pi [§3.2] ;
- aucune précision n'est mesurée : la « mean confidence » est une moyenne des scores de confiance, sans vérité terrain [§4.1] ;
- la portée de la puissance n'est pas décrite.

Résultat marquant : après conversion par OpenVINO, YOLOv3-tiny sur NCS2 tombe à 0 % de confiance moyenne, contre 59,7 % sur les Jetson [Table 3]. La conversion vers un accélérateur peut dégrader la détection sans que rien ne l'indique.

Magalhães et al. (2022) comparent un gros détecteur, RetinaNet ResNet-50, sur Jetson TX2, Coral Edge TPU, ZCU104 et KV260, avec une mAP mesurée et une puissance à l'entrée de la carte [§2.5]. Les FPGA atteignent 14 à 25 FPS, les GPU embarqués 3 à 5 FPS [Abstract]. Mais la TX2 tourne en FP32 ou FP16 et les FPGA en INT8 : l'écart mêle l'effet du matériel et celui de la quantification. Les Jetson Nano ont été exclues faute de mémoire pour la conversion [§4]. La plupart des valeurs ne figurent que dans des graphiques.

Baselines Jetson Orin. Deux travaux récents fournissent les baselines les plus utiles du corpus :

- Rey et al. (2025) mesurent YOLOv8n et s en 640×640 sous TensorRT (FP32, FP16, INT8), avec une puissance par télémétrie (tegrastats) et une énergie par inférence [§3.2, Table 4]. Leur jeu de données est privé et facile : une seule cible, en intérieur. La PTQ INT8 y coûte 6,4 à 12,3 points de mAP50-95, alors que la discussion parle d'une perte non significative [§6].
- Alqahtani et al. (2024)[PP] donnent la seule mAP COCO val du corpus sur Orin Nano et sur Hailo-8. Leur protocole est réutilisable : service à une requête à la fois, repos mesuré puis soustrait, trois répétitions [§III]. La précision numérique des moteurs TensorRT et Hailo n'est pas indiquée, et la mAP est identique sur Jetson Nano et Orin Nano, ce qui laisse douter qu'elle ait été mesurée sur chaque carte [§IV-C].

Jetson et FPGA dans un même papier. Dans `benchmarks.csv`, quatre articles seulement ont des lignes à la fois sur FPGA et sur Jetson, et aucun sur Orin : les seules lignes Orin viennent de Rey et d'Alqahtani, sans FPGA.

- Schlosser et al. (2023) mesurent le même YOLOv4 sur Jetson AGX Xavier (TensorRT FP32, FP16, INT8) et sur ZCU104 (DPU), avec une mAP COCO calculée à partir des sorties de chaque carte et une puissance de carte au multimètre [§3, §4.1]. En INT8, la Jetson est plus rapide (22,0 contre 54,42 ms d'inférence), plus précise (38,6 contre 37,5) et plus efficace (2,05 contre 1,31 FPS/W) [Table 1]. Réserves : un seul modèle et deux grosses cartes de développement ; le post-traitement de la Jetson est en numpy alors que celui de la DPU est optimisé [§4.2] ; des moyennes sur six passes, sans pire cas [§4.1]. Les puissances relatives, repos déduit, sont égales (7,0 W, [Table 1]). Le même banc recommande le mode 15 W de la Jetson pour l'efficacité [§4.4].
- SkyNet (Zhang et al., 2019)[PP] : même modèle sur TX2 (FP32) et Ultra96 (virgule fixe 9/11 bits), mesuré par les organisateurs de DAC-SDC 2019. La TX2 va plus vite (67,33 contre 25,05 FPS) pour 13,50 contre 7,26 W, portée non dite [Tables 5–6].
- Castelino et al. (2024) (autoencodeur de détection d'artefacts, pas un détecteur d'objets) donnent le résultat inverse : ZCU104 (DPU, INT8) en 4,0 ms contre 10,36 ms pour une Xavier NX en INT8 [§IV-B, Table VI]. Mais la Xavier NX est bridée à 10 W et les puissances n'ont pas la même portée (rails PMBus contre jtop).
- Aggarwal et al. (2024) comparent un apprentissage continu sur ZCU102 (413 ms par image d'entraînement) et sur Jetson Nano (33 ms), mais à batch 1 contre batch 8 [Table VI] (section 10).

Les baselines GPU des autres travaux de détection sur FPGA sont faibles ou reprises : Jetson Nano en FP32 plutôt qu'en INT8 (Zhang et al., 2024, §5.3), CPU et GPU de serveur repris d'un autre article (Yan et al., 2024, Table 3), GPU de bureau (Fata and Elmannai, 2026, Table 10). Dans les deux articles où une Jetson exécute le même détecteur que le FPGA (Schlosser et al. et SkyNet), elle est plus rapide ; elle est aussi plus efficace en FPS/W (puissance de carte chez Schlosser, de portée non dite chez SkyNet) *(déduit)*.

7.2 Microcontrôleurs

Sur MCU, la contrainte principale est la mémoire. Le survey de El Zeinaty et al. (2025) recense des détecteurs de 55 à 789 MMACs, avec des pics de SRAM de 53 à 511 kB et des mAP VOC de 51 à 75 % [Table 8], chiffres repris des articles d'origine. Ce tableau ne donne ni latence ni énergie, bien que le survey présente ces métriques comme essentielles [§4.2].

Lin et al. (2021) exécutent les premiers blocs du réseau par patchs, ce qui réduit le pic de SRAM, et cherchent conjointement l'architecture et l'ordonnancement [§3]. MCUNetV2-M4 atteint 64,6 % de mAP VOC en 247 kB de SRAM sur un Cortex-M4 [Table 3]. Ni la latence ni l'énergie de ces détecteurs ne sont données.

Moosmann et al. (2023) mesurent un petit YOLO en 88×88 sur quatre MCU, avec la puissance du MCU seul [§III]. Sur le MAX78000, qui intègre un accélérateur CNN, une inférence dure 5,5 ms et coûte 196 µJ ; le Cortex-M7 d'un STM32H7A3 met 359 ms [§III, Fig. 2][m]. L'accélérateur intégré fait donc la différence (plus de 65 fois plus rapide). La mAP de 58,5 % porte sur 3 classes de VOC et au plus 3 objets par image [Table II] ; elle ne se compare pas aux mAP de MCUNetV2 sur 20 classes. Le papier lui-même donne trois valeurs différentes selon l'endroit (voir la fiche).

Aucun article du corpus ne mesure un même détecteur sur MCU et sur FPGA. Le détecteur FPGA le plus proche d'un MCU en taille est le LP-YOLO Slim de Gutierrez et al. (2026)[PP] : environ 0,35 M de paramètres pour LP-YOLO [§III-A], poids sur 2 bits, 2,91 W de carte sur Zynq XC7Z020 [§V-D]. Il n'a pas été porté sur MCU, et ses 2,91 W sont une puissance de carte, alors que TinyissimoYOLO mesure le MCU seul : un portage des deux côtés, à portée d'énergie égale, manque.

7.3 Tableau des détecteurs hors FPGA

TABLE 3 – Détecteurs hors FPGA dans `benchmarks.csv` (sélection). Précisions non comparables d'une ligne à l'autre. ^{m} : confiance *medium*. Portée : c = *chip*, b = *board*, u = *unspecified*.

Travail	Modèle, précision	Plateforme	Métrique	Préc.	FPS	ms	W
Rey	YOLOv8n 640 INT8	Orin Nano	mAP50-95, jeu privé	71,9	44,9	23,16	7,48 (c)
Rey	YOLOv8n 640 FP16	Orin Nano	mAP50-95, jeu privé	84,4	40,0	26,70	7,42 (c)
Rey	YOLOv8n 640 INT8	Orin NX	mAP50-95, jeu privé	71,9	65,83	15,16	10,31 (c)
Rey	YOLOv8n 640 FP32	RPi 5 (CPU)	mAP50-95, jeu privé	84,4	8,47	118	5,44 (c)^{m}
Alqahtani	YOLOv8n 640, précision `n/r`	Orin Nano	mAP COCO val	31	`n/r`	16	`n/r`^{m}
Alqahtani	YOLOv8n 640, précision `n/r`	RPi 5 + Hailo-8	mAP COCO val	32	`n/r`	76	`n/r`^{m}
Schlosser	YOLOv4 512 INT8	AGX Xavier	mAP50-95 COCO	38,6	`n/r`	22,0	16,5 (b)
Schlosser	YOLOv4 512 FP16	AGX Xavier	mAP50-95 COCO	39,8	`n/r`	35,32	20 (b)
SkyNet	FP32	TX2	IoU, 1 objet	73,1	67,33	`n/r`	13,5 (u)
Feng	YOLOv3-tiny 416 FP16	Xavier NX (sans TensorRT)	confiance moy.	59,7	41,1	`n/r`	13,5 (u)
Feng	YOLOv3-tiny 416 FP16	RPi 4 + NCS2	confiance moy.	0	18,8	`n/r`	6,5 (u)^{m}
Tinyissimo-YOLO	88×88 INT8	MAX78000	mAP VOC 3 classes	58,5	180	5,5	`n/r`^{m}
Tinyissimo-YOLO	88×88 INT8	STM32H7A3	mAP VOC 3 classes	58,5	`n/r`	359	`n/r`^{m}
MCUNetV2	M4 + YOLOv3 INT8	STM32F412	mAP VOC	64,6	`n/r`	`n/r`	`n/r`
MCUNetV2	H7 + YOLOv3 INT8	STM32H743	mAP VOC	68,3	`n/r`	`n/r`	`n/r`

Sources : Rey (Rey et al., 2025, Table 4, §4) ; Alqahtani (Alqahtani et al., 2024, §IV-B–C) ; Schlosser (Schlosser et al., 2023, Table 1) ; SkyNet (Zhang et al., 2019, Table 5)^{PP} ; Feng (Feng et al., 2022, Table 3) ; TinyissimoYOLO (Moosmann et al., 2023, §III, Table II) ; MCUNetV2 (Lin et al., 2021, Table 3). Chez Rey et al., la portée *chip* correspond à une télémétrie embarquée (rails du module Jetson, PMIC du Raspberry Pi), pas à un wattmètre sur la carte. La latence de Schlosser est l'inférence seule ; la mesure de SkyNet est faite par les organisateurs de DAC-SDC.

8 Comparer les plateformes : ce que le corpus permet, et ce qu'il ne permet pas

8.1 Débit et puissance rapportés

La figure 1 place toutes les lignes de `benchmarks.csv` qui ont à la fois un FPS et une puissance : 82 sur 299, dont 53 sur FPGA. Elle se lit avec prudence. Les tâches, les modèles et les résolutions diffèrent d'un point à l'autre. Les points pleins ont une puissance mesurée (sur la puce ou la carte) ; les points creux, une puissance estimée ou de portée inconnue.

Pour la détection (panneau de gauche, 50 lignes), ce que la figure montre :

— les GPU Jetson se regroupent entre 7 et 15 W, de 1,6 à 67 images par seconde. Les points pleins sont les Orin Nano et Orin NX de Rey et al., mesurés par télémétrie embarquée (Rey

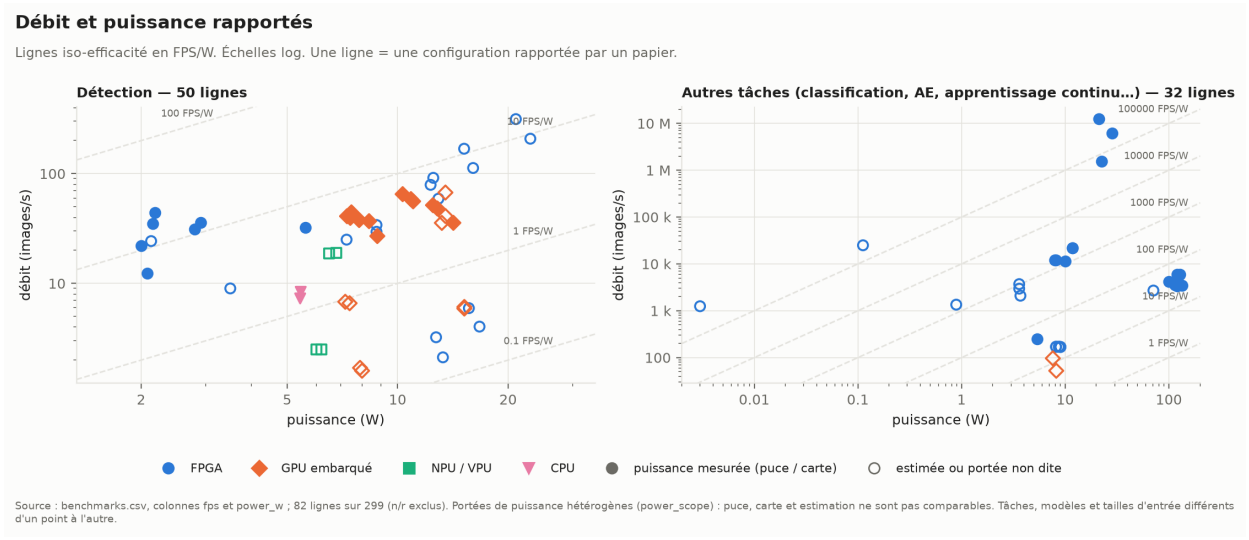


FIGURE 1 – Débit et puissance rapportés, en échelles logarithmiques ; les droites pointillées sont des lignes d'iso-efficacité en FPS/W. Figure générée par `bin/pdb plot` à partir de `benchmarks.csv` (01/10/2026, 82 lignes sur 299).

et al., 2025, Table 4) ; les points creux, les Jetson Nano et Xavier NX de Feng et al., sans TensorRT et de portée non décrite (Feng et al., 2022, Table 3) ;

— les FPGA s'étalent sur deux ordres de grandeur de débit (de 2,1 à 314 FPS) et sur un ordre de grandeur de puissance (de 2 à 23 W) ;

— seules 7 des 23 lignes FPGA de détection ont une puissance mesurée, toutes au niveau de la carte : six LP-YOLO sur PYNQ-Z1, de 2,00 à 2,91 W pour 12 à 44 FPS (Gutierrez et al., 2026, §V)[PP], et CoDeNet sur Ultra96, 32,2 FPS à 5,6 W (Dong et al., 2020, Table 3)[m]. Les 16 autres sont estimées par Vivado ou de portée inconnue.

Ce que la figure ne permet pas : conclure qu'une famille est plus efficace qu'une autre. Aucun couple de points ne partage à la fois le modèle, la résolution, la précision numérique et la méthode de mesure *(déduit)*. Les petites FPGA mesurées à 2–3 W portent des LP-YOLO à 256 ou 320 pixels, filtrés sur les véhicules ; les Orin portent des YOLOv8 à 640 pixels.

8.2 Les comparaisons à tâche égale

Six articles comparent une FPGA et un GPU embarqué sur le même réseau, mesurés par les mêmes auteurs (table 4). Deux protocoles plus anciens s'y ajoutent : fpgaConvNet face à un Tegra X1 bridé à 76,8 MHz pour égaliser le budget de puissance (Venieris and Bouganis, 2017, §3.1), et DAC-SDC 2018, mesuré par les organisateurs (Xu et al., 2021).

Les sources : Zhang et al. (2019)[PP], Schlosser et al. (2023), Montgomerie-Corcoran et al. (2023), Castelino et al. (2024), Sun et al. (2022) et Aggarwal et al. (2024). Ce que la table montre :

— sur les trois détecteurs, le GPU embarqué est au moins aussi sobre par image. Chez SkyNet[PP] et Schlosser et al., il est aussi plus rapide et plus précis, y compris en INT8 des deux côtés : le YOLOv4 de Schlosser et al. tient 2,05 FPS/W sur AGX Xavier contre 1,31 sur ZCU104. SATAY est en moyenne 3,6 fois plus rapide que la TX2 [§VI-B], mais pas plus sobre ;

— la seule comparaison favorable au FPGA est celle de Castelino et al. : 2,6 fois moins de latence et 3,6 fois moins d'énergie que la Xavier NX [§IV-B]. Mais les portées diffèrent (rails de la puce contre jtop), la Jetson est bridée à 10 W, et le FPGA perd 2,5 points de F1. Il s'agit d'un autoencodeur, pas d'un détecteur ;

— F-LIC est à parité d'énergie avec la Xavier NX, sur une FPGA de centre de calcul ; Chameleon est quatre fois moins sobre que la Jetson Nano *(déduit)*.

Aucune de ces comparaisons n'oppose une FPGA embarquée à une Jetson Orin *(déduit)*.

TABLE 4 – Comparaisons FPGA contre GPU embarqué à réseau égal, dans un même article. Les énergies marquées *(déduit)* sont calculées ici (puissance divisée par le débit, ou inverse du FPS/W publié).

Travail	Tâche, modèle	FPGA	GPU embarqué	Réserves
SkyNet[PP] [Tables 5–7]	détection mono-objet, DAC-SDC'19	Ultra96, FM9/W11 : IoU 0,716 ; 25,05 FPS ; 7,26 W ; ≈ 0,29 J/image *(déduit)*	TX2, FP32 : IoU 0,731 ; 67,33 FPS ; 13,50 W ; ≈ 0,20 J/image *(déduit)*	portée `n/r` (organisateurs) ; batch 4 sur FPGA
Schlosser [Table 1]	YOLOv4 512, COCO	ZCU104, DPU INT8 : mAP 37,5 ; 54,42 ms ; 23 W ; 1,31 FPS/W	AGX Xavier, TensorRT INT8 : mAP 38,6 ; 22,0 ms ; 16,5 W ; 2,05 FPS/W	carte au multimètre des deux côtés ; post-traitement optimisé sur DPU, en Python sur Jetson
SATAY [Table IV][m]	YOLOv5n 640	ZCU104 : 317 mJ	TX2 : 277 mJ	méthode `n/r` ; en 320, TX2 70,7 mJ contre 117,5 à 431 mJ
Castelino [Table VI, §IV-B][m]	CAE d'anomalies, images hyperspectrales	ZCU104, DPU INT8 : F1 92,77 ; 4,0 ms ; 5,38 W ; 21,52 mJ	Xavier NX 10 W, TensorRT INT8 : F1 95,29 ; 10,36 ms ; 7,6 W ; ≈ 79 mJ *(déduit)*	puce (PMBus) contre jtop ; Jetson bridée
F-LIC [Fig. 6][m]	compression apprise, encodage 720p	VCU118 : 1,41 J	Xavier NX : 1,43 J	méthode `n/r` ; grosse FPGA
Chameleon [Table VI][m]	apprentissage continu, MobileNetV1, CORe50	ZCU102, FP16 : 413 ms ; 1,22 J par image	Jetson Nano : 33 ms ; 0,31 J par image	batch 1 contre 8 ; méthode `n/r`

8.3 Énergie par inférence : des ordres de grandeur

Sources des lignes nouvelles : MLPerf Tiny (Borras et al., 2022, Table 5)[PP] ; LP-YOLO Slim, 35,66 FPS à 2,91 W, soit 12,25 FPS/W (Gutierrez et al., 2026, §V-D)[PP] ; Karki et al., 34,05 FPS de bout en bout et 8,760 W estimés par Vivado (Karki et al., 2026, §IV-C, Table III)[PP] ; Alqahtani et al., 0,13 à 0,22 mWh par requête, latence de service web comprise (Alqahtani et al., 2024, §IV)[PP]. Pour mémoire, le YOLOv3-tiny de Zhai et al. donnerait environ 136 mJ par image (12,51 W divisés par 91,65 FPS, Zhai et al., 2023, Table 8), de portée non décrite. Ces calculs sont les nôtres *(déduit)*.

La table ne classe pas les plateformes. Trois écarts la dominent : la taille du réseau (de 88×88 pixels sur MCU à 640 sur Jetson), la portée de la mesure (outil, puce, carte, carte moins le repos), et le périmètre de l'inférence (noyau seul, bout en bout, service web). Le seul détecteur sur petite FPGA mesuré à la carte (LP-YOLO, 82 mJ) et les Orin mesurés par télémétrie (179 à 185 mJ) diffèrent sur ces trois points à la fois.

8.4 Pourquoi la comparaison reste ouverte

Pour comparer honnêtement un FPGA embarqué, une Jetson et un MCU, il faudrait, sur un même réseau :

— la même précision numérique et le même batch ;
— une précision mesurée sur chaque cible ;
— une latence de bout en bout, post-traitement compris ;
— une puissance de puce et de carte, avec le repos mesuré.

Le protocole de Schlosser et al. (2023) est le plus proche de cette grille dans le corpus : même YOLOv4, mAP mesurée sur la cible par un hôte séparé, puissance de carte absolue et relative (repos soustrait), quatre modes de puissance Jetson [§3, §4.1, §4.4]. Il lui manque l'égalité du post-traitement et une cible récente, et il ne porte que sur un modèle. MLPerf Tiny fixe aussi un protocole (latence

TABLE 5 – Énergie par inférence (ou par événement d'apprentissage) rapportée dans le corpus. Tâches, modèles et portées diffèrent : ces valeurs donnent des ordres de grandeur, pas un classement. Les valeurs marquées *(déduit)* sont calculées ici à partir de la puissance et du débit publiés.

Travail	Tâche, modèle	Plateforme	Énergie	Portée, source
TinyissimoYOLO	détection 88×88, 3 classes	MAX78000	196 µJ	MCU seul, §III
TinyissimoYOLO	idem	Apollo4b	6,1 mJ	MCU seul, §III
Manjunath	encodeur AE 120×120	Artix-7	0,25 mJ	estimée (Vivado), Table 5
ImageHD	clustering continu, CIFAR-10	ZCU104	0,97 mJ	estimée (Vitis), Table VII
MLPerf Tiny[PP]	classification CIFAR-10, FINN W1A1	Pynq-Z2	2,5 mJ	appareil sous test (Joulescope), Table 5
Castelino	CAE d'anomalies, hyperspectral	ZCU104	21,5 mJ	puce (PMBus), §IV-B
Castelino (baseline)	idem	Xavier NX (10 W)	≈ 79 mJ	jtop, Table VI *(déduit)*
LP-YOLO Slim[PP]	détection de véhicules, 256	PYNQ-Z1	≈ 82 mJ	carte (rail 12 V), §V-D *(déduit)*
Rey	YOLOv8n 640 INT8	Orin NX	179 mJ	télémétrie, Table 4
Rey	YOLOv8n 640 INT8	Orin Nano	185 mJ	télémétrie, Table 4
SkyNet[PP]	détection mono-objet	TX2 / Ultra96	≈ 200 / 290 mJ	`n/r`, Tables 5–6 *(déduit)*
Karki[PP]	YOLOv26n 416 INT8	ZCU104	≈ 257 mJ	estimée (Vivado), §IV-C, Table III *(déduit)*[m]
SATAY	YOLOv5n 640	ZCU104	317 mJ	non décrite, Table IV[m]
SATAY (baseline)	YOLOv5n 640	Jetson TX2	277 mJ	non décrite, Table IV[m]
Schlosser	YOLOv4 512 INT8	AGX Xavier / ZCU104	≈ 0,49 / 0,76 J	carte, Table 1 *(déduit)*
Alqahtani[PP]	YOLOv8 n à m, service web	Orin Nano	0,47 à 0,79 J	carte, repos soustrait, §IV *(déduit)*[m]
F-LIC	encodage 720p (compression apprise)	VCU118	1,41 J	non décrite, Fig. 6[m]
F-LIC (baseline)	idem	Xavier NX	1,43 J	non décrite, Fig. 6[m]
Chameleon	image d'apprentissage continu	ZCU102 / Jetson Nano	1,22 / 0,31 J	non décrite, Table VI[m]
O-FSCIL	ajout d'une classe (5 exemples)	GAP9	11,35 mJ	non décrite, Table IV
QLR-CL	événement d'apprentissage ($l = 27$)	VEGA	0,13 J	puce, Table IV
QLR-CL	idem	STM32L4	4,80 J	Table IV

à batch 1, énergie au Joulescope), mais l'article ne compare aucune autre plateforme (Borras et al., 2022, §1, §4.4)[PP]. Les meilleures baselines Jetson récentes (Rey, Alqahtani[PP]) n'ont pas de FPGA. Les travaux FPGA récents se comparent à des Jetson anciennes ou bridées (SATAY, SkyNet[PP], Castelino), à un GPU de bureau (Fata and Elmannai, 2026, Table 10), ou à rien (Karki[PP], LP-YOLO[PP]). Aucun article du corpus ne met une FPGA embarquée et une Jetson Orin, ni une FPGA et un MCU, dans un même banc d'essai.

9 Autoencodeurs sur FPGA

La bibliographie de départ supposait que les autoencodeurs (AE) sur FPGA étaient surtout étudiés sur des séries temporelles et en physique des particules, et peu sur des images. Les 15 fiches de l'axe, complétées par l'AE d'anomalie audio de MLPerf Tiny, permettent de préciser ce constat : l'image est désormais traitée, mais surtout en compression, et la détection d'anomalies visuelles sur FPGA se réduit à un seul cas, hyperspectral.

9.1 Physique des particules et signaux : la très faible latence

Govorkova et al. (2021) placent des AE et VAE au déclenchement de premier niveau du LHC, pour sélectionner des collisions anormales sans hypothèse sur la nouvelle physique. Trois choix sont transposables à d'autres domaines :

- le score d'anomalie (erreur quadratique) est calculé sur la puce, par une couche ajoutée à hls4ml [§IV] ;
- le VAE est réduit à son encodeur : le score s'obtient à partir de la divergence de Kullback-Leibler (D_{KL}) calculée sur (μ, σ), ce qui supprime l'échantillonnage et le décodeur [§IV] ;
- le choix entre QAT et PTQ se fait selon la stabilité du score d'anomalie, et non selon la perte d'entraînement [§VI].

Le VAE dense (encodeur seul) tient en 80 ns sur VU9P ; le seul AE convolutif, sur une entrée de 19×3 valeurs, prend 47 % des LUT et est rejeté [Table III]. Ces chiffres sont des estimations de Vivado HLS ; une seule validation sur carte est faite, et elle est qualitative [§VII].

Trois autres travaux restent sur des signaux :

- Govorkova et al. (2026) compressent des impulsions de calorimètre (32 échantillons en 2 valeurs) sur un PolarFire durci : 25 ns de latence en version « nano », sans bloc mathématique [§5.2]^m. La puissance est estimée par l'outil ;
- Que et al. (2024) sont les seuls à échantillonner en matériel (générateur Mersenne Twister suivi d'une transformée de Box-Muller), avec un latent de dimension 1, sur des séries d'ondes gravitationnelles [§III]. Une inférence dure 17 cycles (85 ns) ; les « 5,6 ns » mis en avant sont un débit inverse amorti sur 10 000 inférences [§IV-E]. La puissance est estimée par Vivado [§IV-E] ;
- Yue et al. (2024) demandent au même VAE de compresser à la source les traces d'un détecteur à pixels et de signaler les défauts du capteur. Comme au LHC, seul l'encodeur (latent 8) est embarqué, et le score d'anomalie est une KLD « clipée », sans échantillonnage [§2–3]. La synthèse hls4ml donne 15 ns de latence [Table 1], sans FPGA identifié ni puissance [§1, Conclusion]. Les données sont simulées et la détection reste faible sur les pixels morts : 10 % de sensibilité pour 90 % de rejet du fond [Table 2].

Enfin, Campos et al. (2024) quantifient un encodeur de compression de données de calorimètre (ECON-AE), dont seul l'encodeur est synthétisé [§7, Table 3].

9.2 Images : compression apprise et encodeurs

Compression apprise sur grosse FPGA. F-LIC (Sun et al., 2022) implante un codec d'images appris (VAE avec hyper-prior) en virgule fixe 8 bits sur VCU118. Il encode du 720p à 40,69 FPS

et du 1080p à 19,15 FPS, avec un PSNR de 29,94 dB à 0,2542 bpp [Fig. 6]^{m}. Deux techniques sont réutilisables pour un décodeur :

— deux poids partagent un même DSP (« casDSP ») ;
— la déconvolution saute les zéros insérés par le stride.

Deux limites : le codage entropique tourne sur le PC hôte [Fig. 1], et l'encodeur 720p occupe 4 146 DSP. L'énergie par image est comparable à celle d'une Jetson Xavier NX (1,41 contre 1,43 J en encodage 720p), pas meilleure, et la méthode de mesure n'est pas décrite [Fig. 6].

Compression apprise sur SoC, via la DPU. Mazouz et al. (2025)^{PP} portent un LIC (AE convolutif avec hyperprior) sur trois DPU B4096 d'une ZCU102. Un élève distillé à 160 canaux, élagué de 30 % et quantifié en précision mixte 8/32 bits, encode du 720p à 43,63 FPS [§III-A–D, Table VII] ; le décodage tombe à 30,5 FPS [Table X]. La couche GDN, qu'un ReLU remplace d'habitude, est gardée en virgule fixe 32 bits comme opérateur XIR personnalisé [§III-B, §III-D]. C'est le premier encodeur-décodeur d'images complet sur SoC du corpus, mais l'énergie annoncée (2,88 J par image à 43,6 FPS, soit environ 125 W *(déduit)*) est incompatible avec une ZCU102, et la méthode de mesure n'est pas décrite [Table VII]. En amont, Fang and Chen (2025)^{PP} ramènent la perte de BD-rate d'un Ballé2018 à GDN quantifié en INT8 de 30,0 à 6,30 % [Table I], mais en simulation seulement : aucune mesure FPGA n'apparaît dans le texte.

Compression à bord, sur VPU. La mission Φ-Sat-2 (Guerrisi et al., 2023) compresse les images par un AE convolutif d'un facteur 8 fixe, sans codage entropique [§II-A]. L'encodeur tourne à bord sur une VPU Myriad 2, pas sur un FPGA : 10,92 s par acquisition pour 1,4 W, contre 0,95 s sur un GPU GTX 1650 [Table V]^{m}. Le passage à la VPU ne coûte presque rien en qualité : SSIM de 0,9647 contre 0,9651 [§III-B–C].

Encodeurs d'images sur petite FPGA. Manjunath et al. (2021) ternarisent l'encodeur convolutif d'un agent d'apprentissage par renforcement (entrée 120×120 RGB, simulateur) et l'implantent en Verilog sur Artix-7. En 32 bits et 64 PE, une image coûte 0,8 ms et 0,25 mJ [Table 5]. La puissance est estimée par Vivado [§VIII-A], et le décodeur n'est pas implanté. Isik et al. (2023)^{PP} débruitent MNIST sur XCZU7EV (2,91 ms, 5,93 W « on-chip »). C'est la seule ligne de `benchmarks.csv` en confiance *low* : l'« accuracy » d'un débruiteur n'est pas définie, et le tableau comparatif est physiquement incohérent [Table II].

9.3 Détection d'anomalies sur SoC FPGA : surtout des signaux

Le seul cas d'images : artefacts hyperspectraux. Castelino et al. (2024) détectent les artefacts d'images hyperspectrales de CubeSat par un seuil sur l'erreur de reconstruction d'un CAE de 2,95 M de paramètres (entrée 144×144) [§III-B–D]. Encodeur et décodeur tournent en INT8 sur la DPU B4096 d'une ZCU104 : 4,0 ms et 21,52 mJ par inférence, 5,38 W lus sur les rails PMBus [§IV-B, Table VI]. La Jetson Xavier NX (mode 10 W, INT8) met 10,36 ms pour 7,6 W [Table VI], avec une puissance lue par jtop, de portée différente. Trois réserves limitent la portée du résultat :

— le seuil est choisi sur les données de test [§III-C] ;
— le jeu de test est très déséquilibré (837 anomalies pour 79 à 89 images normales) [Table IV] ;
— la PTQ seule donne un F1 nul ; il faut 50 epochs de fine-tuning pour revenir à 92,77 % [Table III, §IV].

Les autres AE d'anomalie portent sur des signaux.

— Malviya et al. (2022) détectent des anomalies de vibrations d'hélicoptère, encodées en scalogrammes 64×64, avec un AE convolutif de 8 755 paramètres sur PYNQ-Z2. Encodeur et décodeur sont dans deux bitstreams séparés (40 403 et 38 169 LUT), pour 2,1 + 3,8 ms

[Table 4, §IV-B]^{m}. La précision est remesurée sur le FPGA et baisse (85,02 % contre 88,8 % sur CPU) en virgule fixe 23 bits [§IV-B].

— Tsukada et al. (2020) (ONLAD) apprennent en ligne un AE à une couche cachée par OS-ELM (voir section 10). Leurs données sont tabulaires ou des images aplaties.

— Coutinho et al. (2019) : malgré le titre, l'« autoencodeur empilé » n'est qu'un classifieur MLP sur MNIST, sans reconstruction [§III-A, §V-D]. Leur comparaison met une puissance dynamique de quelques mW face à des puissances totales [Tables 16–17].

— Dans MLPerf Tiny, Borras et al. (2022)^{PP} implantent avec hls4ml l'AE dense de la tâche « anomaly detection », sur données audio sous-échantillonnées (128 entrées) [§3.3]. Sur Pynq-Z2, une inférence prend 0,019 ms et 30,1 μJ, mesurés au Joulescope sur l'appareil [Table 5, §4.4.2] ; l'AUC n'est pas dans le texte extrait.

Côté algorithmes d'anomalie visuelle. Le survey de Cui et al. (2023) classe les méthodes non supervisées d'inspection industrielle en cinq familles [§III, Table 2]. Sur MVTec AD, les méthodes à banque de caractéristiques ou à flots dominent : AUROC image de 99,1 pour PatchCore et de 99,4 pour FastFlow [Table 10, reconstruite par Docling]. Ce sont aussi les plus coûteuses en mémoire, alors que les méthodes par reconstruction (AE) sont rapides en inférence [§IV]. Le survey ne chiffre aucun coût matériel, ne traite ni le réglage du seuil ni le compromis entre fausses alarmes et alarmes manquées au-delà de son énoncé [§VI-B], et exclut les scènes avec personnes [§II-A]. Sur un jeu réel (huile alimentaire), PatchCore tombe à 92,2 d'AUROC [Table 12].

9.4 Bilan

Trois constats se dégagent du tableau 6 :

— **l'image est traitée, surtout en compression** : sur grosse FPGA (F-LIC), sur SoC via la DPU (LIC distillé, arxiv-2503.04832^{PP}), sur VPU (Φ-Sat-2), ou à petite échelle sur petite FPGA (Manjunath, arxiv-2301.07050^{PP}) ;

— **la détection d'anomalies visuelles par AE sur FPGA se réduit à un cas** : le CAE d'artefacts hyperspectraux sur ZCU104, avec un seuil réglé sur le test. Aucun travail ne la fait sur des images RGB réelles ou sur un benchmark standard comme MVTec AD ; les autres AE d'anomalie portent sur des signaux, de l'audio ou des données tabulaires. Sur les 19 lignes « anomalie » de `benchmarks.csv`, 8 portent sur un SoC ou une petite FPGA (Zynq-7000, Zynq UltraScale+, Artix-7), et une seule sur des images de caméra ;

— **l'AE complet tient désormais sur un SoC UltraScale+**, encodeur et décodeur compris, via la DPU (ZCU104, ZCU102). Les travaux plus anciens implantaient l'encodeur seul, ou coupaient le réseau en deux bitstreams (Malviya). Le coût des déconvolutions sur une puce plus petite (KV260) ou en dataflow (hls4ml, FINN) reste non mesuré.

Le survey de Cui et al. (2023) désigne les candidats à porter : les méthodes par reconstruction sont les moins chères en inférence, celles à banque de caractéristiques les plus précises sur MVTec AD, mais aussi les plus gourmandes en mémoire [§IV, Table 10].

Table 6 – Autoencodeurs du corpus : donnée traitée, partie implantée et type de mesure.

Travail	Donnée	Partie en matériel	Puce	Puissance
AE du LHC [a]	tabulaire 19×3	AE complet ; VAE encodeur seul	VU9P	non mesurée
PicoCal [b]	impulsions 1D	encodeur	PolarFire	estimée
VAE de Que [c]	séries temporelles	VAE complet (latent 1)	U250, Z7045	estimée
ECON-AE [d]	calorimètre	encodeur	`n/r` (voir fiche)	non mesurée
VAE smart pixel [e]	traces simulées (105 valeurs)	encodeur (synthèse hls4ml)	`n/r`	non mesurée
F-LIC [f]	images 720p et 1080p	VAE hyper-prior, sans codage entropique	VCU118	méthode non décrite
LIC distillé [g]	images 720p (Kodak tuilé)	encodeur et décodeur, 3 DPU	XCZU9EG (ZCU102)	méthode non décrite, incohérente
DRAQ [h]	images (Kodak)	aucune (quantification simulée)	—	—
Φ-Sat-2 [i]	images satellite	encodeur	VPU (pas de FPGA)	méthode non décrite
Manjunath [j]	images 120×120 (simulateur)	encodeur	Artix-7	estimée
CAE débruiteur [k]	MNIST bruité	AE complet	XCZU7EV	méthode non décrite
CAE d'artefacts [l]	images hyperspectrales 144×144	AE complet sur DPU	XCZU7EV (ZCU104)	rails PMBus (5,38 W)
Malviya [m]	vibrations en scalogrammes	AE, 2 bitstreams	XC7Z020	méthode non décrite
MLPerf Tiny AD [n]	audio (ToyADMOS)	AE dense complet	XC7Z020, Artix-7	énergie de l'appareil (Joulescope)
ONLAD [o]	tabulaire, MNIST aplati	AE OS-ELM + apprentissage	XC7Z020	carte (3,1 W)
SSAE [p]	MNIST	MLP (encodeurs empilés)	XC6VLX240T	dynamique

[a] Govorkova et al. (2021) ; [b] Govorkova et al. (2026) ; [c] Que et al. (2024) ; [d] Campos et al. (2024) ; [e] Yue et al. (2024) ; [f] Sun et al. (2022) ; [g] Mazouz et al. (2025)[PP] ; [h] Fang and Chen (2025)[PP] ; [i] Guerrisi et al. (2023) ; [j] Manjunath et al. (2021) ; [k] Isik et al. (2023)[PP] ; [l] Castelino et al. (2024) ; [m] Malviya et al. (2022) ; [n] Borras et al. (2022)[PP] ; [o] Tsukada et al. (2020) ; [p] Coutinho et al. (2019).

10 Apprentissage embarqué et continu sur FPGA

Un modèle figé ne s'adapte pas à la dérive des données. L'apprentissage sur l'appareil y répond, mais la rétropropagation coûte cher en mémoire et en énergie. Le corpus compte six travaux qui apprennent sur FPGA, en deux familles :

- sans rétropropagation : ONLAD (OS-ELM), les accélérateurs SLDA et SONN de Piyasena, ImageHD (calcul hyperdimensionnel) ;
- avec rétropropagation : EF-Train et Chameleon. Elle y est lente et en flottant (FP32 ou FP16).

Tous apprennent sur un SoC Zynq (PYNQ-Z1, ZCU102 ou ZCU104), et tous font de la classification, du clustering ou de l'anomalie non visuelle.

ONLAD : apprentissage séquentiel sans rétropropagation. Tsukada et al. (2020) entraînent en ligne un AE à une couche cachée par OS-ELM :

- les poids d'entrée sont aléatoires et figés, et seuls les poids de sortie sont mis à jour par moindres carrés récursifs [§2.2, §3.3] ;
- à batch 1, la seule inversion matricielle devient une division scalaire, sans changer la solution [§3.1] ;
- un facteur d'oubli léger gère la dérive : l'AUC passe de 0,55–0,59 sans oubli à 0,78–0,90 sous dérive simulée [Table 5].

Le cœur (Vivado HLS, virgule fixe 32 bits, 100 MHz) tourne sur PYNQ-Z1. C'est l'une des rares mesures de carte FPGA du corpus : 3,1 W au wattmètre, ARM compris [§6.4]. Limites : les AUC sont évaluées en TensorFlow et non sur le cœur en virgule fixe [§5–6] ; la dérive est simulée par permutation de classes ; les baselines CPU et GPU, en TensorFlow à batch 1, sont dominées par les surcoûts d'appel, ce que les auteurs reconnaissent [§6.2.1].

SLDA : un CNN figé sur la DPU, la dernière couche apprise en PL. Piyasena et al. (2021) remplacent la dernière couche d'un ResNet18 figé par un Deep SLDA, qui met à jour en flux une moyenne par classe et une covariance partagée [§III-A]. Le ResNet18 tourne sur une DPU B4096 (Vitis AI), le SLDA sur un accélérateur Vitis HLS à 200 MHz, sur la même ZCU102 [§IV, §V-E] :

- le système complet apprend à presque la vitesse de l'inférence : 168,67 FPS contre 170,20 pour la DPU seule, avec 78,79 % de précision (objet) sur CoRE50, pour 9,00 W contre 8,16 W et 0,056 contre 0,048 J par image [Table V, reconstruite par Docling] ;
- la précision est donc bonne, mais la puissance est lue sur les rails par l'outil de la carte, sans portée précisée [§V-E]. L'inversion de la covariance reste sur l'ARM, et son coût n'apparaît pas dans les latences [§IV-D] ;
- l'accélérateur seul est 24 fois plus rapide que l'ARM A53 en variante PlasticCov (47,85 μs par échantillon), mais l'A53 bat le FPGA en StaticCov (2,80 contre 4,76 μs) [Table IV]. L'avantage du FPGA dépend de la variante.

La thèse de Piyasena (2021) ajoute un accélérateur de réseau auto-organisé (SONN), abandonné parce que sa mémoire et sa latence croissent avec le nombre de classes [§4.1]. Elle ajoute aussi un pilote Linux qui supprime environ 150 μs de latence d'hôte due à OpenCL [§4.4].

ImageHD : calcul hyperdimensionnel sur un extracteur figé. Arockiaraj et al. (2026) combinent un MobileNetV2 INT8 figé, un encodage en hypervecteurs binaires et une mémoire de clusters bornée, fusionnée par un kMeans++ simplifié [§IV]. L'accélérateur streaming (Vitis HLS) tient sur une ZCU104 à 300 MHz : 0,27 ms et 0,97 mJ par image sur CIFAR-10 [Tables VI–VII]. Deux réserves fortes :

- la précision de clustering est très basse (0,27 sur CIFAR-10, 0,19 sur CORe50), alors que l'article parle d'une « strong accuracy » [Table VI, §IX] ;

— latence et puissance du FPGA viennent de l'analyseur de Vitis, c'est-à-dire d'une estimation, alors que CPU et GPU sont mesurés (powertop, nvidia-smi). Les gains de 258× à 383× en énergie mélangent estimation et mesure [§VII-D].

La rétropropagation sur SoC FPGA existe, mais lente et en flottant.

— **EF-Train** (Tang et al., 2022)[PP] exécute la propagation avant, la rétropropagation et la mise à jour des poids sur un même noyau de convolution, en FP32 et à 100 MHz [§3, §5.2, §6]. Une réorganisation des données en DRAM supprime l'essentiel du coût de réallocation [§4, Tables 3–5]. Sur ZCU102, un petit CNN CIFAR-10 s'entraîne à 28,15 GFLOPS et 2,08 ms par image (batch 128). La précision atteint 64,82 % contre 65,22 % sur V100 [Table 7, §6.3]. La puissance (6,89 W) vient du rapport Vivado [§6.3], et il n'y a ni stratégie contre l'oubli, ni baseline Jetson.

— **Chameleon** (Aggarwal et al., 2024) rejoue des activations latentes de MobileNetV1 depuis deux tampons, l'un sur la puce et l'autre hors puce [§III]. Avec 3,5 Mo de tampons, il atteint 79,48 % sur CoRE50, contre 79,07 % pour Latent Replay avec 48 Mo [Tables II et VI]. Son accélérateur FP16 sur ZCU102 (150 MHz, une couche à la fois) met 413 ms et 1,22 J par image d'apprentissage. Une Jetson Nano, à batch 8, met 33 ms et 0,31 J [§IV-A, Table VI]. Les auteurs reconnaissent que le GPU est plus rapide [§V-D] ; la méthode de mesure de l'énergie n'est pas décrite.

Venieris et al. jugeaient l'entraînement sur FPGA « à peine exploré » en 2018 (Venieris et al., 2018, §3.2). Le constat a changé : il est fait, mais aucune rétropropagation sur FPGA du corpus n'est en INT8, ni assez rapide pour un flux vidéo.

Table 7 – Apprentissage sur FPGA dans le corpus : ce qui est appris, en quelle précision, et comment la puissance est obtenue.

Travail	Ce qui est appris	Arithmétique	Carte	Coût d'apprentissage	Puissance
ONLAD [a]	poids de sortie d'un AE (OS-ELM)	fixe 32 bits	PYNQ-Z1	`n/r` (AUC en logiciel)	carte, wattmètre (3,1 W)
SLDA [b]	dernière couche (SLDA), CNN figé sur DPU	fixe 16/32 bits	ZCU102	47,85 μs / échantillon (PlasticCov)	rails, portée non dite
SONN [c]	réseau auto-organisé sur caractéristiques	INT8	ZCU102	croît avec les classes	rails, portée non dite
ImageHD [d]	clusters d'hypervecteurs	binaire (HDC)	ZCU104	0,97 mJ / image	estimée (Vitis)
EF-Train [e]	tout le CNN (rétropropagation)	FP32	PYNQ-Z1, ZCU102	2,08 ms / image (batch 128)	estimée (Vivado)
Chameleon [f]	haut du MobileNetV1 (rétropropagation)	FP16	ZCU102	413 ms, 1,22 J / image	méthode non décrite

[a] Tsukada et al. (2020) ; [b] Piyasena et al. (2021) ; [c] Piyasena (2021) ; [d] Arockiaraj et al. (2026) ; [e] Tang et al. (2022)[PP] ; [f] Aggarwal et al. (2024).

Références hors FPGA. Plusieurs travaux fixent des ordres de grandeur qu'un apprentissage continu sur FPGA devrait battre ou justifier :

— QLR-CL sur VEGA, un ASIC PULP 22 nm : 62 mW de puce (Ravaglia et al., 2021, §V-D). L'étage adaptatif y est rétropropagé en FP32 (Ravaglia et al., 2021, §III-C) ;

— O-FSCIL sur le MCU GAP9 : 11 à 23 mJ par nouvelle classe (Wibowo et al., 2024, Table IV) ;

— l'entraînement INT8 sous 256 KB de SRAM d'un STM32F746 (Lin et al., 2022, §3.2), et TinyCL, un ASIC 65 nm qui rétropropage en virgule fixe 16 bits (86 mW de synthèse) (Ressa et al., 2024, §III-A, §IV-B)[PP].

Tous sont détaillés en section 11. La rétropropagation en précision réduite n'existe donc dans le corpus que sur MCU et sur ASIC.

Ce qui manque. Toutes les fiches d'apprentissage sur FPGA font de la classification, du clustering ou de l'anomalie non visuelle. La détection incrémentale existe, mais sans FPGA :

- RILOD (Li et al., 2019)[PP] ajoute une classe à un RetinaNet par distillation, sans les anciennes données. Sur Pascal VOC (19+1), la mAP est de 65,0 contre 74,7 avec toutes les données [Table 1, reconstruite]. Sur Jetson TX2, l'apprentissage prend 233 s, sans puissance rapportée [Table 7, reconstruite] ;
- la Latent Distillation de Pasti et al. (2024)[PP] fige le backbone d'un NanoDet partagé entre maître et élève. Elle annonce 56 % de FLOPs en moins par mise à jour que les méthodes de distillation [§5.1], mais aucune mesure sur cible : le portage tinyML est renvoyé aux travaux futurs [§6].

`benchmarks.csv` n'a toujours aucune ligne qui croise détection d'objets et apprentissage continu. Trois questions restent ouvertes dans le corpus :

- une rétropropagation de la queue d'un réseau en précision réduite (INT8, sparse update) sur SoC FPGA, l'étage figé sur la DPU ou en logique programmable, avec l'énergie mesurée ;
- l'incrémental de classes en détection (tête YOLO ou NanoDet) sur FPGA, avec la latence et l'énergie de mise à jour ;
- un apprentissage en ligne compatible avec la certification : aucune fiche d'apprentissage continu ne traite la non-régression, le versionnage ou le retour arrière.

11 Apprentissage continu sur microcontrôleurs et ASIC

Replay latent quantifié sur ASIC PULP (QLR-CL). Ravaglia et al. (2021) découpent un MobileNet-V1 (entrée 128×128) en deux étages [§III-C] :

- un *étage figé*, quantifié en INT8 ;
- un *étage adaptatif*, réentraîné en FP32 par rétropropagation, sur un mélange de nouvelles images et d'activations intermédiaires stockées en 8 bits (*latent replays*).

Sur Core50, stocker ces activations en UINT-8 ne coûte que 0,26 point face au FP32 [§V-B]. En dessous de 6 bits, l'apprentissage ne converge plus. Sur VEGA (ASIC 22 nm, 62 mW mesurés sur la puce), un événement d'apprentissage coûte 2,07 s et 0,13 J si seule la dernière couche est réentraînée. Si l'on remonte à la couche 23, il faut 877 s et 54,3 J [Table IV]. Avec le même code, VEGA est en moyenne 65 fois plus rapide et 37 fois plus économe qu'un STM32L4 [§V-E]. La comparaison au Snapdragon 845, elle, suppose une enveloppe de 4 W non mesurée [§V-E].

Apprentissage incrémental de classes à partir de peu d'exemples, sur GAP9 (O-FSCIL). Wibowo et al. (2024) figent un MobileNetV2 et ajoutent chaque nouvelle classe sous la forme d'un prototype : la moyenne des caractéristiques de ses 5 exemples, sans rétropropagation [§IV, §V-B]. Sur GAP9, en INT8, ajouter une classe coûte 11,35 mJ pour 62,44 % de précision moyenne sur CIFAR-100. Le plus gros backbone atteint 66,51 % pour 22,75 mJ [Tables II et IV]. Le titre combine deux configurations : les 68,62 % viennent d'un ResNet-12 FP32 sur GPU, les 12 mJ du plus petit modèle déployé [Table II].

Entraînement sous 256 KB (MCU). Lin et al. (2022) rendent possible le fine-tuning d'un CNN réellement quantifié en INT8 sur un STM32F746 (320 KB de SRAM) [§2, §3.1]. La méthode combine trois éléments :

- une mise à l'échelle des gradients (QAS), qui rattrape le FP32 : 73,5 contre 73,3 % de précision moyenne [Table 1] ;

— une mise à jour creuse, qui n'apprend que des biais et des sous-ensembles de couches ou de canaux, choisis hors ligne sous budget mémoire [§2] ;
— un graphe de rétropropagation compilé et élagué à la compilation.

Sur Visual Wake Words, le MCU atteint 89,1 % avec 206 KB de SRAM mesurés [§3.2]. C'est du transfert d'apprentissage sur un jeu fixe, sans mesure d'oubli ; l'énergie n'est pas rapportée, et les précisions sur les autres jeux sont simulées sur GPU [§3.1].

Distillation de données et croissance du modèle. Rüb et al. (2024) remplacent le tampon de replay par un jeu distillé (environ 1 % des données) et agrandissent le réseau quand la précision passe sous un seuil [§III-B, §V-G]. Rien n'est déployé : les coûts sont des FLOPs relatifs. La « perte de 1 % » du résumé compare le modèle adaptatif au plus grand modèle, tous deux sous distillation. Face à l'entraînement cumulatif, la perte est forte : 58,2 contre 85,4 % en fin de séquence sur CIFAR-10 [§V-B]. Un réseau dont l'architecture croît se prête mal à un FPGA à architecture fixe *(déduit)*.

Accélérateur de CL en ASIC (TinyCL). Ressa et al. (2024)[PP] exécutent propagation avant, rétropropagation et mise à jour sur une même unité de MAC, en virgule fixe 16 bits, avec un replay GDumb [§III-A, §III-C–F]. Synthétisé en 65 nm, l'ASIC consomme 86 mW d'après la synthèse, sans mesure sur silicium ; la mémoire de replay prend 80 % de la surface et 76 % de la puissance [§IV-B, Fig. 7]. Une epoch dure 1,76 s contre 103 s sur une Tesla P100 à batch 1, une baseline sous-utilisée [§IV-C]. Le modèle reste un jouet (deux convolutions et une couche dense, CIFAR-10) [§IV-A].

Algorithmes et surveys. Le survey de Masana et al. (2020)[PP] compare treize méthodes de class-IL sur GPU, en classification. Sur CIFAR-100 en 10 tâches [Tables 2 et S2] :
— la meilleure méthode sans exemplaires, LwF, atteint 30,2 % ;
— le simple fine-tuning avec 2 000 exemplaires atteint 37,9 % ;
— l'entraînement joint, borne haute, atteint 66,3 %.

Il exclut par construction les méthodes online, à architecture croissante ou pré-entraînées, celles qu'utilise le matériel contraint [§2.3]. Le survey de Luo et al. (2024) traite l'apprentissage continu embarqué en une page, sans aucun chiffre système [§5.2].

Bilan transversal. Chaque travail d'apprentissage sur cible ne mesure qu'une partie des familles de plateformes :
— QLR-CL : ASIC et MCU ;
— O-FSCIL et l'entraînement sous 256 KB : MCU ;
— TinyCL : ASIC (synthèse), face à un GPU ;
— ImageHD : FPGA, face au CPU, au GPU et à une Jetson non optimisée ;
— ONLAD : FPGA, face au CPU et au GPU ;
— SLDA : FPGA, face à l'ARM A53 et à une Jetson Xavier NX (Piyasena et al., 2021, Table IV) ;
— Chameleon : FPGA, Jetson Nano et EdgeTPU simulé (Aggarwal et al., 2024, Table VI) ;
— EF-Train : FPGA seul ; le V100 ne sert qu'à vérifier la convergence.

Aucun algorithme d'apprentissage continu n'est mesuré à la fois sur MCU, FPGA et Jetson, et aucun papier n'a à la fois une FPGA et un MCU. `benchmarks.csv` compte 36 lignes d'apprentissage sur cible (continu, entraînement ou fine-tuning, ONLAD compris) : 26 sur FPGA, 5 sur MCU, 3 sur ASIC et 2 sur Jetson, ces dernières venant toutes de Chameleon (Jetson Nano). Les mesures de SLDA sur Xavier NX ne figurent que dans la colonne `baseline`.

12 Analyse critique : ce que les chiffres publiés permettent de conclure

12.1 La puissance : rarement mesurée sur la carte, presque jamais sur la puce FPGA

Recompte de `benchmarks.csv` sur les 216 lignes mesurées sur FPGA (44 articles) :

TABLE 8 – Portée de la puissance dans les lignes FPGA de `benchmarks.csv` (01/10/2026).

Portée	Lignes	Origine
board (carte au wattmètre)	46	9 articles : Zhang 2024 10, FINN 8, FINN-R 8, Elastic-DF 8, LP-YOLO 6, Schlosser 3, Zhang 2015, ONLAD et CoDeNet 1 chacun
chip	1	Castelino (rails PMBus de la ZCU104)
board-dynamic	0	—
estimated (outil)	16	EF-Train 6, Manjunath 4, ImageHD 3, Karki 2, Que 2024 1
unspecified	52	dont SATAY 14, Piyasena 16 (deux articles), Zhai 8
puissance non rapportée (`n/r`)	101	—

Un peu plus de la moitié des lignes FPGA (115 sur 216) ont une puissance, et 47 seulement une puissance mesurée, issues de 10 articles. Les 18 lignes *chip* de la base viennent, sauf une, de Jetson Orin (12, Rey et al., 2025), du Raspberry Pi 5 (2) et de l'ASIC VEGA (3). La seule ligne *chip* FPGA est celle de Castelino et al. (2024, §IV-B) : 5,38 W actifs pour 4,6 W au repos. Les lignes *board* ne sont plus surtout anciennes : à côté de FINN, FINN-R, Zhang 2015 et ONLAD, elles viennent de Zhang et al. (2024), Alonso et al. (2021) (deux Alveo, hors embarqué), Schlosser et al. (2023) et Gutierrez et al. (2026)[PP]. Une seule paire puce / carte existe sur une même FPGA, et la puce y est estimée : 7,8 W par Vivado contre 12,3 W au wattmètre sur KC705 (Zhang et al., 2024, Table 3). Seuls Schlosser et al. donnent aussi la puissance relative, repos soustrait, que le CSV ne reprend pas faute de ligne *board-dynamic* : 7,0 W sur ZCU104 pour 23 W absolus [Table 1].

Plusieurs comparaisons publiées mettent face à face des portées différentes :

- une puissance dynamique de quelques mW contre des puissances totales (Coutinho et al., 2019, Table 16) ;
- la carte entière contre le CPU seul et le GPU seul (Tsukada et al., 2020, §6.4) ;
- le TDP d'un Xeon pris comme puissance mesurée (Zhang et al., 2015, §5.2) ;
- une estimation de l'outil contre des mesures par nvidia-smi (Arockiaraj et al., 2026, §VII-D) ;
- les rails PMBus de la puce contre jtop sur une Xavier NX bridée à 10 W (Castelino et al., 2024, Table VI) ;
- une puissance estimée par Vivado présentée comme « 3x moins » que celle de SATAY (Karki et al., 2026, Résumé, Table III)[PP].

Le survey de Shuvo et al. (2023) rappelle, dans sa partie sur les conditions de test, qu'un processeur peut afficher une très faible consommation en omettant les accès à la mémoire externe, et qu'il faut séparer résultats simulés et mesurés [§IX-I].

12.2 Des métriques qui flattent

- **Latence amortie** : un débit inverse présenté comme une latence (Que et al., 2024, §IV-E).
- **GOp/s par DSP** : cette métrique efface les écarts de précision (de W1A8 à W32A32) et de carte (Montgomerie-Corcoran et al., 2023, Table III). Venieris et al. la jugeaient valable seulement dans une même famille de FPGA (Venieris et al., 2018, §3.1). Le « +29 % face à Vitis AI » de Yan et al. (2024) est un GOPS/DSP face à un YOLOv3 d'un autre travail, pas une mesure de Vitis AI sur la même carte [Table 2].

— **Gains de surface face à des designs plus rapides** : « −26 % de FF, −15 % de DSP » face à des accélérateurs de 10,5 à 62,9 GOPS, pour 2,33 GOPS et un autre modèle (Kim and Kim, 2025, Table 9).
— **GOPS/W dynamiques** face à des GOPS/W totaux (Zhai et al., 2023, Table 8).
— **GOPS projetés** sur un FPGA hypothétique, comparés à des GOPS mesurés (Coutinho et al., 2019, Tables 16–17).
— **GOp/s seul**, sans FPS, pour un détecteur (Blott et al., 2018, Table 5) ; « 10,65x » en GOp/s entre des UNet de tailles différentes, à FPS voisins (Toupas et al., 2024, §V-D)[PP].
— **Gains face à un CPU ou un GPU de bureau**, de portée de puissance non dite : « 70x / 37x », alors que le GPU fait 44,64 FPS contre 24,3 sur la KV260 (Fata and Elmannai, 2026, Table 10) ; gains d'énergie de 258 à 383 fois qui mêlent une estimation FPGA et des mesures CPU et GPU (Arockiaraj et al., 2026, Table VII).
— **Conclusion contredite par son propre tableau** : 2,6 FPS/W présentés comme efficaces, moins que quatre des cinq travaux cités (Al Amin and Obermaisser, 2025, §IV-B, Table I).
— **Chiffres incohérents entre résumé et texte** : 35x contre 58x (Ressa et al., 2024, §I-B, §IV-C)[PP] ; perte de précision de ≈ 5 % contre ≈ 7 % (Karki et al., 2026, Résumé, §IV-B)[PP].
— **Précisions non comparables** : IoU d'un seul objet (DAC-SDC, SkyNet[PP]), AP à une classe (Zhai), « accuracy » sur 72 images (Al Amin), « accuracy » non définie d'un débruiteur (arxiv-2301.07050[PP], seule ligne en confiance *low*), mAP sur des véhicules filtrés de VOC (LP-YOLO[PP]), mAP sur un jeu privé à une cible (Rey et al., 2025, Table 4).

Le recompte confirme cette dispersion. Les 87 lignes FPGA de détection (17 articles) emploient 14 libellés de précision différents, et 12 n'ont aucune précision. Seules 22 portent une précision sur COCO, dont 14 pour SATAY, quantification simulée par ONNX Runtime ; six d'entre elles seulement (Schlosser, Karki[PP]) sont mesurées sur la carte de façon établie. Sur les 216 lignes FPGA, le FPS manque 132 fois, la latence 88 fois, la puissance 101 fois et la précision 48 fois. Shuvo et al. le formulent ainsi : « EI requires a universal and comprehensive set of quality metrics to execute meaningful comparisons of DL models, optimization algorithms, and hardware platforms » [§XIV-C]. La grille de Venieris et al. (débit, latence à batch 1, ressources, puissance, précision, mémoire) existe depuis 2018 ; aucune fiche du corpus ne la reprend en entier.

12.3 La précision est rarement remesurée sur la cible

Quantifier et compiler un réseau change sa précision, et la plupart des articles ne la remesurent pas sur la carte :

— AP identique en FP32 et en INT16 à trois décimales (Zhai et al., 2023, Table 8) ;
— quantification simulée par ONNX Runtime (Montgomerie-Corcoran et al., 2023, §IV-A) ;
— comparaison au C++ généré, pas au FPGA (Aarrestad et al., 2021, §8) ;
— AUC mesurées en TensorFlow (Tsukada et al., 2020, §5) ;
— seules des pertes relatives de mAP sont données (Ding et al., 2019, §6.2) ;
— $-1,5$ point de mAP VOC, sans dire si la mesure est faite sur la carte (Yan et al., 2024, Table 2) ;
— écart entre le modèle QAT et le modèle flottant non rapporté (Gutierrez et al., 2026)[PP] ;
— mAP identiques sur Jetson Nano et Orin Nano, sans preuve d'une mesure par carte (Alqahtani et al., 2024, §IV-C)[PP].

Quand la mesure sur cible est faite, elle révèle une perte, parfois forte et imprévisible (table 9).

Sources : Schlosser et al. (2023), Karki et al. (2026)[PP], Rey et al. (2025), Feng et al. (2022), Castelino et al. (2024), Zhang et al. (2019)[PP], Toupas et al. (2024)[PP], Malviya et al. (2022) et Coutinho et al. (2019). Trois constats :

— les deux seuls détecteurs mesurés sur une carte FPGA (Schlosser, Karki[PP]) perdent jusqu'à 30 points de mAP@0.5 :0.95 selon le modèle et le jeu. Chez Karki et al., les écarts ne s'expliquent pas : YOLOv11n perd 2 points sur VOC, YOLOv8n en perd 15 sur VOC et 30

TABLE 9 – Pertes de précision mesurées après quantification ou compilation. La mesure est faite sur la cible chez Schlosser, Karki, Rey, Feng, Malviya et Coutinho ; pour Castelino, SkyNet et SMOF, le texte extrait ne le précise pas.

Travail	Modèle, jeu	Avant → après	Source
Schlosser	YOLOv4, COCO	Vitis AI Quantizer : −2,2 % de mAP, la compilation DPU en retire encore ; TensorRT INT8 38,6 contre DPU INT8 37,5 (mAP@0.5 :0.95)	§4.3, Table 1
Karki[PP]	YOLOv8n, KITTI / VOC	0,64 → 0,34 ; 0,60 → 0,45 (CPU contre FPGA INT8)	Table II
Karki[PP]	YOLOv11n, VOC / COCO / personnes	0,59 → 0,57 ; 0,31 → 0,29 ; 0,75 → 0,71	Table II
Rey (Jetson, hors FPGA)	YOLOv8n / v8s, jeu privé	TensorRT INT8 : 84,2 → 71,9 ; 86,1 → 79,7 (mAP50-95)	Table 4
Feng (NCS2, hors FPGA)	YOLOv3-tiny, vidéo	OpenVINO : confiance moyenne 0 % contre 59,7 % sur Jetson	Table 3
Castelino	CAE d'anomalies	INT8 en PTQ seule : F1 0 ; après fine-tuning 92,77 (FP32 94,05)	Table III
SkyNet[PP]	SkyNet, DAC-SDC	IoU 0,741 → 0,727 (FM9/W11) → 0,680 (FM8/W10)	Table 7
SMOF[PP]	YOLOv8n, COCO	37,10 → 35,10 (bfp8)	Table III
Malviya	CAE de vibrations	88,8 % sur CPU → 85,02 % sur FPGA	§IV-B
Coutinho	SSAE, MNIST	93,4 % → 93,38 %	§V-D1

sur KITTI. Les modifications pour le DPU (ReLU, attention approchée) coûtent en plus ≈ 3 points en moyenne [§IV-B, Table I] ;

— les auteurs minimisent parfois la perte qu'ils mesurent : Rey et al. concluent « without a significant loss of accuracy » [§6] malgré 12,3 points perdus par YOLOv8n en INT8 ;

— la perte dépend de la chaîne autant que du réseau : la PTQ seule effondre l'autoencodeur de Castelino et al., et TensorRT garde une meilleure mAP que Vitis AI sur le même YOLOv4.

Pour un détecteur de personnes certifiable, cet écart entre le modèle évalué et le modèle déployé est aussi un problème de preuve *(déduit)*.

12.4 Baselines GPU embarqué faibles ou absentes

Quand un article FPGA se compare à un GPU embarqué, la baseline est souvent ancienne, bridée ou sous-optimisée :

— TX2 sans TensorRT mentionné (Ding et al., 2019, Table 2) ;
— TX2 de 2017, framework `n/r` (Montgomerie-Corcoran et al., 2023, Table IV) ;
— Jetson Nano reprise d'un autre article, avec un autre modèle (Zhai et al., 2023, Table 7) ;
— Jetson Nano en FP32 (Zhang et al., 2024, §5.3) ;
— Xavier NX bridée à 10 W, puissance par jtop (Castelino et al., 2024, §IV, Table VI) ;
— Jetson Nano à batch 8 contre ZCU102 à batch 1 (Aggarwal et al., 2024, Table VI) ;
— Jetson AGX Orin exécutée en PyTorch à batch 1 (Arockiaraj et al., 2026, §VII-C) ;
— aucune baseline GPU embarqué : Kim et al., Karki et al.[PP], LP-YOLO[PP], SMOF[PP].

Les protocoles équitables du corpus restent rares. fpgaConvNet face au Tegra X1 (2017), à budget de puissance égal, et DAC-SDC (2018), mesuré par les organisateurs, sont anciens. Le seul protocole récent est celui de Schlosser et al. (2023) (ZCU104 contre AGX Xavier, 2023), et la Jetson y gagne en latence, en précision et en FPS/W [Table 1]. Aucune source du corpus ne compare une FPGA embarquée à une Jetson Orin.

12.5 Reproductibilité

Aucune des 101 fiches ne mentionne de bitstream publié. Parmi la cinquantaine de fiches expérimentales, une dizaine publient ou annoncent un code propre lié au FPGA : Magalhães, FINN-R en partie, la branche hls4ml de l'AE du LHC, REQ-YOLO (probablement la partie entraînement), DAC-SDC (code des meilleures équipes), le streamer de FCMP, l'extension HAWQ de Campos, SMOF[PP] (GitHub, Toupas et al., 2024), Chameleon (GitHub annoncé, Aggarwal et al., 2024) et SkyNet[PP] (lien annoncé, absent du texte extrait). Les détecteurs les plus proches du cas d'application ne publient rien : Schlosser et al., Karki et al.[PP], LP-YOLO[PP], ni Yan, Zhang 2024, Kim ou Fata. Réserve : la lecture se limite au texte extrait, et les liens en note de bas de page sont souvent perdus.

13 Zones non couvertes et pistes de recherche

Les gaps ci-dessous viennent de `gaps.md` (8[e] passage, 01/10/2026), établi sur les 101 fiches de la base et les 299 lignes de `benchmarks.csv`, dont 216 sur FPGA. La carte compte 36 gaps, dont 29 étayés. Un gap *étayé* est constaté dans plusieurs fiches ; cela ne prouve pas qu'il est absent de la littérature. Un gap à l'état d'*hypothèse* ne repose que sur l'absence de papiers dans la base. Le tableau 10 reprend les gaps d'architecture, de mesure et d'apprentissage ; le tableau 11 ceux de sûreté et de détection de personnes, deux axes ajoutés à la base avec le thème de thèse.

Trois gaps du passage précédent ont changé, parce qu'une fiche nouvelle les contredit :

— « autoencodeur complet sur une petite FPGA » est supprimé. Un CAE complet, score par erreur de reconstruction, tourne en INT8 sur le DPU d'une ZCU104 (Castelino et al., 2024, §IV-B, Table VI), et une compression apprise place encodeur et décodeur sur ZCU102 (Mazouz et al., 2025, Table X)[PP]. Le coût des déconvolutions sur une puce plus petite passe dans le gap de l'anomalie visuelle ;

— « les apprentissages sur FPGA évitent la rétropropagation » devient « rétropropagation lente, en flottant, énergie non mesurée ». EF-Train fait FP / BP / WU en FP32 sur ZCU102, à 2,08 ms par image pour un petit CNN CIFAR-10 à batch 128 (Tang et al., 2022, Table 7)[PP] ; Chameleon le fait en FP16, à 413 ms et 1,22 J par image, contre 33 ms sur Jetson Nano (Aggarwal et al., 2024, Table VI) ;

— « CL sur FPGA : précision faible » devient « énergie de portée non précisée ». SLDA sur DPU atteint 78,79 % sur CORe50 (objet), à 9,00 W lus sur les rails sans portée déclarée (Piyasena et al., 2021, Table V).

Gap : la sûreté du détecteur embarqué. Les deux moitiés du sujet de thèse existent, mais séparément. Les études de fiabilité sur FPGA portent sur de petits classifieurs : 19 et 1 848 poids (Libano et al., 2019, §I, §III), un QNN FINN sur MNIST et CIFAR-10 (Gambardella et al., 2019, §V). Inversement, aucun des 18 papiers de l'axe `yolo-fpga` ne traite la sûreté, et le seul YOLO « certifié » du corpus tourne sur Jetson (Belcaid et al., 2024, §V). Les guides renvoient l'implémentation à plus tard : l'EASA écarte la conformité du matériel spécialisé (EASA and Daedalean, 2020, §4.2.3) et n'accepte pas l'apprentissage en ligne (EASA, 2024, MOC LM-02).

Gap : la détection de personnes sur FPGA. Un seul papier mesure un détecteur de personnes sur FPGA : YOLOv8, v11 et v26 nano en INT8 sur ZCU104, mAP@0.5 :0.95 de 0,66 à 0,79 sur un jeu maison non publié, 34,05 FPS de bout en bout et une puissance estimée par Vivado (Karki et al., 2026, Table II, §IV-C)[PP]. Aucun travail ne donne de taux de personnes manquées à taux de fausse alarme fixé.

TABLE 10 – Gaps FPGA d'architecture, de mesure et d'apprentissage : statut dans le corpus et piste principale.

Gap	Statut	Piste
Autoencodeurs		
Anomalie visuelle (images RGB) par AE sur FPGA embarquée	étayé (15 fiches) ; un contre-exemple partiel (hyperspectral, seuil réglé sur le test)	chaîne CAE INT8 de Vitis AI sur MVTec AD et KV260, seuil fixé hors du test, AUROC et puissance de carte ; coût des déconvolutions en dataflow
Compression apprise embarquée avec codage entropique	hypothèse (4 fiches)	chaîne complète sur plateforme à basse consommation, énergie par image mesurée
VAE en matériel pour un latent de grande dimension	hypothèse (2 fiches)	coût d'un générateur gaussien parallèle ; score D_{KL} déterministe en alternative
Détection		
Protocole de mesure commun pour YOLO sur FPGA	étayé (24 fiches)	mAP sur la cible, batch 1 de bout en bout avec pire cas et gigue, puissance de puce et de carte
Détection en dataflow quantifié sur SoC à mémoire limitée	étayé (13 fiches), nuancé	YOLO récent en W4–W8 par FINN ou SATAY sur KV260, face à la DPU sur la même carte
Petits objets : effet de la quantification par taille d'objet	étayé (2 fiches) pour l'absence ; hypothèse sinon	AP_{small} COCO mesurée sur la cible
Post-traitement et interaction PS–PL	étayé (14 fiches)	NMS matériel à côté de la DPU, latence de bout en bout et borne
Mesure et comparaison		
Précision non vérifiée sur la cible	étayé (20 fiches), nuancé	mAP COCO val sur la carte, étape par étape (quantification, compilation, opérateurs, NMS)
Baseline Jetson récente mesurée dans les mêmes conditions	étayé (27 fiches)	même modèle, même précision numérique, même batch, même méthode de puissance sur KV260 et Orin Nano
Puissance puce contre carte	étayé (40 fiches + `benchmarks.csv`)	télémétrie de la puce + wattmètre de carte + puissance au repos sur la même carte
Métriques flatteuses et comparaisons biaisées	étayé (20 fiches)	recalculer les rapports annoncés à métriques imposées
Code et bitstreams rarement publiés	étayé ($\approx$ 55 fiches expérimentales)	publier projet, bitstream et scripts de mesure
Toolflows		
Comparaison croisée hls4ml / FINN / Vitis AI	étayé (12 fiches), nuancé	un même YOLO à opérateurs ReLU exporté en QONNX, trois flots, une carte
Réseaux qui ne tiennent pas on-chip	étayé (13 fiches)	quantification agressive + FCMP ou éviction SMOF, coût à batch 1 et variabilité temporelle
Modèle de puissance validé dans les toolflows	hypothèse (outils) ; étayé (7 fiches) pour l'estimation non validée	vérifier la documentation actuelle de FINN et hls4ml
NAS et quantification mixte guidées par des mesures FPGA	hypothèse (nuancée)	HAWQ + ILP appliqués à la tête d'un YOLO
Apprentissage continu		
Rétropropagation sur SoC FPGA : lente, en flottant, énergie non mesurée	étayé (7 fiches)	queue de réseau en précision réduite en PL, étage figé sur la DPU
Adaptation à la dérive en détection d'anomalies	hypothèse (5 fiches)	ONLAD sur les caractéristiques d'un encodeur CNN figé
CL sur FPGA : énergie de portée non précisée ou estimée	étayé (6 fiches)	remesurer SLDA avec wattmètre et télémétrie
Apprentissage continu limité à la classification	étayé (13 fiches), nuancé (détection incrémentale sans FPGA)	Latent Distillation sur une tête YOLO, backbone figé sur la DPU

TABLE 11 – Gaps de sûreté, de certification et de détection de personnes : statut dans le corpus et piste principale. Aucun texte normatif n'est dans la base.

Gap	Statut	Piste
Sûreté, fiabilité et certification		
Aucun détecteur quantifié sur FPGA dans les travaux de fiabilité, de vérification ou de surveillance	étayé (25 fiches)	un YOLOv3-tiny ou v8n sur KV260, objet commun aux expériences de fiabilité, de mAP sur cible et de moniteur
Fiabilité SEU d'un détecteur : criticité par couche, TMR sélectif, DPU contre dataflow	étayé (9 fiches)	injection (configuration, BRAM, multi-fautes) sur un YOLO INT8 FINN et DPU ; durcissement sélectif chiffré en LUT et BRAM
Critère d'erreur dangereuse pour la détection de personnes ; FIT et précision vers SIL / PL	étayé (11 fiches)	non-détection définie sur une séquence ; taux de défaillance par heure combinant inexactitude et fautes
Preuve d'équivalence modèle → bitstream ; qualification des toolflows	étayé (14 fiches)	sorties bit à bit du modèle QONNX et du bitstream ; analyse TQL de FINN ou Vitis AI
Vérification formelle non scalable aux CNN de détection	étayé (4 fiches) ; hypothèse pour les outils récents	vérifier un moniteur ou un post-traitement RTL, plus petit que le réseau
Moniteurs d'exécution (OOD, incertitude, conforme) non portés sur FPGA	étayé (20 fiches)	prédiction conforme, puis moniteur à boîtes en PL ; surcoût mesuré sur KV260
Correspondance des cadres aéronautique et automobile avec les normes industrielles	étayé (11 fiches) ; hypothèse pour la littérature	tableau objectif par objectif vers IEC 61508, ISO 13849 et ISO/IEC TR 5469
Déterminisme du FPGA : WCET et gigue non mesurés	étayé (12 fiches)	traces de latence sur 10^5 images, DPU, FINN et TensorRT
Apprentissage en ligne exclu des cadres de certification	étayé (13 fiches)	SLDA StaticCov avec test de non-régression avant activation
Détection de personnes en milieu industriel		
Jeu de données et cas d'assurance en zone industrielle	étayé (8 fiches) ; hypothèse pour la littérature	recherche de jeux EPI ; ODD d'exemple selon AMLAS
Détecteurs de personnes quantifiés sur FPGA	étayé (9 fiches) ; un seul travail mesuré	YOLOv8n sur CrowdHuman, rappel à FPPI fixé sur KV260, flottant et INT8
Chaîne détection → suivi → ré-identification sur FPGA	étayé (4 fiches) ; hypothèse pour la Re-ID	suivi simple (SORT) sur le PS de la KV260
Robustesse adverse des détecteurs quantifiés	hypothèse (3 fiches)	rejouer un patch adverse sur le YOLO quantifié, avec et sans moniteur

13.1 Quatre questions de thèse candidates

Les pistes suivantes sont des propositions de ce document *(déduit)*. Elles combinent des gaps étayés et s'appuient sur des briques déjà présentes dans le corpus. Elles partagent un même objet expérimental : un YOLO quantifié de détection de personnes sur KV260.

1. **Un détecteur de personnes certifiable sur SoC FPGA.** Établir, pour un même YOLO, une chaîne de preuve d'équivalence entre le modèle QONNX et le bitstream, une borne de latence mesurée (pire cas, gigue) et un moniteur d'exécution sur la même puce. Briques existantes :
   — le YOLOv3-tiny allégé en w2a4 par FINN sur Zynq-7020 (Gutierrez et al., 2026)[PP], flot dataflow plus inspectable que la DPU *(déduit)* ;
   — la criticité par couche et le TMR sélectif de Libano et al. (2019) ;
   — la prédiction conforme sur YOLO (Andéol et al., 2023)[PP] et le moniteur à boîtes d'abstraction (Henzinger et al., 2019).
   Verrou : relier ces mesures à un taux de défaillance dangereuse par heure, dans un cadre de sécurité machine que le corpus ne contient pas encore.
2. **Un protocole de mesure et une comparaison croisée des flots.** Un même YOLO compilé par Vitis AI, et par FINN ou hls4ml via QONNX, sur KV260, face à une Jetson Orin Nano. Mesures : mAP COCO et rappel de personnes sur la cible, latence de bout en bout et sa distribution, puissance de puce et de carte. Le protocole partirait du cadre d'Schlosser et al. (2023) (mAP sur la carte, puissance de carte, repos déduit) et de ceux de DAC-SDC (Xu et al., 2021), de FINN (Umuroglu et al., 2016) et d'Alqahtani et al. (Alqahtani et al., 2024)[PP].
3. **Détection d'anomalies visuelles par autoencodeur convolutif sur SoC FPGA.** Briques existantes :
   — le CAE INT8 sur DPU et son score par reconstruction (Castelino et al., 2024), dont le seuil est réglé sur le test ;
   — l'encodeur ternaire paramétrable de Manjunath et al. (2021) ;
   — le score calculé sur la puce et le VAE réduit à l'encodeur de Govorkova et al. (2021).
   Verrou : une AUROC mesurée sur la carte, sur MVTec AD, avec un seuil fixé hors du test, sur KV260 ou en dataflow. Le même score pourrait servir de moniteur hors distribution pour le détecteur *(déduit)*.
4. **Adaptation en ligne certifiable par construction.** Un étage figé INT8 sur la DPU et une petite tête apprenante : SLDA (Piyasena et al., 2021), recette QLR-CL (Ravaglia et al., 2021) ou OS-ELM pour la dérive (Tsukada et al., 2020). Un test de non-régression sur un jeu de référence embarqué et un retour arrière précéderaient toute activation des nouveaux paramètres. Mesures : latence et énergie par mise à jour, surcoût de la validation. L'extension à la détection reste ouverte : la détection incrémentale n'existe que sans FPGA (Pasti et al., 2024)[PP].

13.2 Gaps entre plateformes

La passerelle MCU–FPGA est un sujet à part entière. Côté MCU, les accélérateurs intégrés (MAX78000, GAP9, STM32N6) atteignent des énergies de l'ordre de la centaine de microjoules par inférence pour de très petits détecteurs (Moosmann et al., 2023, §III). Côté FPGA, les détecteurs compressés du DAC-SDC ont des tailles comparables : 64 Ko de poids pour SystemsETHZ (Xu et al., 2021, §4.2). Le premier détecteur dataflow de la base de taille proche d'un MCU est un YOLOv3-tiny allégé en w2a4 par FINN sur XC7Z020, à 2,91 W au rail 12 V (Gutierrez et al., 2026, §V-VI)[PP], mais il n'est pas porté sur MCU. Aucun travail du corpus ne mesure un même réseau sur les deux. Un MCU à NPU est pourtant le concurrent le plus direct d'un petit FPGA en détection à très basse consommation *(déduit)*.

TABLE 12 – Gaps entre plateformes (repris de `gaps.md`, 8e passage).

Gap	Statut	Piste
Pas de synthèse quantitative FPGA / Jetson / MCU à NPU, ni de synthèse sûreté × accélérateur FPGA de détection	étayé (16 surveys)	argument de positionnement pour l'état de l'art d'un pré-projet
Aucune passerelle MCU–FPGA pour la détection	hypothèse (8 fiches)	porter TinyissimoYOLO, ou le LP-YOLO Slim w2a4 de Gutierrez et al. (2026)[PP], sur petit FPGA et sur MCU, et mesurer l'énergie par inférence à portée égale
Aucune comparaison MCU / FPGA / Jetson pour un même algorithme d'apprentissage continu	étayé (9 fiches)	porter SLDA ou QLR-CL (étage figé INT8) sur KV260, sur Orin et sur GAP9, avec la même mesure de puissance
Baseline Jetson Orin mesurée face à une FPGA dans les mêmes conditions	étayé (27 fiches) ; partiellement comblé sur AGX Xavier (Schlosser et al., 2023)	même modèle, même précision numérique, même méthode de puissance sur KV260 et Orin Nano
Anomalie par AE hors FPGA	2 lignes sur Xavier NX (Castelino et al., 2024), aucune sur MCU	baseline MCU pour tout AE d'anomalie porté sur FPGA

14 Conclusion

Chaque famille de plateformes occupe une niche :

— le MCU à accélérateur pour les très petits modèles, à quelques centaines de microjoules par inférence ;

— le GPU embarqué pour la facilité de déploiement, à quelques dizaines de FPS pour 7 à 15 W de télémétrie ;

— le FPGA pour la latence déterministe et la précision arbitraire.

Le corpus ne permet pas de les départager sur un même réseau : les mesures ne partagent ni le modèle, ni la précision numérique, ni la méthode de puissance, et la précision est rarement vérifiée sur la cible. Cette absence de comparaison propre est elle-même un résultat, et une opportunité de recherche.

Pistes concrètes.

1. **Lire en priorité** les quatre articles de la base sans fiche. Le survey d'anomalie visuelle de Cui et al. (2023) oriente l'axe autoencodeurs ; Lin et al. (2022)[PP] et Rüb et al. (2024) complètent l'apprentissage embarqué sur MCU.
2. **Expérience à reproduire** : le protocole d'Alqahtani et al. (2024)[PP] (mAP COCO val, service à une requête, repos soustrait), étendu à une KV260 par Vitis AI et à un MCU à NPU, avec en plus la télémétrie de la puce et la précision numérique déclarée.
3. **Gap à creuser** : la passerelle MCU–FPGA pour la détection. Porter TinyissimoYOLO (Moosmann et al., 2023) sur petit FPGA (FINN ou hls4ml) et mesurer l'énergie par inférence en portée puce, comme sur le MAX78000.

Glossaires

Chaque entrée renvoie aux sections où le terme apparaît ; dans le texte, chaque apparition est un lien vers son entrée. Une valeur suivie de [constructeur, hors corpus] vient de la documentation du constructeur, [hors corpus] de connaissances générales : ni l'une ni l'autre n'est tirée du corpus.

Glossaire des plateformes

Artix-7 Famille FPGA Xilinx 28 nm bas coût, sans processeur durci [constructeur, hors corpus]. XC7A100T : 63 400 LUT, 240 DSP, 135 BRAM36 [constructeur, hors corpus](135 BRAM aussi chez Manjunath et al., 2021, §VIII-A) ; XC7A35T : 20 800 LUT, 90 DSP, 50 BRAM36 [constructeur, hors corpus]. Autoencodeur de Manjunath à 100 MHz, Arty A7-100T sous MLPerf Tiny (Borras et al., 2022)[PP], hls4ml (Schulte et al., 2025)[PP]. 8, 9

Cortex-A9 Cœur applicatif Arm ARMv7-A [constructeur, hors corpus]; processeur (PS) des Zynq-7000, à 650 MHz sur PYNQ-Z1 (Tsukada et al., 2020, Table 1). Exécute le post-traitement YOLOv2 sous PetaLinux chez Kim and Kim (2025, §4.2). 6

Cortex-M4 Cœur Arm ARMv7E-M pour MCU, avec instructions DSP/SIMD et FPU simple précision optionnelle (Cortex-M4F) [constructeur, hors corpus]. Présent dans les STM32F412 et STM32L4, le MAX78000 et l'Apollo4b du corpus. 7

Cortex-M7 Cœur Arm ARMv7E-M haut de gamme pour MCU : pipeline superscalaire, caches, FPU optionnelle [constructeur, hors corpus]. Présent dans les STM32F746, STM32H743 et STM32H7A3 du corpus. 7

DPU (DPUCZDX8G) IP d'inférence CNN de Xilinx (AMD) livrée avec Vitis AI : processeur programmable en INT8, implanté dans la PL des Zynq UltraScale+, auquel le compilateur XIR envoie un flot d'instructions. Configurations B512 à B4096, où le nombre est le parallélisme crête en opérations par cycle (B4096 : 4 096 op./cycle, soit environ 1,2 TOPS à 300 MHz *(déduit)*), jusqu'à quatre cœurs par puce [constructeur, hors corpus]. Sur XCZU7EV, une B4096 occupe 55,8 k LUT, 710 DSP, 142 BRAM et 46 URAM, et la puissance estimée par Vivado va de 4,79 W (B512) à 8,76 W (B4096) (Karki et al., 2026, Table III)[PP]. C'est le flot « industriel » du corpus : détection YOLO sur ZCU104 et KV260, comparaisons FPGA / Jetson, tête d'apprentissage continu couplée au DPU (Piyasena et al., 2021). 4–10, 12, 13

Google Edge TPU (Coral) ASIC d'inférence INT8 de Google, 4 TOPS pour 2 W (Luo et al., 2024, Table 4), vendu en clé USB, module ou Coral Dev Board [constructeur, hors corpus]. Mesuré sur Coral Dev Board (Magalhães et al., 2022) et en clé USB sur Raspberry Pi, où YOLOv8 doit passer en 320 × 320 (Alqahtani et al., 2024, §III-B2)[PP]. 3, 7

Hailo-8 NPU dataflow de la société Hailo, 26 TOPS INT8 pour environ 2,5 W typiques [constructeur, hors corpus]; utilisée via la carte Raspberry Pi AI HAT+ et HailoRT (Alqahtani et al., 2024, §III-B2)[PP]. Seul point NPU du corpus : YOLOv8n, mAP COCO 32 en 76 ms (Alqahtani et al., 2024, §IV)[PP]. 3, 7

Jetson Gamme NVIDIA de modules embarqués (SoC ARM + GPU CUDA, mémoire LPDDR partagée) et de kits de développement ; logiciel JetPack et TensorRT (FP32, FP16, INT8), modes de puissance réglables [constructeur, hors corpus]. Référence GPU embarqué du corpus, mais peu de travaux la comparent à un FPGA sur le même modèle. 0, 2, 3, 5–8, 10–13

- **Jetson AGX Xavier** CPU 8 cœurs Carmel, GPU et NVDLA (Belcaid et al., 2024, §V-A)[PP]; GPU Volta 512 cœurs + 64 Tensor Cores, 2 NVDLA, 32 Go, 10 à 30 W, 32 TOPS INT8 [constructeur, hors corpus]. 16,5 à 28 W mesurés à l'entrée de la carte selon la précision, face à une ZCU104 (Schlosser et al., 2023, Table 1) ; cible de la détection certifiée en vidéosurveillance (Belcaid et al., 2024)[PP]. 3, 5–8, 13
- **Jetson Nano** GPU Maxwell 128 cœurs (0,5 TFLOPS), 4 Go de LPDDR4, 5 à 10 W (Feng et al., 2022, Table 2) ; CPU quadri Cortex-A57 [constructeur, hors corpus]. Ne supporte que le FP16 selon Feng et al. (2022, §3.3.1). Mesures YOLO (Alqahtani et al., 2024)[PP], apprentissage continu (Aggarwal et al., 2024), et point de comparaison repris par plusieurs papiers FPGA. 3, 6–8, 10–13

Jetson Orin Génération Jetson à GPU Ampere et CPU Cortex-A78AE [constructeur, hors corpus]: Orin Nano, Orin NX, AGX Orin. Les lignes Orin du corpus viennent de deux papiers seulement, et aucun travail ne compare une FPGA à une Orin. 3, 7, 8, 12, 13

Jetson AGX Orin 15 à 40 W et 275 TOPS selon Luo et al. (2024, Table 4) ; le constructeur annonce 12 × Cortex-A78AE et un GPU Ampere 2 048 cœurs + 64 Tensor Cores, avec 275 TOPS et 15 à 60 W pour la version 64 Go, 200 TOPS et 15 à 40 W pour la version 32 Go [constructeur, hors corpus]: la table du corpus mêle les deux. Baseline GPU d'ImageHD (Arockiaraj et al., 2026)[PP]. 3, 12

Jetson Orin Nano 6 × Cortex-A78AE, GPU Ampere 1 024 cœurs + 32 Tensor Cores, 8 Go, 7 à 15 W (Rey et al., 2025, Table 2) ; 40 TOPS INT8 annoncés au lancement pour la version 8 Go [constructeur, hors corpus]. Version 4 Go dans Alqahtani et al. (2024, Table I)[PP], qui y mesurent l'énergie au repos la plus élevée de leur banc (362 mWh sur 5 min, Fig. 2a). 3, 7, 8, 12, 13

Jetson Orin NX 8 × Cortex-A78AE, 16 Go, 10 à 25 W (Rey et al., 2025, Table 2). La même table donne un GPU Ampere de 2 048 cœurs + 64 Tensor Cores, alors que la fiche constructeur annonce 1 024 cœurs + 32 Tensor Cores et 100 TOPS INT8 pour la version 16 Go [constructeur, hors corpus]. YOLOv8n INT8 : 65,8 FPS pour environ 10,3 W (Rey et al., 2025, Table 4). 3, 7, 8

Jetson TX2 GPU Pascal 256 cœurs, CPU double Denver 2 et quadri Cortex-A57, 8 Go de LPDDR4, 7,5 à 15 W [constructeur, hors corpus]; 665 GFLOPS crête à 1 300 MHz selon Zhang et al. (2019, §6.4)[PP]. Plateforme GPU imposée de DAC-SDC (Xu et al., 2021, §2.2) ; baseline de SkyNet, SATAY et ReQ-YOLO. 3, 5–8, 10, 12

Jetson Xavier NX GPU Volta 384 cœurs + 48 Tensor Cores (1,3 TFLOPS), 8 Go de LPDDR4x, 10 à 15 W (Feng et al., 2022, Table 2) ; CPU 6 cœurs Carmel, 21 TOPS INT8 [constructeur, hors corpus]. Baseline GPU de Piyasena et al. (2021), de F-LIC (Sun et al., 2022) et de Castelino et al. (2024)[PP] (mode 10 W). 3, 7–9, 11–13

Kintex-7 Famille FPGA Xilinx 28 nm de milieu de gamme, sans processeur durci [constructeur, hors corpus]. Usage : carte KC705 de Zhang et al. (2024). « Kintex » seul désigne aussi la Kintex UltraScale annoncée par Campos et al. (2024, §6.1), dont la référence (xcu250) est en fait celle de l'Alveo U250. 3, 6

XC7K325T Puce Kintex-7 : 203 k LUT, 840 DSP, 16 Mbit de BRAM, sur carte KC705 avec 1 Go de DDR3 (Zhang et al., 2024, §5.1) ; 445 BRAM36 [constructeur, hors corpus]. Porte YOLOv2 et YOLOv3-Tiny en INT8 tout sur FPGA, 401 à 418 GOP/s, 7,8 W puce et 12,3 W carte (Zhang et al., 2024, §5.3). 6

Intel NCS2 (Myriad) Neural Compute Stick 2 : clé USB 3.0 d'Intel à VPU Movidius Myriad X, environ 1 TFLOPS et 1 W en plus de l'hôte (Feng et al., 2022, Table 2), 0,64 W au repos (Feng et al., 2022, Table 4) ; OpenVINO, FP16. Son prédécesseur, la Myriad 2, est la VPU des satellites Φ-Sat (Guerrisi et al., 2023) et a été testée sous ions lourds (Rech, 2024, §IV-C.3). 3, 7, 9, 12

PolarFire Famille FPGA Microchip 28 nm à configuration non volatile (flash SONOS), donc insensible aux SEU de la mémoire de configuration [constructeur, hors corpus]; MPF300T : environ 300 k éléments logiques [constructeur, hors corpus]. Seul usage : hls4ml sur MPF300TS et MPF100T, limité aux couches denses et ReLU, environ 0,1 W estimés (Govorkova et al., 2026, §5.2)[PP]. Intéresse la thèse pour la tenue aux radiations. 5, 9

PULP Parallel Ultra-Low Power : plateforme matérielle open source de l'ETH Zürich et de l'université de Bologne, à cluster de cœurs RISC-V partageant une mémoire L1 [constructeur, hors corpus]. Base des puces VEGA et GAP9 du corpus. 10

GAP9 MCU RISC-V de GreenWaves, dérivé de PULP, avec accélérateur NE16 [constructeur, hors corpus]. Fabric controller + cluster de 9 cœurs, L3 externe de 8 Mo, 650 mV et 240 MHz, environ 44 à 50 mW (Wibowo et al., 2024, §V-A, §VI-C)[PP]; cluster 8 cœurs à 175 MHz et 1,6 Mo de SRAM selon El Zeinaty et al. (2025, Table 4). Apprentissage few-shot à 12 mJ par classe et TinyissimoYOLO. 3, 8, 10, 11, 13

VEGA SoC PULP en 22 nm GlobalFoundries : cluster de 9 cœurs RISC-V (8 de calcul + 1 contrôleur), 4 FPU FP32/FP16 partagées, 128 Ko de L1, 1,5 Mo de L2, 4 Mo de MRAM (Ravaglia et al., 2021, §IV-A)[PP]; 375 MHz et 62 mW mesurés sur la puce seule (Ravaglia et al., 2021, §V-D)[PP].

Plateforme de l'apprentissage continu par latent replay sur MCU. 3, 8, 10–12

STM32 Famille de MCU 32 bits de STMicroelectronics à cœur Arm Cortex-M ; déploiement de réseaux par X-CUBE-AI ou TensorFlow Lite Micro [constructeur, hors corpus]. Cible des travaux MCUNet, TinyissimoYOLO et de la comparaison à VEGA ; la référence « petite cible » face à laquelle un FPGA doit se justifier. 3

STM32F412 Cortex-M4, 256 Ko de SRAM, 1 Mo de Flash (Lin et al., 2021, §4)[PP]; 100 MHz [constructeur, hors corpus]. Porte MCUNetV2-YOLOv3-M4 (64,6 % de mAP VOC, El Zeinaty et al., 2025, Table 4). 7

STM32F746 Cortex-M7 à 216 MHz, 320 Ko de SRAM, 1 Mo de Flash (Lin et al., 2022, §3.1)[PP] ; (El Zeinaty et al., 2025, Table 4). Cible de l'entraînement sur l'appareil en 256 Ko et de MCUNetV2. 4, 10, 11

STM32H743 Cortex-M7 à 480 MHz, 1 Mo de SRAM (El Zeinaty et al., 2025, Table 4) ; 512 Ko de SRAM et 2 Mo de Flash selon Lin et al. (2021, §4)[PP] (le constructeur annonce 1 Mo de RAM au total, dont 512 Ko de SRAM AXI d'un seul tenant [constructeur, hors corpus]). MCU de référence des détecteurs MCUNet, EtinyNet et XiNet. 7

STM32H7A3 Cortex-M7, utilisé à 3,3 V et 192 MHz (Moosmann et al., 2023, Table II)[PP]; jusqu'à 280 MHz, 1,4 Mo de SRAM et 2 Mo de Flash [constructeur, hors corpus]. TinyissimoYOLO y tourne en 359 ms par inférence (Moosmann et al., 2023, §III)[PP]. 7

STM32L4 Série basse consommation à Cortex-M4. STM32L476RG à 80 MHz (carte NUCLEO-64), 65 fois plus lent que VEGA en moyenne (Ravaglia et al., 2021, §V-E)[PP]; 128 Ko de SRAM [constructeur, hors corpus]. STM32L4R9 (Cortex-M4F, 3,3 V, 120 MHz) dans Moosmann et al. (2023, Table II)[PP]. 8, 11

STM32N6 MCU à Cortex-M55 et NPU, 800 MHz, 4,2 Mo de SRAM (El Zeinaty et al., 2025, Table 4) ; NPU Neural-ART d'environ 600 GOPS [constructeur, hors corpus]. Cité comme MCU à accélérateur IA ; aucune mesure dans le corpus. 3, 13

Stratix Famille FPGA haut de gamme d'Intel (ex-Altera) ; Stratix V en 28 nm, Stratix 10 en 14 nm [constructeur, hors corpus]. Présente seulement par citation : baseline Stratix 10 GX d'EF-Train (163 GOPS, 20,6 W, Tang et al., 2022[PP]) et valeur projetée de 12 TOP/s à 141,2 W (Abdelouahab et al., 2018, Table 8)[PP]. 6

VCU118 Carte d'évaluation Xilinx à Virtex UltraScale+ XCVU9P (16 nm) : 1 182 240 LUT et 6 840 DSP (Sun et al., 2022, Fig. 6) ; 2 160 BRAM36 et 960 URAM [constructeur, hors corpus]. Grosse carte de laboratoire : compression d'image F-LIC, SATAY (YOLO, 23 à 67 W, Montgomerie-Corcoran et al., 2023, Table III), SMOF, et VAE de déclenchement au LHC (Govorkova et al., 2021). 5, 6, 8, 9

Virtex-7 Famille FPGA Xilinx 28 nm haut de gamme, sans processeur durci [constructeur, hors corpus]. Cartes du corpus : VC707 (XC7VX485T, 18,61 W carte, Zhang et al., 2015, §5.2), VC709 et ADM-PCIE-7V3 (XC7VX690T). « Virtex » seul désigne aussi les Virtex UltraScale(+) (VCU110, VCU118) et le Virtex-6 ; aucune n'est une cible embarquée. 3, 4, 6

XC7VX690T Puce Virtex-7 : 433 k LUT, 3 600 DSP, 52,9 Mbit de BRAM, deux DDR3 de 4 Go sur VC709 (Zhang et al., 2024, §5.1) ; 1 470 BRAM36 [constructeur, hors corpus]. Sert à ReQ-YOLO sur carte PCIe ADM-PCIE-7V3 (16 Go de DDR3, 200 MHz, Ding et al., 2019, §6.3.1 ; 21 W, Ding et al., 2019, Table 2) et à YOLOv2/v3 batch 2 (914 GOP/s, Zhang et al., 2024, Table 3). 6

XC6VLX240T (Virtex-6) Puce Virtex-6 (40 nm) du kit ML605 : 241 152 cellules logiques, 150 720 LUT, 768 DSP48E1, 416 BRAM36 [constructeur, hors corpus]; les 768 multiplieurs sont aussi donnés par Coutinho et al. (2019, Table 3). Seul usage : l'autoencodeur empilé (SSAE) MNIST de Coutinho (Coutinho et al., 2019, §V-D). 9

Zynq Famille de SoC FPGA de Xilinx (AMD) : un processeur ARM durci (PS) et une logique programmable (PL) sur la même puce, reliés par des bus AXI. Le corpus en utilise deux générations, le Zynq-7000 et le Zynq UltraScale+ MPSoC (dont le module Kria K26). C'est la cible principale de la thèse. 3, 6, 7, 10

Zynq UltraScale+ MPSoC Seconde génération de Zynq, en 16 nm FinFET (Libano et al., 2019, §III) ; PS quadri Cortex-A53, double Cortex-R5F et GPU Mali-400 (variantes EG et EV), encodeur

vidéo VCU sur les EV ; PL à DSP48E2, BRAM et, sur les plus grosses puces, URAM [constructeur, hors corpus]. Cible du DPU de Vitis AI. Dans le texte, « UltraScale » seul peut aussi désigner les FPGA Kintex et Virtex UltraScale(+) sans processeur (VCU110, VCU118, Alveo). 3, 5, 9

XCK26 (Kria K26) Puce Zynq UltraScale+ (variante EV, avec VCU) du module Kria K26, non vendue seule [constructeur, hors corpus]: 256 k cellules logiques, 117 120 LUT, 234 240 FF, 1 248 DSP [constructeur, hors corpus]; 144 BRAM (648 Ko), 64 URAM (environ 2,25 Mo) et 4 Go de DDR4 sur le module (Fata and Elmannai, 2026, Table 6). 6

Kria KV260 Kit de démarrage AMD Xilinx « Vision AI » : module K26 sur carte porteuse à interfaces caméra. Cible matérielle visée par la thèse. Mesures du corpus : YOLOv3-Tiny INT8 sur DPU, 24,3 FPS pour 2,13 W (portée non précisée, Fata and Elmannai, 2026, Table 10) ; YOLOv5 INT8 (Al Amin and Obermaisser, 2025)[PP]; RetinaNet sur un cœur DPU (Magalhães et al., 2022). Aucun protocole commun de mesure entre ces travaux. 4–7, 9, 12–14

XCZU3EG Plus petite puce Zynq UltraScale+ du corpus : 70 560 LUT, 141 120 FF, 360 DSP, 216 BRAM36, sans URAM [constructeur, hors corpus]; « 71 kLUT, 432 BRAM » (BRAM18) selon Blott et al. (2018, §4)[PP]. CodeNet en occupe la totalité des BRAM et des DSP (Dong et al., 2020, §5)[PP]. Puce de l'Ultra96, proche de la gamme Kria. 6

Ultra96 Carte Avnet à XCZU3EG, 2 Go de LPDDR4 [constructeur, hors corpus]. Carte des éditions 2019 et suivantes de DAC-SDC : SkyNet (pic théorique 144 GOPS à 200 MHz, 7,26 W, Zhang et al., 2019, §6.4[PP]), CodeNet (4,3 W à vide, 5,6 W en charge, carte, Dong et al., 2020, §5[PP]) et FINN-R. 3, 5–8

XCZU7EV Puce Zynq UltraScale+ EV : 504 k cellules logiques, 230 400 LUT, 460 800 FF, 1 728 DSP, 312 BRAM36 et 96 URAM [constructeur, hors corpus]. Isik et al. (2023, §IV-C) annoncent « 504 K LUT », ce qui est en fait le nombre de cellules logiques. Puce de la ZCU104. 9

ZCU104 Carte d'évaluation Xilinx à XCZU7EV, 2 Go de DDR4 côté PS [constructeur, hors corpus]. Carte FPGA la plus mesurée du corpus : DPU B512 à B4096 pour les YOLO récents (Karki et al., 2026)[PP], comparaison à la Jetson AGX Xavier (23 W à l'entrée de la carte, Schlosser et al., 2023, Table 1), RetinaNet sur deux cœurs DPU (25 FPS, Magalhães et al., 2022, §3), SATAY, injection de fautes accélérée (Gambardella et al., 2019). 3, 5–10, 12, 13

XCZU9EG Puce Zynq UltraScale+ EG : 274 080 LUT, 548 160 FF, 2 520 DSP, 912 BRAM36, sans URAM [constructeur, hors corpus]; 2 520 DSP aussi dans Yan et al. (2024, Table 2). Puce de la ZCU102, irradiée au ChipIR par Libano et al. (2019, §III-B). 9

ZCU102 Carte d'évaluation Xilinx à XCZU9EG, 4 Go de DDR4 (Yan et al., 2024, Table 2). Sert à YOLOv5s INT4 (79 FPS, 12,3 W, Yan et al., 2024, §7.4), à l'apprentissage continu (DPU B4096 et SLDA, Piyasena et al., 2021 ; Chameleon, Aggarwal et al., 2024 ; EF-Train) et à SMOF. 4, 6–10, 12, 13

Zynq-7000 Première génération de Zynq, en 28 nm (Libano et al., 2019, §III) ; PS double Cortex-A9 (simple cœur pour les variantes S) et PL de type Artix-7 ou Kintex-7 selon la taille [constructeur, hors corpus]. Petite cible bas coût du corpus : FINN, concours DAC-SDC, autoencodeurs HLS, ONLAD, YOLOv2 en RTL (Kim and Kim, 2025), et essais sous neutrons (LANSCE) de Libano et al. (2019, §III-B). 3, 9

XC7Z020 (Zynq-7020) Puce Zynq-7000 la plus utilisée du corpus : 53 200 LUT, 106 400 FF, 220 DSP (Kim and Kim, 2025, §3.1), 280 BRAM18 (Tsukada et al., 2020, Table 1), soit 140 BRAM36 et 4,9 Mbit [constructeur, hors corpus], que Xu et al. (2021, §2.2) arrondissent à 630 Ko. PS double Cortex-A9, à 650 MHz sur PYNQ-Z1 (Tsukada et al., 2020, Table 1). Cartes du corpus : PYNQ-Z1, PYNQ-Z2, Zybo Z7-20. 3–7, 9, 13

PYNQ-Z1 Carte de développement Digilent [constructeur, hors corpus] à XC7Z020-1CLG400C, 512 Mo de DDR3, « environ 1-4 W » (Xu et al., 2021, §2.2). Carte FPGA imposée du concours DAC-SDC ; 3,1 W carte entière pour ONLAD (Tsukada et al., 2020, §6.4) ; 2,0 à 2,9 W pour un détecteur FINN (Gutierrez et al., 2026, §V)[PP]. Utilisée aussi par FINN-R et EF-Train. 5, 6, 8, 10

PYNQ-Z2 Carte de développement TUL à XC7Z020 (xc7z020clg400-1) (Aarrestad et al., 2021, §8) ; 512 Mo de DDR3 [constructeur, hors corpus]. Seule carte FPGA du corpus mesurée

selon le protocole MLPerf Tiny (Joulescope), avec hls4ml et FINN (Borras et al., 2022, §4.4)[PP]; sert aussi au CNN QKeras 7 bits de Aarrestad et al. (2021) et au CAE vibratoire de Malviya et al. (2022). 3, 5, 8, 9

XC7Z035 Puce Zynq-7000 de milieu de gamme : 171 900 LUT, 343 800 FF, 900 DSP, 500 BRAM36 [constructeur, hors corpus]. Un seul usage mesuré : l'accélérateur YOLO et suivi de véhicules de Zhai et al. (2023, §4.1) (XC7Z035-FFG676-2). 6

ZC706 Carte d'évaluation Xilinx à Zynq-7000 XC7Z045 : double Cortex-A9, 218 600 LUT, 545 BRAM (Umuroglu et al., 2016, §5.1)[PP], 900 DSP [constructeur, hors corpus]. Carte du premier FINN, dont la puissance murale est dominée par environ 7 W de repos (Umuroglu et al., 2016, §5.1)[PP]; la même puce (Z7045) porte le VAE de Que et al. (2024). 5

Glossaire des modèles

AE (autoencodeur) Réseau entraîné à reconstruire son entrée au travers d'un goulot : l'encodeur la comprime en un code latent, le décodeur la reconstruit, avec une perte de reconstruction (MSE). Deux usages dans le corpus : la compression (seul l'encodeur est embarqué) et la détection d'anomalies (le score est l'erreur de reconstruction). Sur FPGA, il traite surtout des données non visuelles ou de petites images. 0–2, 5, 7–10, 12–14

CAE Autoencodeur convolutif : convolutions et sous-échantillonnage à l'encodeur, suréchantillonnage ou déconvolution au décodeur. Dans le corpus : détection d'artefacts d'images hyperspectrales par un CAE de 2,95 M paramètres sur DPU (ZCU104) (Castelino et al., 2024, §III-B–D), CAE de vibrations de 8 755 paramètres (Malviya), encodeurs de compression embarquée (Manjunath, Guerrisi), CAE débruiteur. 8, 9, 12, 13

ECON-AE Autoencodeur de compression des données du calorimètre HGCAL de CMS (ECON-T) ; seul l'encodeur (convolution et couche dense) est embarqué, évalué par l'EMD. Campos et al. le quantifient en précision mixte guidée par la Hessienne puis le synthétisent avec hls4ml, sans mesure sur carte (Campos et al., 2024, §7). 9

ONLAD Détecteur d'anomalies en ligne : autoencodeur à une couche cachée entraîné par OS-ELM, avec un mécanisme d'oubli léger pour suivre la dérive, et implanté en virgule fixe 32 bits sur XC7Z020 (ONLAD Core) (Tsukada et al., 2020, §2–4). Apprentissage continu en 3,1 W de carte (Tsukada et al., 2020, §6.4), sur des données tabulaires ou des images aplaties. 9–13

SSAE *Stacked Sparse Autoencoder* : encodeurs d'autoencodeurs parcimonieux pré-entraînés sans supervision puis empilés, terminés par un softmax supervisé. Coutinho et al. implantent un SSAE 784-100-50-10 en réseau systolique virgule fixe sur Virtex-6 : 93,38 % sur MNIST contre 93,4 % sous Matlab (Coutinho et al., 2019, §V-D). 9, 12

VAE Autoencodeur variationnel : l'encodeur sort une moyenne μ et un écart-type σ, le latent est tiré par reparamétrisation, et la perte ajoute à la reconstruction une divergence de Kullback-Leibler vers une loi normale. Au déclenchement du LHC, seul l'encodeur est implanté et le score d'anomalie est la D_{KL}, en 80 ns sur VU9P (Govorkova et al., 2021) ; Que 2024 implante un VAE complet avec générateur gaussien. 5, 9, 13

AlexNet Classifieur ImageNet de 2012 : 5 couches de convolution (noyaux hétérogènes, jusqu'à 11×11) puis 3 couches denses, ReLU. Banc d'essai historique des accélérateurs : Zhang et al. (2015) atteignent 61,62 GFLOPS sur ses convolutions en FP32 sur Virtex-7 (Zhang et al., 2015, Tables 7–8) ; EF-Train l'entraîne sur ZCU102. 4, 5

CenterNet Détecteur *sans ancres* (« Objects as Points ») : chaque objet est représenté par le pic de son centre sur une carte de chaleur par classe, la tête régressant taille et décalage ; les détections sont les maxima locaux, ce qui supprime le NMS. Dans le corpus, sa tête est reprise par CoDeNet, précisément pour éviter un NMS coûteux sur FPGA (Dong et al., 2020, §4.1). 6

CoDeNet Détecteur co-conçu avec son accélérateur : backbone ShuffleNetV2, convolution déformable adaptée au matériel (depthwise, offsets bornés, carrés et entiers), tête CenterNet sans NMS, W4A8 (Dong et al., 2020, §3.1, §4.1). Sur Ultra96 en 256×256 : 51,1 d'AP50 VOC07 à 32,2 FPS pour 5,6 W de carte, modèle de 0,76 MB (Dong et al., 2020, Table 3, §5). 6, 8, 12

Chameleon Apprentissage continu en ligne par replay d'activations latentes (couche 21 de MobileNetV1) depuis deux tampons, court terme sur la puce et long terme hors puce ; seule la partie haute est rétropropagée, en FP16 (Aggarwal et al., 2024, §III, §V-A). Sur ZCU102 : 413 ms et 1,22 J par image, 7 fois moins d'énergie que le Latent Replay, mais plus lent qu'une Jetson Nano (Aggarwal et al., 2024, Table VI). 8, 10–13

EF-Train Accélérateur d'entraînement complet de CNN sur FPGA : un même noyau de convolution exécute propagation avant, rétropropagation et mise à jour, en FP32, avec une réorganisation des données en DRAM (*data reshaping*) (Tang et al., 2022, §3–4)[PP]. 2,08 ms par image (batch 128) et 6,89 W estimés sur ZCU102 (Tang et al., 2022, Table 7)[PP]; aucune stratégie contre l'oubli. 10–13

FastFlow Détection d'anomalies par flot normalisant (*normalizing flow*) 2D appris sur les caractéristiques d'un CNN figé : il estime la densité des images normales, et une faible vraisemblance signale l'anomalie. 99,4 d'AUROC image sur MVTec AD (Cui et al., 2023, Table 10) (reconstruite par Docling) ; comme PatchCore, absent des FPGA du corpus. 9

GDumb Méthode d'apprentissage continu « naïve » : un tampon équilibré par classe est rempli avidement, et le modèle est réentraîné sur ce seul tampon au moment de l'inférence. Il sert de référence de replay : 19,8 % sur CIFAR-100 en 10 tâches (Masana et al., 2020, Table S2)[PP], et de politique de mémoire à TinyCL. 11

TinyCL Accélérateur d'apprentissage continu en RTL : une unité de 9 blocs MAC réutilisée pour propagation avant, rétropropagation et mise à jour, en virgule fixe 16 bits, avec une mémoire de replay gérée selon GDumb (Ressa et al., 2024, §III-A, §III-C–F)[PP]. Synthétisé en 65 nm : 86 mW d'après la synthèse, sans silicium (Ressa et al., 2024, Table I)[PP]. 10, 11

HDC *Hyperdimensional Computing* : les données sont encodées en hypervecteurs de très grande dimension (souvent binaires), manipulés par liaison, superposition et permutation ; l'apprentissage se réduit à accumuler des prototypes et l'inférence à une recherche de similarité (distance de Hamming). Opérations de bits sans rétropropagation, adaptées au FPGA. 10

ImageHD Apprentissage continu non supervisé : un MobileNetV2 INT8 figé alimente un encodeur HDC binaire, les hypervecteurs sont regroupés en clusters dans une mémoire bornée, fusionnés par un kMeans++ simplifié ; accélérateur dataflow en Vitis HLS sur ZCU104 (Arockiaraj et al., 2026, §IV, §VI). 0,27 ms et 0,97 mJ par image sur CIFAR-10, puissance estimée (Arockiaraj et al., 2026, Table VII). 8, 10–12

LIC *Learned Image Compression* : codec d'images appris, de type VAE avec hyper-prior (Ballé et al.) : transformée d'analyse vers un latent quantifié, hyper-analyse qui estime μ et σ du latent pour le codage entropique, transformée de synthèse au décodage ; activations GDN. Sur SoC, Mazouz et al. (2025)[PP] distillent, élaguent et quantifient un LIC pour trois DPU B4096 (ZCU102) : 43,63 FPS en encodage (Mazouz et al., 2025, Table VII)[PP]. 9

F-LIC Codec LIC à hyper-prior en virgule fixe 8 bits sur VCU118, en pipeline couche par couche ; deux poids partagent un DSP (casDSP) et la déconvolution saute les zéros ; le codage entropique reste sur l'hôte (Sun et al., 2022). Il encode du 720p à 40,69 FPS (Sun et al., 2022, Fig. 6) : premier codec appris temps réel sur FPGA du corpus, mais sur une grosse puce. 8, 9

GDN *Generalized Divisive Normalization* : activation des LIC qui divise chaque canal par la racine d'une combinaison pondérée des carrés des autres canaux ; l'inverse (iGDN) sert au décodeur. Non supportée par la DPU, elle est implantée comme opérateur XIR personnalisé en virgule fixe 32 bits (Mazouz et al., 2025, §III-B, §III-D)[PP]; la remplacer par ReLU coûte 0,5 à 0,7 dB de BD-PSNR (Mazouz et al., 2025, Fig. 15)[PP]. 9

LwF *Learning without Forgetting* : régularisation par distillation, sans exemplaires ; pendant l'apprentissage d'une nouvelle tâche, les sorties de l'ancien modèle sur les nouvelles données servent de cibles douces. Meilleure méthode sans exemplaires du banc de Masana et al. (2020)[PP] (30,2 % sur CIFAR-100 en 10 tâches, Table 2), mais en dessous du simple fine-tuning avec 2 000 exemplaires (37,9 %) ; baseline de Chameleon. 11

RILOD Apprentissage incrémental pour détecteur en une étape (RetinaNet) : une nouvelle classe est ajoutée avec ses seules données, en distillant depuis l'ancien modèle figé les sorties de classification, les boîtes et des caractéristiques intermédiaires (Li et al., 2019, §4.2)[PP]. Exécuté

sur Jetson TX2 ou en mode edge-cloud, sans FPGA : seul travail de détection incrémentale du corpus. 10

MCUNetV2 Réseaux pour MCU obtenus par recherche conjointe d'architecture et d'ordonnancement : les premiers blocs sont exécutés par patchs pour abaisser le pic de SRAM, le champ récepteur est redistribué, déploiement INT8 avec TinyEngine (Lin et al., 2021, §3). Avec une tête YOLOv3, MCUNetV2-M4 atteint 64,6 % de mAP VOC en 247 kB de SRAM (Lin et al., 2021, Table 3), sans latence ni énergie publiées. 7

MobileNet Famille de backbones légers fondés sur la convolution séparable en profondeur (depthwise puis pointwise 1×1) ; V2 ajoute les blocs résiduels inversés à goulot linéaire. Extracteur figé de plusieurs travaux d'apprentissage continu : MobileNetV1 dans Chameleon et QLR-CL, MobileNetV2 INT8 dans ImageHD (Arockiaraj et al., 2026, §IV-A) ; MobileNetV1 sert aussi de cas d'Elastic-DF. 8, 10, 11

NMS *Non-Maximum Suppression* : post-traitement des détecteurs à ancres ou à grille, qui trie les boîtes par score et supprime celles dont l'IoU avec une boîte mieux notée dépasse un seuil. Séquentiel et dépendant des données, il tourne presque toujours sur le cœur ARM, hors des chiffres de l'accélérateur : REQ-YOLO l'exclut de sa latence (Ding et al., 2019, Table 1), et il devient le goulot de bout en bout chez Karki et al. (2026)[PP] (§IV-C). 6, 13

OS-ELM *Online Sequential Extreme Learning Machine* : réseau à une couche cachée dont les poids d'entrée sont aléatoires et figés ; seuls les poids de sortie sont mis à jour en ligne par moindres carrés récursifs, sans rétropropagation. Avec un lot de 1, l'inversion matricielle devient une division scalaire (Tsukada et al., 2020, §3.1), ce qui rend l'apprentissage sur FPGA bon marché (ONLAD). 9, 10, 13

PatchCore Détection d'anomalies industrielles par banque de caractéristiques : les descripteurs de patchs d'images normales, extraits par un CNN pré-entraîné, sont réduits en un *coreset* ; le score est la distance au plus proche voisin. 99,1 d'AUROC image sur MVTec AD mais 92,2 sur un jeu réel d'huile alimentaire (Cui et al., 2023, Tables 10, 12) (Table 10 reconstruite par Docling) ; coûteux en mémoire, non porté sur FPGA dans le corpus. 9

QLR-CL *Quantized Latent Replay* : un MobileNet-V1 est coupé en un étage figé quantifié en INT8 et un étage adaptatif réentraîné en FP32, nourri d'un mélange de nouvelles images et de *latent replays* stockés quantifiés (Ravaglia et al., 2021, §III-C). Sur l'ASIC PULP VEGA (22 nm), 65 fois plus rapide et 37 fois plus économe qu'un STM32L4 (Ravaglia et al., 2021, §V-E). 8, 10, 11, 13

QNN *Quantized Neural Network* : réseau dont poids et activations sont en entiers de faible largeur (8 bits et moins), souvent entraîné en QAT. C'est la forme visée par FINN, FINN-R et hls4ml. Dans le corpus, la vérification formelle bit-exacte d'un QNN (encodage SMT) existe mais ne passe pas encore à l'échelle (Henzinger et al., 2020), et des fautes « stuck-at » sont injectées dans des QNN FINN sur ZCU104 (Gambardella et al., 2019). 13

BNN *Binarized Neural Network* : cas limite du QNN, poids et activations sur 1 bit (±1) ; le produit scalaire devient XNOR et comptage de bits (*popcount*), la BN suivie du signe un simple seuil, ce qui convient aux LUT d'un FPGA. Base de FINN (Umuroglu et al., 2016) ; un BNN s'encode aussi exactement en SAT pour la vérification (Narodytska et al., 2017, §4)[PP].

CNV Classifieur binaire de référence de FINN pour CIFAR-10 et SVHN : 3 fois (conv 3×3, conv 3×3, max-pooling) avec 64, 128 et 256 canaux, puis 2 couches denses de 512 (Umuroglu et al., 2016, §5.1). 80,1 % sur CIFAR-10 ; CNV-max tient 21 900 images/s sur ZC706 (Umuroglu et al., 2016, Tables 3–4). Réutilisé par FCMP et pour l'injection de fautes. 5

ReLU *Rectified Linear Unit*, $\max(0, x)$; variantes ReLU6 (écrêtée à 6) et leaky-ReLU (pente faible pour $x < 0$, utilisée par YOLOv2 à v4). Linéaire par morceaux, elle coûte une comparaison et se fusionne avec la quantification ; c'est l'activation de repli des toolflows : FINN-R remplace le leaky-ReLU de Tincy YOLO par ReLU (Blott et al., 2018, §5.1), Karki et al. y ramènent SiLU pour la DPU. 5, 6, 9, 12, 13

ResNet Famille de réseaux à connexions résiduelles (ResNet-18, -50…), qui permettent d'entraîner des réseaux profonds ; les connexions longues imposent des tampons en dataflow. Dans le corpus : ResNet-50 W1A2 en dataflow FINN sur Alveo (Petrica et al., 2020, Table IV) et Elastic-DF ;

ResNet18 figé sur DPU comme extracteur de SLDA (Piyasena et al., 2021, §III-A) ; un ResNet-8 dérivé dans hls4ml. 4–7, 10

SiLU *Sigmoid Linear Unit* (ou Swish), $x \cdot \sigma(x)$: activation lisse de YOLOv5, v8 et suivants. Elle exige une sigmoïde, absente des opérateurs natifs de la DPU et de FINN (Pappalardo et al., 2022, §VI-D) ; d'où des remplacements : ReLU chez Karki et al. (Karki et al., 2026, §III-B)[PP], HardSwish dans SATAY (Montgomerie-Corcoran et al., 2023, §III-B), OPReLU chez Yan et al. (Yan et al., 2024, §4). 5, 6

SkyNet Réseau de détection mono-objet conçu de bas en haut : un « Bundle » (DW-Conv3, PW-Conv1, BN, ReLU6) empilé 6 fois, choisi et dimensionné par recherche PSO contrainte par la latence GPU et FPGA, avec bypass et réordonnancement des cartes pour les petits objets, tête YOLO à 2 ancres (Zhang et al., 2019, §4–5)[PP]. Vainqueur de DAC-SDC'19 sur TX2 et sur Ultra96 (IoU 0,716, 25,05 FPS, 7,26 W) (Zhang et al., 2019, Tables 5–6)[PP]. 3, 5–8, 12

SLDA *Streaming Linear Discriminant Analysis* (Deep SLDA) : la dernière couche d'un CNN figé est remplacée par un classifieur linéaire qui met à jour en flux une moyenne par classe et une covariance partagée, sans rétropropagation ni replay (Piyasena et al., 2021, §III-A). Piyasena et al. l'accélèrent en PL derrière un ResNet18 sur DPU : apprentissage à 168,67 FPS contre 170,20 en inférence seule (Piyasena et al., 2021, Table V) (tableau reconstruit par Docling). 10, 11, 13

SqueezeNet Classifieur compact fondé sur des modules « fire » (compression 1×1 puis expansion 1×1 et 3×3). Dans le corpus, l'équipe SystemsETHZ de DAC-SDC le binarise et lui ajoute une tête YOLO : IoU de 0,49 à 25,97 FPS sur XC7Z020 (Xu et al., 2021, §5.2.1). 6

SSD *Single Shot MultiBox Detector* : détecteur en une étape à ancres (*default boxes*), qui prédit boîtes et classes sur plusieurs cartes de caractéristiques de résolutions décroissantes, puis applique un NMS. Dans le corpus : TGIIF, un SSD élagué en 8 bits, gagne le volet FPGA de DAC-SDC'18 (IoU 0,62 à 11,96 FPS sur XC7Z020) (Xu et al., 2021, §5.2.1) ; SSD MobileNet V1 est mesuré sur cartes embarquées (Alqahtani et al., 2024, §IV)[PP]. 6

VGG16 Classifieur de 2014 à 13 couches de convolution 3×3 et 3 couches denses, très régulier mais lourd. Sa régularité convient aux moteurs de calcul uniques, à l'inverse d'AlexNet (Venieris et al., 2018, §2.7) ; EF-Train l'entraîne sur ZCU102 (46,99 GFLOPS) (Tang et al., 2022, Table 8)[PP]. 4

YOLO *You Only Look Once* : famille de détecteurs en une étape, qui prédisent en une seule passe, sur une grille, des boîtes, un score d'objet et des scores de classe, puis filtrent les doublons par NMS. Depuis YOLOv3, l'architecture se découpe en backbone (extraction de caractéristiques), neck (fusion multi-échelle) et tête de détection. C'est le détecteur dominant du corpus : de Tiny YOLO sur Virtex-7 (REQ-YOLO) à YOLOv26n sur DPU, et le cas d'application visé par la thèse (détection de personnes). 2, 4–7, 10, 13

- **YOLOv2** Deuxième version (YOLO9000) : backbone Darknet-19, boîtes d'ancrage (*anchors*) obtenues par k-means, normalisation par lot, couche *passthrough* (reorg) qui ramène des caractéristiques fines, une seule échelle de sortie ; entrée usuelle 416×416. Face à YOLOv1, elle ajoute les ancres et supprime les couches denses. Dans le corpus, cible d'accélérateurs HLS sur Zynq : Zhang 2024 (YOLOv2 et v2-tiny) et Kim 2025, dont le YOLOv2 compte 29,5 GFLOPs mais ne tient que 2,33 GOPS sur XC7Z020 (Kim and Kim, 2025, Table 9). 6
  - **YOLOv2-tiny** Version réduite de YOLOv2 (« Tiny YOLO ») : pile de convolutions 3×3 et de max-pooling sans Darknet-19, sans passthrough, une seule échelle. REQ-YOLO la compresse (9 couches de convolution, entrée 416×416) (Ding et al., 2019, §2.1) ; FINN-R en dérive Tincy YOLO, et CoDeNet la prend comme référence (57,1 d'AP50 VOC en FP32) (Dong et al., 2020, Table 3). 6
- **YOLOv3** Troisième version : backbone Darknet-53 (blocs résiduels), prédiction à trois échelles par un neck de type FPN, trois ancres par échelle, classes en sigmoïdes indépendantes. Face à YOLOv2 : multi-échelle et backbone plus profond, d'où de meilleurs petits objets. Dans le corpus : version élaguée sur Zynq XC7Z035 avec suivi Deepsort (Zhai 2023), tête de MCUNetV2 sur MCU, base de Gaussian YOLOv3, mesures sur GPU embarqué (Feng 2022). 6, 7, 12
  - **YOLOv3-tiny** Version réduite de YOLOv3 : backbone de convolutions 3×3 et de max-pooling, deux échelles de sortie au lieu de trois. Détecteur le plus porté sur FPGA dans le corpus :

SATAY, Zhang 2024, Zhai 2023, la référence w4a4 de LP-YOLO (0,761 de mAP@0,5 VOC à 12,35 FPS sur PYNQ-Z1 (Gutierrez et al., 2026, §V)[PP]), Fata 2026 (élagué à 70 %, 2,54 MB en INT8 sur DPU (Fata and Elmannai, 2026, Table 5)), et le détecteur certifiable sur GPU de Belcaid et al. (2024). 4–8, 12, 13

YOLOv4 Quatrième version (2020) : backbone CSPDarknet53, module SPP et neck PANet, tête de YOLOv3 à ancres ; ajoute des techniques d'entraînement (Mosaic, CIoU) et l'activation Mish. Dans le corpus, Schlosser et al. la portent en INT8 sur DPU (ZCU104) et sur AGX Xavier : mAP COCO 37,5 contre 38,6, 1,31 contre 2,05 FPS/W (Schlosser et al., 2023, Table 1) ; Feng 2022 la mesure sur GPU embarqués. 5–8, 12

YOLOv5 Version d'Ultralytics (2020), sans article : réimplémentation en PyTorch, à ancres, backbone CSP, neck PANet, activation SiLU, déclinée en cinq tailles (n, s, m, l, x) par des multiplicateurs de profondeur et de largeur. Face à YOLOv4 : écosystème (entraînement, export ONNX) plus que rupture d'architecture. Dans le corpus : SATAY, Yan 2024, et un YOLOv5 de taille non rapportée sur KV260 (Vitis AI). 5, 6

YOLOv5n Taille *nano*, la plus petite de YOLOv5. SATAY la déploie sur ZCU104 en 640×640 : 26,7 de mAP50 COCO, 21,41 ms (Montgomerie-Corcoran et al., 2023, Table III), seul résultat de SATAY sur SoC embarqué. 5, 6, 8

YOLOv5s Taille *small* de YOLOv5. Yan et al. (2024) la quantifient entièrement en 4 bits (SiLU remplacée par OPReLU) sur ZCU102 : 79,1 FPS à 512×512 pour 12,3 W (Yan et al., 2024, §7.4, Table 3) ; SATAY la couvre aussi. 4, 6

YOLOv8 Version d'Ultralytics (2023) : tête découplée (classification et régression séparées) et *sans ancres*, blocs C2f (avec une opération *split*), activation SiLU, régression par Distribution Focal Loss ; tailles n à x. Face à YOLOv5 : abandon des ancres et tête découplée. Dans le corpus : SATAY, SMOF, Karki (sur DPU) et les mesures sur Jetson Orin de Rey et Alqahtani. 5, 8, 13

YOLOv8n Taille *nano* de YOLOv8. Sur DPU, Karki et al. doivent remplacer SiLU par ReLU et le *split* par des convolutions 1×1 ; la mAP50-95 passe de 0,64 à 0,34 sur KITTI et de 0,60 à 0,45 sur VOC après PTQ INT8 (Karki et al., 2026, Table II)[PP]. SMOF la traite en bfp8 sur VCU118 ; Rey la mesure sur Orin Nano (Rey et al., 2025, Table 4). 5–8, 12, 13

YOLOv8s Taille *small* de YOLOv8. Plus gros modèle couvert par SATAY, mais sur VCU118 (W8A16) et non sur un SoC embarqué (Montgomerie-Corcoran et al., 2023, Table III) ; aussi mesurée sur Jetson Orin (Rey et al., 2025, Table 4). 5, 6

YOLOv11n Taille *nano* de YOLO11 (Ultralytics, 2024), qui succède à YOLOv8 : blocs C3k2 et module d'attention C2PSA dans le backbone, tête sans ancres. L'attention et SiLU ne passent pas sur la DPU : Karki et al. approchent C2PSA par des convolutions 1×1 et un hard-sigmoid (Karki et al., 2026, §III-B)[PP]. Le modèle perd 2 points de mAP50-95 sur VOC (0,59 à 0,57) après INT8 (Karki et al., 2026, Table II)[PP]. 6, 12

YOLOv26n Taille *nano* de YOLO26 (Ultralytics), présentée comme détecteur de bout en bout sans NMS et sans Distribution Focal Loss, pour faciliter l'export vers l'embarqué. Seul détecteur récent mesuré sur FPGA dans le corpus : sur DPU B4096 (ZCU104), après les mêmes adaptations que YOLOv11n, 34,05 FPS de bout en bout sur une vidéo de personnes (Karki et al., 2026, §IV-C)[PP]; le NMS y est conservé (Karki et al., 2026, §III-B)[PP]. 5, 6, 8

LP-YOLO Détecteur YOLO allégé à une seule échelle (environ 0,35 M paramètres, 0,091 GMAC), entraîné en QAT avec Brevitas (w4a4 ; w2a4 pour la variante Slim en 256×256) et compilé par FINN sur PYNQ-Z1 (Gutierrez et al., 2026, §III-A, §IV-B)[PP]. LP-YOLO Slim tient 35,66 FPS pour 2,91 W de carte avec 0,594 de mAP@0,5 VOC (Gutierrez et al., 2026, §V)[PP]: seul détecteur FPGA du corpus mesuré sous 3 W. 5–8, 12, 13

REQ-YOLO Cadre de compression de Tiny YOLO (YOLOv2-tiny) : matrices bloc-circulantes (convolution par FFT) et quantification hétérogène par ADMM, choisie couche par couche pour équilibrer DSP et LUT ; tous les poids tiennent en BRAM (Ding et al., 2019, §3–5). Sur Virtex-7 XC7VX690T : 314,2 FPS pour 21 W, latence de 3,18 ms hors transferts et NMS (Ding et al., 2019, Tables 1–2). Plateforme PCIe, non embarquée. 4, 6, 12

Tincy YOLO Tiny YOLO modifié par FINN-R : poids binaires et activations 3 bits (W1A3)

sauf première et dernière couches (8 bits), leaky-ReLU remplacé par ReLU ; 36 % d'opérations en moins pour une mAP VOC qui passe de 57,1 à 50,1 (Blott et al., 2018, §5.1). Premier YOLO quantifié sur SoC Zynq du corpus, rapporté en GOp/s seulement (133 sur PYNQ-Z1, 288 sur Ultra96), sans FPS ni latence (Blott et al., 2018, Table 5). 5, 6

TinyissimoYOLO Détecteur pour microcontrôleur inspiré de YOLOv1 : entrée 88×88, convolutions 3×3 uniquement, une couche dense, sortie en grille 4×4, INT8 par QAT, dimensionné pour les 442 kB de poids du MAX78000 (Moosmann et al., 2023, §II-A). 58,5 % de mAP sur VOC réduit à 3 classes et 196 µJ par inférence sur MAX78000, MCU seul (Moosmann et al., 2023, §III, Table II). Point de comparaison MCU des détecteurs FPGA. 7, 8, 13, 14

Glossaire des métriques et unités

AP Average Precision : aire sous la courbe précision-rappel d'une classe, les détections étant jugées justes au-delà d'un seuil d'IoU ; de 0 à 1 (ou en %), plus haut = mieux. Le mode d'interpolation de la courbe varie selon le protocole (VOC, COCO) [hors corpus]. Piège : dans la notation COCO, « AP » désigne la moyenne sur les classes et sur les seuils 0,50 à 0,95, c'est-à-dire la mAP50-95 ; une AP à une seule classe (véhicules) n'est pas comparable à une mAP multi-classes (Zhai et al., 2023, Table 8). 6, 12, 13

AP50 AP au seuil d'IoU 0,5 (AP@0.5). Chez Zhai et al., l'AP@0.5 est identique en FP32 et en INT16 à trois décimales, ce qui laisse penser qu'elle n'a pas été remesurée sur la carte (Zhai et al., 2023, Table 8) *(déduit)*. 6

AUROC Aire sous la courbe ROC (taux de vrais positifs contre taux de faux positifs), notée aussi AUC ; de 0,5 (hasard) à 1, souvent donnée en %. Indépendante du seuil, elle ne dit donc rien du point de fonctionnement retenu ; le survey de Cui et al. ne traite pas le réglage du seuil (Cui et al., 2023, §VI-B) (AUROC image de 99,1 pour PatchCore sur MVTec AD, 92,2 sur un jeu réel, Tables 10 et 12). Piège : AUC évaluées en TensorFlow et non sur le cœur FPGA (Tsukada et al., 2020, §5–6). 9, 10, 12, 13

BD-rate Bjøntegaard delta rate : écart moyen de débit, en %, entre deux courbes débit-distorsion à qualité égale ; négatif = gain, positif = perte. La perte d'un Ballé2018 quantifié en INT8 passe de 30,0 à 6,30 %, en simulation seulement (Fang and Chen, 2025, Table I)[PP]. 9

bpp Bits par pixel du flux compressé ; plus bas = meilleure compression à qualité égale. Piège : le débit n'a de sens que si le codage entropique est compté ; chez F-LIC, il tourne sur le PC hôte (Sun et al., 2022, Fig. 1), et Φ-Sat-2 compresse d'un facteur 8 fixe, sans codage entropique (Guerrisi et al., 2023, §II-A). 9

DAC-SDC System Design Contest de la conférence DAC : détection d'un seul objet par image (images de drone), sur PYNQ-Z1 et Jetson TX2, mesurée par les organisateurs. Score = IoU×(1+ES), ES logarithmique en énergie avec des bases différentes pour FPGA et GPU ; débit minimal de 5 FPS sur FPGA et 20 sur GPU ; FPGA au wattmètre, TX2 par ses capteurs internes (Xu et al., 2021, §2.3). Son IoU ne se compare pas à une mAP, et le classement ne passe pas d'une catégorie à l'autre. 5–8, 12, 13

F1 Moyenne harmonique de la précision P et du rappel R : $F_1 = 2PR/(P+R)$, de 0 à 1 (ou en %), plus haut = mieux ; dépend du seuil de décision. Exemple de perte à la quantification : F1 nul après PTQ INT8 seule, 92,77 après fine-tuning contre 94,05 en FP32, et 95,29 sur Xavier NX (Castelino et al., 2024, Tables III, VI). 8, 9, 12

FIT Failures In Time : nombre de défaillances pour 10^9 heures de fonctionnement ; plus bas = mieux. Li et al. calculent FIT = $R_{\text{raw}} \times$ taille $\times\, P(\text{SDC})$ avec $R_{\text{raw}} = 20{,}49$ FIT/Mb extrapolé à 16 nm (Li et al., 2017, §4.7), citent la cible de moins de 10 FIT d'ISO 26262 (Li et al., 2017, §1) et ramènent le FIT d'Eyeriss de 8,55 à 0,35 (Li et al., 2017, §6.2). Le passage d'un FIT mesuré à un SIL ou un PL n'est traité nulle part *(déduit)*. 13

FLOPs Nombre d'opérations en virgule flottante d'un modèle (un compte, pas un débit). L'usage confond souvent FLOPs et MAC (facteur 2) [hors corpus]. Les FLOPs ne prédisent pas la latence réelle (Luo et al., 2024, §2.3). 3, 10, 11

FPPI False Positives Per Image : nombre moyen de fausses détections par image. Le taux d'échec (miss rate) se lit à FPPI fixé ; le mMR du protocole Caltech en fait une moyenne logarithmique, plus bas = mieux. Sur CrowdHuman, un FPN atteint un mMR de 50,42 sur le corps entier contre 10,08 sur Caltech (Shao et al., 2018, Table 7)[PP]. Le mMR reste une moyenne : il ne dit pas le taux de personnes manquées à un instant ni le pire cas, ce qui compte pour la sûreté *(déduit)*. 13

FPS Images (frames) traitées par seconde ; plus haut = mieux. Ce n'est pas l'inverse de la latence dès que le calcul est pipeliné ou groupé (une « latence » de 5,6 ns qui est en fait un débit amorti, pour 85 ns réels (Que et al., 2024, §IV-E)). Pièges : FPS du cœur du réseau ou de bout en bout (NMS et pré-traitement sur l'ARM exclus (Magalhães et al., 2022, §2.5), post-traitement mono-cœur qui limite à 34,05 FPS (Karki et al., 2026, §IV-C)[PP]) ; batch (batch 4 sur FPGA pour SkyNet[PP] (Zhang et al., 2019, Tables 5–7), batch 1 contre 8 (Aggarwal et al., 2024, Table VI)) ; résolution d'entrée et jeu (256–320 pixels contre 640). 3–10, 12–14

FPS/W Débit par watt, FPS/P, soit des images par joule ; son inverse est l'énergie par image. Plus haut = mieux. N'a de sens qu'à portée de puissance égale (colonne `power_scope` : puce, carte, carte moins le repos, estimée). À même méthode (carte au multimètre), YOLOv4 INT8 tient 1,31 FPS/W sur ZCU104 contre 2,05 sur AGX Xavier (Schlosser et al., 2023, Table 1) ; 2,6 FPS/W présentés comme efficaces sont sous quatre des cinq travaux cités (Al Amin and Obermaisser, 2025, §IV-B, Table I). 5–8, 12

GFLOPS Giga-opérations flottantes par seconde : le GOPS d'un calcul en virgule flottante. Exemples : 61,62 GFLOPS en FP32 sur Virtex-7 (Zhang et al., 2015, Tables 7–8) ; 28,15 GFLOPS en apprentissage sur ZCU102, à batch 128 (Tang et al., 2022, §6)[PP]. 4, 10

GOP Giga-opérations : nombre d'opérations (10^9) d'une inférence, propriété du modèle et de la résolution, pas du matériel ; à ne pas confondre avec GOPS (un débit). Exemple : 5,7 GOP pour YOLOv3-tiny, en comptant un MAC pour deux opérations (Zhang et al., 2024, Table 5). 5, 6

GOPS Giga-opérations par seconde (GOP/s), débit de calcul ; plus haut = mieux. Pour un réseau, GOPS = GOP × FPS. Pièges : 1 MAC compte pour 2 opérations, convention pas toujours rappelée (Zhang et al., 2024, Table 5) ; GOPS de crête contre GOPS effectifs sur un réseau ; GOPS projetés sur une FPGA hypothétique comparés à des GOPS mesurés (Coutinho et al., 2019, Tables 16–17) ; GOPS sans FPS pour un détecteur (Blott et al., 2018, Table 5), ou « 10,65x » entre réseaux de tailles différentes à FPS voisins (Toupas et al., 2024, §V-D)[PP]. 4, 12

GOPS/DSP Débit rapporté au nombre de DSP utilisés, censé mesurer le rendement de l'architecture ; plus haut = mieux. Valable seulement dans une même famille de FPGA (Venieris et al., 2018, §3.1). Elle efface la précision : 4 multiplications 4 bits par DSP donnent 0,49 GOPS/DSP (Yan et al., 2024, Table 3), un MAC INT16 par DSP 0,018 (Kim and Kim, 2025, Table 9) ; comparaisons de W1A8 à W32A32 (Montgomerie-Corcoran et al., 2023, Table III). 4, 6, 12

GOPS/W Efficacité énergétique de calcul ; plus haut = mieux. Hérite des pièges des GOPS (crête ou effectifs) et de ceux de la puissance (portée) : GOPS/W calculés sur une puissance dynamique comparés à des GOPS/W totaux (Zhai et al., 2023, Table 8). 12

IoU Intersection over Union (indice de Jaccard) entre une boîte prédite A et une boîte de référence B : IoU = $|A \cap B|/|A \cup B|$, de 0 à 1, plus haut = mieux. Sert de seuil pour décider qu'une détection est juste (AP, mAP). Employée seule comme précision, elle n'est pas comparable à une mAP : DAC-SDC note l'IoU moyenne d'un seul objet par image (Xu et al., 2021, §2.3), et SkyNet[PP] passe de 0,741 à 0,727 puis 0,680 en quantifiant (Zhang et al., 2019, Table 7). Un seuil de Jaccard sert aussi à définir une erreur critique de détecteur sous radiations (Rech, 2024, §II-C). 5–8, 12

MAC Multiply-Accumulate : une multiplication suivie d'une addition, opération de base des convolutions ; compte pour 2 opérations dans les GOPS (Zhang et al., 2024, Table 5). Le nombre de MAC par DSP dépend de la précision : un MAC INT16 (Kim and Kim, 2025, §3.1), deux en 8 bits (Zhang et al., 2024, §3.1), quatre en 4 bits (Yan et al., 2024, §4–5). 4, 11

mAP mean Average Precision : moyenne des AP par classe ; plus haut = mieux. Sans seuil précisé, le sens dépend du jeu (mAP50 sur VOC, mAP50-95 sur COCO). Pièges relevés : le jeu de données (COCO à 80 classes, VOC à 20, une classe filtrée de VOC chez LP-YOLO[PP] (Gutierrez et al., 2026, §V), jeu privé à une cible (Rey et al., 2025, Table 4)) ; la résolution d'entrée ; la mesure simulée et

non faite sur la carte (quantification simulée par ONNX Runtime (Montgomerie-Corcoran et al., 2023, §IV-A)) ; seules des pertes relatives données (Ding et al., 2019, §6.2). La mAP ne mesure pas la qualité de l'incertitude (Feng et al., 2020, §IV-D). 4–8, 10, 12–14

mAP50 mAP au seuil d'IoU 0,5 (notée aussi mAP@0.5, mAP@0,5) ; métrique principale de VOC. Plus indulgente que la mAP50-95 sur la localisation : les deux ne se comparent pas. Exemple : 59,4 de mAP@0,5 sur les seuls véhicules de VOC pour LP-YOLO Slim^{PP} (Gutierrez et al., 2026, §V) ; 73,8 sur VOC, 1,5 point sous le flottant, sans dire si la mesure est faite sur la carte (Yan et al., 2024, Table 2). 5, 6

mAP50-95 mAP moyennée sur les seuils d'IoU de 0,50 à 0,95 par pas de 0,05 (notée aussi mAP@0.5:0.95) ; métrique principale de COCO. Mesurée sur la carte, elle révèle des pertes fortes et irrégulières : TensorRT INT8 38,6 contre DPU INT8 37,5 pour YOLOv4 (Schlosser et al., 2023, Table 1) ; YOLOv8n de 0,64 à 0,34 sur KITTI, YOLOv11n de 0,75 à 0,71 sur un jeu de personnes (Karki et al., 2026, Table II)^{PP}; 84,2 à 71,9 en INT8 sur Jetson (Rey et al., 2025, Table 4). 0, 5–7, 12, 13

PSNR Peak Signal-to-Noise Ratio : $10\log_{10}(\mathrm{MAX}^2/\mathrm{MSE})$, en dB, plus haut = mieux. Ne se lit qu'avec le débit (bpp) : un point débit-distorsion, pas une qualité absolue. Exemple : 29,94 dB à 0,2542 bpp pour F-LIC sur VCU118 (Sun et al., 2022, Fig. 6). 9

SIL Safety Integrity Level (IEC 61508) : niveau d'intégrité de sécurité d'une fonction, de SIL 1 à SIL 4 (le plus exigeant), associé à une probabilité de défaillance dangereuse par heure ou à la sollicitation. Dans le corpus, l'IEC 61508 n'apparaît que comme contexte (Li et al., 2017, §1, §2.3) ; l'équivalent automobile (ASIL, ISO 26262) est plus présent (Rech, 2024, §III-C), et aucune fiche ne relie une précision de détection ou un FIT à un SIL *(déduit)*. 13

SSIM Structural Similarity : indice de similarité perceptuelle (luminance, contraste, structure), au plus 1, plus haut = mieux ; MS-SSIM en est la version multi-échelle. Exemple : 0,9647 sur VPU Myriad 2 contre 0,9651 en référence (Guerrisi et al., 2023, §III-B–C). 9

TOPS Téra-opérations par seconde (10^{12} op/s). Dans le corpus, presque toujours une valeur de crête : 275 TOPS annoncés par le constructeur pour une Jetson AGX Orin (Luo et al., 2024, Table 4) ; 66 TOPS de pic binaire sur ZU19EG, environ 16 fois le pic 8 bits (Umuroglu et al., 2016, §3.1). Une crête dépend de la précision et ne prédit pas le débit sur un réseau réel *(déduit)*. 3, 5

WCET Worst-Case Execution Time : borne supérieure, obtenue par analyse statique, du temps d'exécution d'un programme sur une cible donnée ; exigé par DO-178C. Dans le corpus, seul ACETONE le calcule (OTAWA), sur un lpc2138 non représentatif de la cible mesurée, et l'analyse échoue pour AlexNet (De Albuquerque Silva et al., 2022, §4.2, §5). Le WCET entre PS et PL n'est pas traité (Sali et al., 2025, §III-B). 13

BRAM Block RAM : mémoire on-chip en blocs de 36 Kb, divisibles en deux de 18 Kb [hors corpus]; comptée en blocs dans `benchmarks.csv`, pas en Mo. Piège : un nombre de BRAM dit peu de la capacité utilisée, la forme des mémoires de poids s'accordant mal aux blocs (52,9 % d'occupation pour un ResNet-50 (Petrica et al., 2020, Table IV)^{PP}). Les BRAM sont exclues de l'injection de fautes de Libano et al. (Libano et al., 2019, §III-A). 4–6, 13

URAM UltraRAM : blocs de mémoire on-chip plus grands (288 Kb) des UltraScale+ [hors corpus]. Sur KV260, un design place les couches 11 à 21 en URAM et occupe 100 % des BRAM et des URAM (Fata and Elmannai, 2026, Table 9). 4, 6

DSP Bloc arithmétique câblé (DSP48 chez AMD : multiplieur et accumulateur), la ressource rare des petites FPGA (220 sur le XC7Z020 de la PYNQ-Z1 (Xu et al., 2021, §2.2)). La quantification change son rendement (voir GOPS/DSP) ; la QAT et l'élagage font passer un modèle de 6 377 à 174 DSP (Aarrestad et al., 2021, Table 3). 4–6, 9, 12

FF Flip-flop : bascule de registre de la logique programmable. Un gain de FF n'a de sens qu'à débit égal : « −26 % de FF » face à des accélérateurs de 10,5 à 62,9 GOPS, pour 2,33 GOPS (Kim and Kim, 2025, Table 9). 6, 12

LUT Look-Up Table : cellule logique programmable de la FPGA (6 entrées sur les familles AMD récentes [hors corpus]), qui réalise la logique et de petites mémoires. Les architectures streaming déplacent le calcul des DSP vers les LUT : l'accumulation par popcount coûte à peu près moitié

moins de LUT qu'une accumulation signée (Umuroglu et al., 2016, §4.2.1). 4–6, 9, 13
W Watt, unité de puissance. Dans le corpus, de quelques mW (puissance dynamique (Coutinho et al., 2019, Table 16)) et 0,04 W sur MCU GAP9 (Wibowo et al., 2024, Table II) à 2–23 W sur SoC FPGA et environ 120–134 W pour deux Alveo U280 (Alonso et al., 2021, Table 4). Toujours lire la portée : une ZC706 consomme environ 7 W au repos, inclus dans une mesure de carte (Umuroglu et al., 2016, §5.2.2). 3–14
mW Milliwatt (10^{-3} W), ordre de grandeur des MCU et ASIC : 62 mW mesurés sur la puce VEGA (Ravaglia et al., 2021, Table IV). 3, 10, 11
J Joule, unité d'énergie : $E = P \times t = P/\text{FPS}$ par image ; plus bas = mieux. Exemples : 1,41 J par image 720p sur VCU118 (Sun et al., 2022, Fig. 6) ; 1,22 J par image d'apprentissage sur ZCU102 (Aggarwal et al., 2024, Table VI). 8–11, 13
mJ Millijoule. Exemples : 21,52 mJ par inférence sur DPU ZCU104 (Castelino et al., 2024, Table VI) ; 11 à 23 mJ par nouvelle classe sur MCU (Wibowo et al., 2024, Tables II, IV). 6, 8–11
µJ Microjoule (10^{-6} J), énergie d'une inférence de petit réseau : 196 µJ sur MAX78000 (Moosmann et al., 2023, §III) ; 2 535 et 44 330 µJ pour FINN et hls4ml sur Pynq-Z2 (Borras et al., 2022, Table 5)[PP]. 5, 7, 8
s Seconde. Rencontrée pour les traitements lents : environ 12,6 s par image pour YOLOv2 sur XC7Z020 (Kim and Kim, 2025, §4.2) ; 2,07 s par événement d'apprentissage sur VEGA (Ravaglia et al., 2021, Table IV). 4–6, 9–11
ms Milliseconde, unité usuelle de latence d'un détecteur : de 15,2 ms de bout en bout (Zhang et al., 2024, Table 5) à 54,42 ms (Schlosser et al., 2023, Table 1). Préciser ce que couvre la latence (inférence seule, PL seule, bout en bout). 4–10, 13
µs Microseconde : latence d'un petit réseau hls4ml, environ 5 µs sur VU9P et 171 µs sur XC7Z020 (Aarrestad et al., 2021, Tables 3–4). 5
ns Nanoseconde : latences de réseaux minuscules en logique pure, 85 ns (17 cycles) pour un VAE dense (Que et al., 2024, §IV-E), 233 ns chez Libano et al. (Libano et al., 2020, §V). 9
MHz Mégahertz, fréquence d'horloge ; débit et puissance en dépendent. Dans le corpus, de 100 à 300 MHz pour la logique FPGA ; un Tegra X1 bridé à 76,8 MHz pour égaliser le budget de puissance (Venieris and Bouganis, 2017, §3.1). 5, 6, 8, 10
Ko Kilo-octet (KB, kB) ; le corpus ne distingue pas 1 000 et 1 024 octets. Ordre de grandeur des SRAM de MCU : entraînement sous 256 KB, 206 KB mesurés (Lin et al., 2022, §3.2) ; 630 Ko de BRAM sur PYNQ-Z1 (Xu et al., 2021, §2.2). 7, 10, 11, 13
Mo Mégaoctet (MB) ; 1 Mo = 8 Mbit, à ne pas confondre avec le Mbit des fiches techniques. Exemple : 3,5 Mo de tampons de rejeu contre 48 Mo (Aggarwal et al., 2024, Tables II, VI). 3, 5, 10
Mbit Mégabit, unité des capacités mémoire on-chip des FPGA. Un YOLOv3-tiny demanderait environ 34,7 Mbit de poids à 4 bits, pour environ 4,9 Mbit de BRAM sur le XC7Z020 (Gutierrez et al., 2026, §III-A)[PP]. 4

Références

Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergo Jindariani, Kevin Pedro, Nhan Tran, Mia Liu, Edward Kreinar, Zhenbin Wu, and Duc Hoang. Fast convolutional neural networks on FPGAs with hls4ml. *Mach. Learn. : Sci. Technol. 2 045015 (2021)*, 2021. doi : 10.1088/2632-2153/ac0ea1. URL `https://arxiv.org/abs/2101.05108`. arXiv :2101.05108.

Kamel Abdelouahab, Maxime Pelcat, Jocelyn Serot, and François Berry. Accelerating CNN inference on FPGAs : A Survey. Technical Report, Université Clermont Auvergne (HAL), 2018, 2018. URL `https://arxiv.org/abs/1806.01683`. arXiv :1806.01683. Preprint non relu par les pairs.

Shivam Aggarwal, Kuluhan Binici, and Tulika Mitra. Chameleon : Dual Memory Replay for Online

Continual Learning on Edge Devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 43(6)*, 2024. URL https://ieeexplore.ieee.org/document/10375748/.

Rashed Al Amin and Roman Obermaisser. Real-Time Object Detection and Classification using YOLO for Edge FPGAs. In *2025 International Symposium ELMAR*, 2025. doi : 10.1109/ELMAR66948.2025.11193980. URL https://arxiv.org/abs/2507.18174. arXiv :2507.18174.

Tobias Alonso, Lucian Petrica, Mario Ruiz, Jakoba Petri-Koenig, Yaman Umuroglu, Ioannis Stamelos, Elias Koromilas, Michaela Blott, and Kees Vissers. Elastic-DF : Scaling Performance of DNN Inference in FPGA Clouds through Automatic Partitioning. *ACM Trans. Reconfigurable Technol. Syst. 15(2), 15*, 2021. doi : 10.1145/3470567. URL https://doi.org/10.1145/3470567.

Daghash K Alqahtani, Muhammad Aamir Cheema, Maria A Rodriguez, and Adel N Toosi. A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices. arXiv preprint, 2024. URL https://arxiv.org/abs/2409.16808. arXiv :2409.16808. Preprint non relu par les pairs.

Léo Andéol, Thomas Fel, Florence De Grancey, and Luca Mossina. Confident Object Detection via Conformal Prediction and Conformal Risk Control : an Application to Railway Signaling. arXiv preprint, 2023. URL https://arxiv.org/abs/2304.06052. arXiv :2304.06052. Preprint non relu par les pairs.

Jebacyril Arockiaraj, Dhruv Parikh, and Viktor Prasanna. ImageHD : Energy-Efficient On-Device Continual Learning of Visual Representations via Hyperdimensional Computing. In *2026 IEEE 34th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2026. doi : 10.1109/FCCM68464.2026.00041. URL https://arxiv.org/abs/2604.21280. arXiv :2604.21280.

Rob Ashmore, Radu Calinescu, and Colin Paterson. Assuring the Machine Learning Lifecycle : Desiderata, Methods, and Challenges. arXiv preprint, 2019. URL https://arxiv.org/abs/1905.04223. arXiv :1905.04223. Preprint non relu par les pairs.

Mohammed Belcaid, Eric Bonnafous, Louis Crison, Christophe Faure, Eric Jenn, and Claire Pagetti. Certified ML Object Detection for Surveillance Missions. In *ERTS 2024 (12th European Congress on Embedded Real Time Software and Systems), Toulouse*, 2024. URL https://arxiv.org/abs/2406.12362.

Michaela Blott, Thomas B. Preußer, Nicholas J. Fraser, Giulio Gambardella, Kenneth O'brien, Yaman Umuroglu, Miriam Leeser, and Kees Vissers. FINN-R : An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks. *ACM Transactions on Reconfigurable Technology and Systems 11(3)*, 2018. doi : 10.1145/3242897. URL https://arxiv.org/abs/1809.04570. arXiv :1809.04570.

Hendrik Borras, Giuseppe Di Guglielmo, Javier Duarte, Nicolò Ghielmetti, Ben Hawks, Scott Hauck, Shih-Chieh Hsu, Ryan Kastner, Jason Liang, Andres Meza, Jules Muhizi, Tai Nguyen, Rushil Roy, Nhan Tran, Yaman Umuroglu, Olivia Weng, Aidan Yokuda, and Michaela Blott. Open-source FPGA-ML codesign for the MLPerf Tiny Benchmark. arXiv preprint, 2022. URL https://arxiv.org/abs/2206.11791. arXiv :2206.11791. Preprint non relu par les pairs.

Javier Campos, Jovan Mitrevski, Nhan Tran, Zhen Dong, Amir Gholaminejad, Michael W. Mahoney, and Javier Duarte. End-to-end codesign of Hessian-aware quantized neural networks for FPGAs. *ACM Trans. Reconfigurable Technol. Syst. 17(3), 36*, 2024. doi : 10.1145/3662000. URL https://doi.org/10.1145/3662000.

Cornell Castelino, Shashwat Khandelwal, Shanker Shreejith, and Sharatchandra Varma Bogaraju. An Energy-Efficient Artefact Detection Accelerator on FPGAs for Hyper-Spectral Satellite

Imagery. In *27th Euromicro Conference on Digital System Design (DSD), 2024*, 2024. URL https://arxiv.org/abs/2407.17647.

Maria G. F. Coutinho, Matheus F. Torquato, and Marcelo A. C. Fernandes. Deep Neural Network Hardware Implementation Based on Stacked Sparse Autoencoder. *IEEE Access 7, 40674-40694*, 2019. doi : 10.1109/ACCESS.2019.2907261. URL https://doi.org/10.1109/ACCESS.2019.2907261.

Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A Survey on Unsupervised Anomaly Detection Algorithms for Industrial Images. *IEEE Access*, 2023. doi : 10.1109/ACCESS.2023.3282993. URL https://doi.org/10.1109/ACCESS.2023.3282993.

Iryna De Albuquerque Silva, Thomas Carle, Adrien Gauffriau, and Claire Pagetti. ACETONE : Predictable Programming Framework for ML Applications in Safety-Critical Systems. In *34th Euromicro Conference on Real-Time Systems (ECRTS 2022), LIPIcs 231*, 2022. doi : 10.4230/LIPIcs.ECRTS.2022.3. URL https://doi.org/10.4230/LIPIcs.ECRTS.2022.3.

Caiwen Ding, Shuo Wang, Ning Liu, Kaidi Xu, Yanzhi Wang, and Yun Liang. REQ-YOLO : A Resource-Aware, Efficient Quantization Framework for Object Detection on FPGAs. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays 33–42*, 2019. doi : 10.1145/3289602.3293904. URL https://doi.org/10.1145/3289602.3293904.

Zhen Dong, Dequan Wang, Qijing Huang, Yizhao Gao, Yaohui Cai, Tian Li, Bichen Wu, Kurt Keutzer, and John Wawrzynek. CoDeNet : Efficient Deployment of Input-Adaptive Object Detection on Embedded FPGAs. In *FPGA 2021*, 2020. doi : 10.5281/zenodo.4341394. URL https://arxiv.org/abs/2006.08357.

EASA. Concept Paper : Guidance for Level 1 & 2 Machine Learning Applications. Technical report, European Union Aviation Safety Agency (EASA), Concept Paper, Issue 02, 2024.

EASA and Daedalean. Concepts of Design Assurance for Neural Networks (CoDANN). Technical report, EASA AI Task Force and Daedalean AG, public report extract, version 1.0, 2020.

Christophe El Zeinaty, Wassim Hamidouche, Glenn Herrou, and Daniel Menard. Designing Object Detection Models for TinyML : Foundations, Comparative Analysis, Challenges, and Emerging Solutions. *ACM Comput. Surv. 58(2), 50*, 2025. doi : 10.1145/3744339. URL https://doi.org/10.1145/3744339.

Farah Fahim, Benjamin Hawks, Christian Herwig, James Hirschauer, Sergo Jindariani, Nhan Tran, Luca P Carloni, Giuseppe Di Guglielmo, Philip Harris, Jeffrey Krupa, Dylan Rankin, Manuel Blanco Valentin, Josiah Hester, Yingyi Luo, John Mamish, Seda Orgrenci-Memik, Thea Aarrestad, Hamza Javed, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, Sioni Summers, Javier Duarte, Scott Hauck, Shih-Chieh Hsu, Jennifer Ngadiuba, Mia Liu, Duc Hoang, Edward Kreinar, and Zhenbin Wu. hls4ml : An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices. In *TinyML Research Symposium 2021*, 2021. URL https://arxiv.org/abs/2103.05579. arXiv :2103.05579.

Jiaxun Fang and Li Chen. A Multi-Stage Optimization Framework for Deploying Learned Image Compression on FPGAs. arXiv preprint, 2025. URL https://arxiv.org/abs/2511.17135. arXiv :2511.17135. Preprint non relu par les pairs.

John S. Fata and Wafa M. Elmannai. Low-Cost FPGA-Enhanced CNN Accelerator for Real-Time YOLO Object Detection and Classification. *IEEE Access 14, 34614-34642*, 2026. doi : 10.1109/ACCESS.2026.3669451. URL https://doi.org/10.1109/ACCESS.2026.3669451.

Di Feng, Ali Harakeh, Steven Waslander, and Klaus Dietmayer. A Review and Comparative Study on Probabilistic Object Detection in Autonomous Driving. *IEEE Transactions on Intelligent Transportation Systems*, 2020. doi : 10.1109/TITS.2021.3096854. URL https://arxiv.org/abs/2011.10671.

Haogang Feng, Gaoze Mu, Shida Zhong, Peichang Zhang, and Tao Yuan. Benchmark Analysis of YOLO Performance on Edge Intelligence Devices. *Cryptography 6(2), 16*, 2022. doi : 10.3390/cryptography6020016. URL https://doi.org/10.3390/cryptography6020016.

Giulio Gambardella, Johannes Kappauf, Michaela Blott, Christoph Doehring, Martin Kumm, Peter Zipf, and Kees Vissers. Efficient Error-Tolerant Quantized Neural Network Accelerators. In *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2019. doi : 10.1109/DFT.2019.8875314. URL https://arxiv.org/abs/1912.07394.

Ekaterina Govorkova, Ema Puljak, Thea Aarrestad, Thomas James, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, Nicolò Ghielmetti, Maksymilian Graczyk, Sioni Summers, Jennifer Ngadiuba, Thong Q Nguyen, Javier Duarte, and Zhenbin Wu. Autoencoders on FPGAs for real-time, unsupervised new physics detection at 40 MHz at the Large Hadron Collider. *Nature Machine Intelligence 4, 154 (2022)*, 2021. doi : 10.1038/s42256-022-00441-3. URL https://arxiv.org/abs/2108.03986. arXiv :2108.03986.

Katya Govorkova, Julian Garcia Pardinas, Vladimir Loncar, Victoria Nguyen, Sebastian Schmitt, Marco Pizzichemi, Loris Martinazzoli, and Eluned Anne Smith. Enabling Low-Latency Machine learning on Radiation-Hard FPGAs with hls4ml. *Machine Learning : Science and Technology*, 2026. doi : 10.1088/2632-2153/ae8218. URL https://arxiv.org/abs/2602.15751. arXiv :2602.15751.

Giorgia Guerrisi, Fabio Del Frate, and Giovanni Schiavon. Artificial Intelligence Based On-Board Image Compression for the Φ-Sat-2 Mission. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing 16, 8063-8075*, 2023. doi : 10.1109/JSTARS.2023.3296485. URL https://doi.org/10.1109/JSTARS.2023.3296485.

Daniel Gutierrez, Antonio Cuesta, Jorge Fe, Bruno Gutierrez, and Rashed Al Koutayni. NPU Accelerator : Quantized Real-Time Vehicle Detection on PYNQ-Z1 Using FINN. arXiv preprint, 2026. URL https://arxiv.org/abs/2609.24757. arXiv :2609.24757. Preprint non relu par les pairs.

Roni-Jussi Halme, Minna Lanz, Joni Kämäräinen, Roel Pieters, Jyrki Latokartano, and Antti Hietanen. Review of vision-based safety systems for human-robot collaboration. *Procedia CIRP 72, 111-116*, 2018. doi : 10.1016/j.procir.2018.03.043. URL https://doi.org/10.1016/j.procir.2018.03.043.

Thomas A Henzinger, Anna Lukina, and Christian Schilling. Outside the Box : Abstraction-Based Monitoring of Neural Networks. In *ECAI 2020*, 2019. doi : 10.3233/FAIA200375. URL https://arxiv.org/abs/1911.09032.

Thomas A Henzinger, Mathias Lechner, and Đorđe Žikelić. Scalable Verification of Quantized Neural Networks (Technical Report). In *AAAI 2021*, 2020. URL https://arxiv.org/abs/2012.08185.

Seyed Hani Hozhabr and Roberto Giorgi. A Survey on Real-Time Object Detection on FPGAs. *IEEE Access 13, 38195-38238*, 2025. doi : 10.1109/ACCESS.2025.3544515. URL https://doi.org/10.1109/ACCESS.2025.3544515.

Xiaowei Huang, Daniel Kroening, Wenjie Ruan, James Sharp, Youcheng Sun, Emese Thamo, Min Wu, and Xinping Yi. A Survey of Safety and Trustworthiness of Deep Neural Networks : Verification, Testing, Adversarial Attack and Defence, and Interpretability. arXiv preprint, 2018. URL https://arxiv.org/abs/1812.08342. arXiv :1812.08342. Preprint non relu par les pairs.

Murat Isik, Matthew Oldland, and Lifeng Zhou. An Energy-Efficient Reconfigurable Autoencoder Implementation on FPGA. arXiv preprint, 2023. URL https://arxiv.org/abs/2301.07050. arXiv :2301.07050. Preprint non relu par les pairs.

Suraj Karki, Qazi Arbab Ahmed, and Thorsten Jungeblut. No Attention, No Problem : DPU-Aware Attention Approximation in Modern YOLO on FPGA. arXiv preprint, 2026. URL https://arxiv.org/abs/2607.13106. arXiv :2607.13106. Preprint non relu par les pairs.

Huimin Kim and Tae-Kyoung Kim. Design and Implementation of a YOLOv2 Accelerator on a Zynq-7000 FPGA. *Sensors*, 2025. doi : 10.3390/s25206359. URL https://doi.org/10.3390/s25206359.

Dawei Li, Serafettin Tasci, Shalini Ghosh, Jingwen Zhu, Junting Zhang, and Larry Heck. RILOD : Near Real-Time Incremental Learning for Object Detection at the Edge. arXiv preprint, 2019. URL https://arxiv.org/abs/1904.00781. arXiv :1904.00781. Preprint non relu par les pairs.

Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W. Keckler. Understanding error propagation in deep learning neural network (DNN) accelerators and applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis 8*, 2017. doi : 10.1145/3126908.3126964. URL https://doi.org/10.1145/3126908.3126964.

F. Libano, B. Wilson, J. Anderson, M. J. Wirthlin, C. Cazzaniga, C. Frost, and P. Rech. Selective Hardening for Neural Networks in FPGAs. *IEEE Transactions on Nuclear Science 66(1), 216-222*, 2019. doi : 10.1109/TNS.2018.2884460. URL https://doi.org/10.1109/TNS.2018.2884460.

F. Libano, B. Wilson, M. Wirthlin, P. Rech, and J. Brunhaver. Understanding the Impact of Quantization, Accuracy, and Radiation on the Reliability of Convolutional Neural Networks on FPGAs. *IEEE Transactions on Nuclear Science 67(7), 1478-1484*, 2020. doi : 10.1109/TNS.2020.2983662. URL https://doi.org/10.1109/TNS.2020.2983662.

Ji Lin, Wei-Ming Chen, Han Cai, Chuang Gan, and Song Han. MCUNetV2 : Memory-Efficient Patch-based Inference for Tiny Deep Learning. In *NeurIPS 2021 (d'après 2025-el-zeinaty-designing-object-detection-models-tinyml, Table 8 ; non imprimé dans la version arXiv)*, 2021. URL https://arxiv.org/abs/2110.15352. arXiv :2110.15352.

Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. On-Device Training Under 256KB Memory. arXiv preprint, 2022. URL https://arxiv.org/abs/2206.15472. arXiv :2206.15472. Preprint non relu par les pairs.

Xiangzhong Luo, Di Liu, Hao Kong, Shuo Huai, Hui Chen, Guochu Xiong, and Weichen Liu. Efficient Deep Learning Infrastructures for Embedded Computing Systems : A Comprehensive Survey and Future Envision. *ACM Transactions on Embedded Computing Systems*, 2024. doi : 10.1145/3701728. URL https://arxiv.org/abs/2411.01431. arXiv :2411.01431.

Sandro Costa Magalhães, Filipe Neves Santos, Pedro Machado, António Paulo Moreira, and Jorge Dias. Benchmarking Edge Computing Devices for Grape Bunches and Trunks Detection using Accelerated Object Detection Single Shot MultiBox Deep Learning Models. *Engineering Applications of Artificial Intelligence 117, 105604 (2022)*, 2022. doi : 10.1016/j.engappai.2022.105604. URL https://arxiv.org/abs/2211.11647. arXiv :2211.11647.

Vaibhav Malviya, Indrani Mukherjee, and Siddharth Tallur. Edge-Compatible Convolutional Autoencoder Implemented on FPGA for Anomaly Detection in Vibration Condition-Based Monitoring. *IEEE Sensors Letters 6(4), 1-4*, 2022. doi : 10.1109/LSENS.2022.3159972. URL https://doi.org/10.1109/LSENS.2022.3159972.

Nitheesh Kumar Manjunath, Aidin Shiri, Morteza Hosseini, Bharat Prakash, Nicholas R. Waytowich, and Tinoosh Mohsenin. An Energy Efficient EdgeAI Autoencoder Accelerator for Reinforcement Learning. *IEEE Open Journal of Circuits and Systems 2, 182-195*, 2021. doi : 10.1109/OJCAS.2020.3043737. URL https://doi.org/10.1109/OJCAS.2020.3043737.

Marc Masana, Xialei Liu, Bartlomiej Twardowski, Mikel Menta, Andrew D. Bagdanov, and Joost van de Weijer. Class-incremental learning : survey and performance evaluation on image classification. arXiv preprint, 2020. URL https://arxiv.org/abs/2010.15277. arXiv :2010.15277. Preprint non relu par les pairs.

Alaa Mazouz, Sumanta Chaudhuri, Marco Cagnanzzo, Mihai Mitrea, Enzo Tartaglione, and Attilio Fiandrotti. Lightweight Embedded FPGA Deployment of Learned Image Compression with Knowledge Distillation and Hybrid Quantization. arXiv preprint, 2025. URL https://arxiv.org/abs/2503.04832. arXiv :2503.04832. Preprint non relu par les pairs.

Payal Mittal. A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artificial Intelligence Review 57(9), 242*, 2024. doi : 10.1007/s10462-024-10877-1. URL https://doi.org/10.1007/s10462-024-10877-1.

Alexander Montgomerie-Corcoran, Petros Toupas, Zhewen Yu, and Christos-Savvas Bouganis. SATAY : A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices. In *2023 International Conference on Field Programmable Technology (ICFPT) 179-187*, 2023. doi : 10.1109/ICFPT59805.2023.00025. URL https://doi.org/10.1109/ICFPT59805.2023.00025.

Julian Moosmann, Marco Giordano, Christian Vogt, and Michele Magno. TinyissimoYOLO : A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers. In *IEEE AICAS 2023 (d'après le DOI 10.1109/AICAS57966.2023.10168657)*, 2023. doi : 10.1109/AICAS57966.2023.10168657. URL https://arxiv.org/abs/2306.00001. arXiv :2306.00001.

Nina Narodytska, Shiva Prasad Kasiviswanathan, Leonid Ryzhyk, Mooly Sagiv, and Toby Walsh. Verifying Properties of Binarized Deep Neural Networks. arXiv preprint, 2017. URL https://arxiv.org/abs/1709.06662. arXiv :1709.06662. Preprint non relu par les pairs.

Alessandro Pappalardo, Yaman Umuroglu, Michaela Blott, Jovan Mitrevski, Ben Hawks, Nhan Tran, Vladimir Loncar, Sioni Summers, Hendrik Borras, Jules Muhizi, Matthew Trahms, Shih-Chieh Hsu, Scott Hauck, and Javier Duarte. QONNX : Representing Arbitrary-Precision Quantized Neural Networks. arXiv preprint, 2022. URL https://arxiv.org/abs/2206.07527. arXiv :2206.07527. Preprint non relu par les pairs.

Francesco Pasti, Marina Ceccon, Davide Dalle Pezze, Francesco Paissan, Elisabetta Farella, Gian Antonio Susto, and Nicola Bellotto. Latent Distillation for Continual Object Detection at the Edge. arXiv preprint, 2024. URL https://arxiv.org/abs/2409.01872. arXiv :2409.01872. Preprint non relu par les pairs.

Lucian Petrica, Tobias Alonso, Mairin Kroes, Nicholas Fraser, Sorin Cotofana, and Michaela Blott. Memory-Efficient Dataflow Inference for Deep CNNs on FPGA. arXiv preprint, 2020. URL https://arxiv.org/abs/2011.07317. arXiv :2011.07317. Preprint non relu par les pairs.

Duvindu Piyasena. *FPGA Acceleration of Continual Learning at the Edge*. PhD thesis, PhD thesis, Nanyang Technological University, 2021. URL https://hdl.handle.net/10356/153778.

Duvindu Piyasena, Siew-Kei Lam, and Meiqing Wu. Accelerating Continual Learning on Edge FPGA. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL) 294-300*, 2021. doi : 10.1109/FPL53798.2021.00059. URL https://doi.org/10.1109/FPL53798.2021.00059.

Zhiqiang Que, Minghao Zhang, Hongxiang Fan, He Li, Ce Guo, and Wayne Luk. Low Latency Variational Autoencoder on FPGAs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems 14(2), 323-333*, 2024. doi : 10.1109/JETCAS.2024.3389660. URL https://doi.org/10.1109/JETCAS.2024.3389660.

Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML Platform for On-Device Continual Learning with Quantized Latent Replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems 11.4 (2021), 789-802*, 2021. doi : 10.1109/JETCAS.2021.3121554. URL https://arxiv.org/abs/2110.10486. arXiv :2110.10486.

Paolo Rech. Artificial Neural Networks for Space and Safety-Critical Applications : Reliability Issues and Potential Solutions. *IEEE Transactions on Nuclear Science 71(4), 377-404*, 2024. doi : 10.1109/TNS.2024.3349956. URL https://doi.org/10.1109/TNS.2024.3349956.

Eugenio Ressa, Alberto Marchisio, Maurizio Martina, Guido Masera, and Muhammad Shafique. TinyCL : An Efficient Hardware Architecture for Continual Learning on Autonomous Systems. arXiv preprint, 2024. URL https://arxiv.org/abs/2402.09780. arXiv :2402.09780. Preprint non relu par les pairs.

Lucas Rey, Ana M Bernardos, Andrzej D Dobrzycki, David Carramiñana, Luca Bergesio, Juan A Besada, and José Ramón Casar. A Performance Analysis of You Only Look Once Models for Deployment on Constrained Computational Edge Devices in Drone Applications. *Electronics 2025, 14(3), 638*, 2025. doi : 10.3390/electronics14030638. URL https://arxiv.org/abs/2502.15737. arXiv :2502.15737.

Annachiara Ruospo, Ernesto Sanchez, Lucas Matana Luza, Luigi Dilillo, Marcello Traiola, and Alberto Bosio. A Survey on Deep Learning Resilience Assessment Methodologies. *Computer 56(2), 57-66*, 2023. doi : 10.1109/MC.2022.3217841. URL https://doi.org/10.1109/MC.2022.3217841.

Marcus Rüb, Philipp Tuchel, Axel Sikora, and Daniel Mueller-Gritschneder. A Continual and Incremental Learning Approach for TinyML On-device Training Using Dataset Distillation and Model Size Adaption. 2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS), 2024. URL https://arxiv.org/abs/2409.07114. arXiv :2409.07114.

Safa Mohammed Sali, Mahmoud Meribout, and Ashiyana Abdul Majeed. Real Time FPGA Based CNNs for Detection, Classification, and Tracking in Autonomous Systems : State of the Art Designs and Optimizations. arXiv preprint, 2025. URL https://arxiv.org/abs/2509.04153. arXiv :2509.04153. Preprint non relu par les pairs.

Michael Schlosser, Daniel König, and Michael Teutsch. A Framework for Benchmarking Real-Time Embedded Object Detection. In *In : Andres, B., Bernard, F., Cremers, D., Frintrop, S., Goldlücke, B., Ihrke, I. (eds) Pattern Recognition. DAGM GCPR 2022. Lecture Notes in Computer Science, vol 13485. Springer, Cham*, 2023. doi : 10.1007/978-3-031-16788-1_33. URL https://arxiv.org/abs/2304.11580.

Jan-Frederik Schulte, Benjamin Ramhorst, Chang Sun, Jovan Mitrevski, Nicolò Ghielmetti, Enrico Lupi, Dimitrios Danopoulos, Vladimir Loncar, Javier Duarte, David Burnette, Lauri Laatu, Stylianos Tzelepis, Konstantinos Axiotis, Quentin Berthet, Haoyan Wang, Paul White, Suleyman Demirsoy, Marco Colombo, Thea Aarrestad, Sioni Summers, Maurizio Pierini, Giuseppe Di Guglielmo, Jennifer Ngadiuba, Javier Campos, Ben Hawks, Abhijith Gandrakota, Farah Fahim, Nhan Tran, George Constantinides, Zhiqiang Que, Wayne Luk, Alexander Tapper, Duc Hoang, Noah Paladino, Philip Harris, Bo-Cheng Lai, Manuel Valentin, Ryan Forelli, Seda Ogrenci, Lino Gerlach, Rian Flynn, Mia Liu, Daniel Diaz, Elham Khoda, Melissa Quinnan, Russell Solares, Santosh Parajuli, Mark Neubauer, Christian Herwig, Ho Fung Tsoi, Dylan Rankin, Shih-Chieh

Hsu, and Scott Hauck. hls4ml : A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware. arXiv preprint, 2025. URL https://arxiv.org/abs/2512.01463. arXiv :2512.01463. Preprint non relu par les pairs.

Gesina Schwalbe and Martin Schels. A Survey on Methods for the Safety Assurance of Machine Learning Based Systems. In *10th European Congress on Embedded Real Time Software and Systems (ERTS 2020)*, 2020.

Shuai Shao, Zijian Zhao, Boxun Li, Tete Xiao, Gang Yu, Xiangyu Zhang, and Jian Sun. CrowdHuman : A Benchmark for Detecting Human in a Crowd. arXiv preprint, 2018. URL https://arxiv.org/abs/1805.00123. arXiv :1805.00123. Preprint non relu par les pairs.

Md. Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I. Morshed. Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices : A Review. *Proceedings of the IEEE 111(1), 42-91*, 2023. doi : 10.1109/JPROC.2022.3226481. URL https://doi.org/10.1109/JPROC.2022.3226481.

Heming Sun, Qingyang Yi, Fangzheng Lin, Lu Yu, Jiro Katto, and Masahiro Fujita. F-LIC : FPGA-based Learned Image Compression with a Fine-grained Pipeline. In *2022 IEEE Asian Solid-State Circuits Conference (A-SSCC) 1-3*, 2022. doi : 10.1109/A-SSCC56115.2022.9980666. URL https://doi.org/10.1109/A-SSCC56115.2022.9980666.

Yue Tang, Xinyi Zhang, Peipei Zhou, and Jingtong Hu. EF-Train : Enable Efficient On-device CNN Training on FPGA Through Data Reshaping for Online Adaptation or Personalization. arXiv preprint, 2022. URL https://arxiv.org/abs/2202.10935. arXiv :2202.10935. Preprint non relu par les pairs.

Petros Toupas, Zhewen Yu, Christos-Savvas Bouganis, and Dimitrios Tzovaras. SMOF : Streaming Modern CNNs on FPGAs with Smart Off-Chip Eviction. arXiv preprint, 2024. URL https://arxiv.org/abs/2403.18921. arXiv :2403.18921. Preprint non relu par les pairs.

Mineto Tsukada, Masaaki Kondo, and Hiroki Matsutani. A Neural Network-Based On-device Learning Anomaly Detector for Edge Devices. *IEEE Transactions on Computers*, 2020. doi : 10.1109/TC.2020.2973631. URL https://doi.org/10.1109/TC.2020.2973631.

Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. FINN : A Framework for Fast, Scalable Binarized Neural Network Inference. In *FPGA '17 : ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016. doi : 10.1145/3020078.3021744. URL https://arxiv.org/abs/1612.07119. arXiv :1612.07119.

Stylianos I Venieris and Christos-Savvas Bouganis. fpgaConvNet : A Toolflow for Mapping Diverse Convolutional Neural Networks on Embedded FPGAs. In *NIPS 2017 Workshop on Machine Learning on the Phone and other Consumer Devices*, 2017. URL https://arxiv.org/abs/1711.08740. arXiv :1711.08740.

Stylianos I. Venieris, Alexandros Kouris, and Christos-Savvas Bouganis. Toolflows for Mapping Convolutional Neural Networks on FPGAs : A Survey and Future Directions. *ACM Comput. Surv. 51(3), 56*, 2018. doi : 10.1145/3186332. URL https://doi.org/10.1145/3186332.

Yoga Esa Wibowo, Cristian Cioflan, Thorir Mar Ingolfsson, Michael Hersche, Leo Zhao, Abbas Rahimi, and Luca Benini. 12 mJ per Class On-Device Online Few-Shot Class-Incremental Learning. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024. doi : 10.23919/DATE58400.2024.10546598. URL https://arxiv.org/abs/2403.07851. arXiv :2403.07851.

Xiaowei Xu, Xinyi Zhang, Bei Yu, Xiaobo Sharon Hu, Christopher Rowen, Jingtong Hu, and Yiyu Shi. DAC-SDC Low Power Object Detection Challenge for UAV Applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence 43(2), 392-403*, 2021. doi : 10.1109/TPAMI.2019.2932429. URL https://doi.org/10.1109/TPAMI.2019.2932429.

Zhihong Yan, Bingqian Zhang, and Dong Wang. An FPGA-Based YOLOv5 Accelerator for Real-Time Industrial Vision Applications. *Micromachines 15(9), 1164*, 2024. doi : 10.3390/mi15091164. URL https://doi.org/10.3390/mi15091164.

Mang Ye, Jianbing Shen, Gaojie Lin, Tao Xiang, Ling Shao, and Steven C H Hoi. Deep Learning for Person Re-identification : A Survey and Outlook. arXiv preprint, 2020. URL https://arxiv.org/abs/2001.04193. arXiv :2001.04193. Preprint non relu par les pairs.

Alexander Yue, Haoyi Jia, and Julia Gonski. Variational Autoencoders for At-Source Data Reduction and Anomaly Detection in High Energy Particle Detectors. *2025 Mach. Learn. : Sci. Technol. 6 035017*, 2024. doi : 10.1088/2632-2153/adf0c0. URL https://arxiv.org/abs/2411.01118.

Jiaqi Zhai, Bin Li, Shunsen Lv, and Qinglei Zhou. FPGA-Based Vehicle Detection and Tracking Accelerator. *Sensors 23(4), 2208*, 2023. doi : 10.3390/s23042208. URL https://doi.org/10.3390/s23042208.

Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays 161–170*, 2015. doi : 10.1145/2684746.2689060. URL https://doi.org/10.1145/2684746.2689060.

Dezheng Zhang, Aibin Wang, Ruchan Mo, and Dong Wang. End-to-end acceleration of the YOLO object detection framework on FPGA-only devices. *Neural Computing and Applications 36(3), 1067-1089*, 2024. doi : 10.1007/s00521-023-09078-8. URL https://doi.org/10.1007/s00521-023-09078-8.

Xiaofan Zhang, Haoming Lu, Cong Hao, Jiachen Li, Bowen Cheng, Yuhong Li, Kyle Rupnow, Jinjun Xiong, Thomas Huang, Honghui Shi, Wen-Mei Hwu, and Deming Chen. SkyNet : a Hardware-Efficient Method for Object Detection and Tracking on Embedded Systems. arXiv preprint, 2019. URL https://arxiv.org/abs/1909.09709. arXiv :1909.09709. Preprint non relu par les pairs.